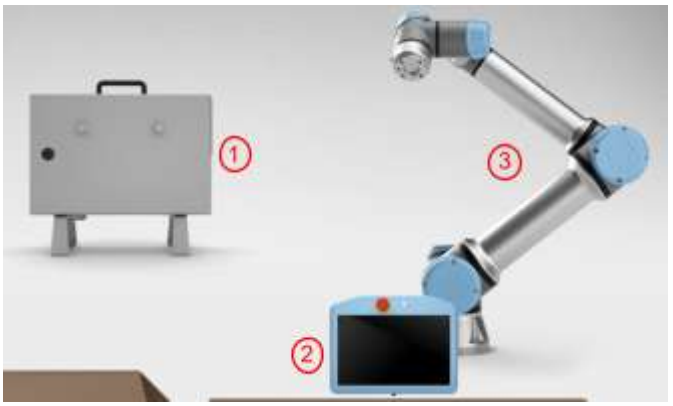
SOMMAIRE Général :

Exercise 0 : Learning how to use the software..... 2
I. Presentation and initialisation of the system..... 2
II. Move menu.....5
III. Program Menu.....9
Exercise 1 : Creation of a new tool in the robot system and movement types..... 11
I. Definition of a new tool..... 11
II. Set up a working plan.....17
III. Start programming.....18
Exercise 2 : Pick and Place objects..... 25
I. Introduction to the clamp.....25
II. First Pick and Place programs..... 30
III. Improving the placement.....40
Exercise 3 : Palletizing.....41
I. Presentation of the Palletizing function..... 41
II. First program : Depalletizing..... 43
III. Placing with palletizing..... 49
Exercise 4 : Presence Detection by Material..... 52
I. No Grip (to close in the empty space).....52
II. Free space (to go down without any resistance).....56
Exercise 5 : Communication between 2 cobots..... 61
I. Establishing Communication.....61
II. One-way Communication.....62
III. Both-way Communication..... 69
ANNEXE : Modbus addresses signification:..... 74
Exercise 6 : Practical example : Simulation of a production line.....76
Concept of line production :.....76

Exercise 0 : Learning how to use the software

I. Presentation and initialisation of the system

A. Starting the system



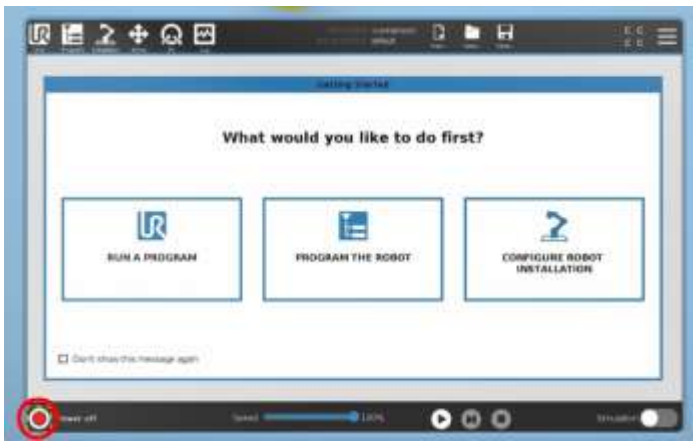
First of all, the robot is constituted of three parts :

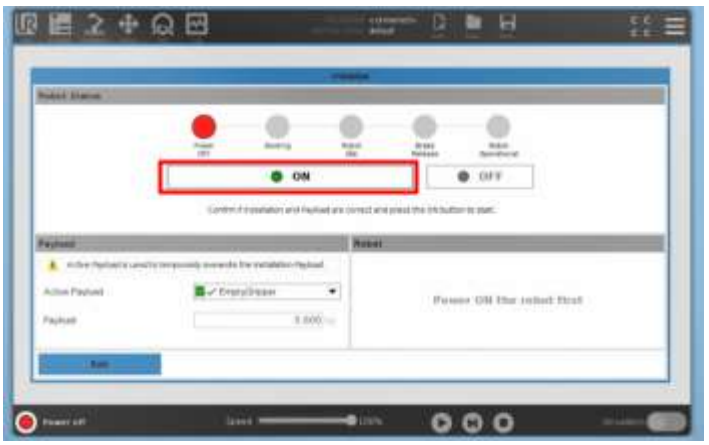
- The control Box (1)
- The teach pendant (2)
- The robot arm (6 axes each 360° rotation) (3)



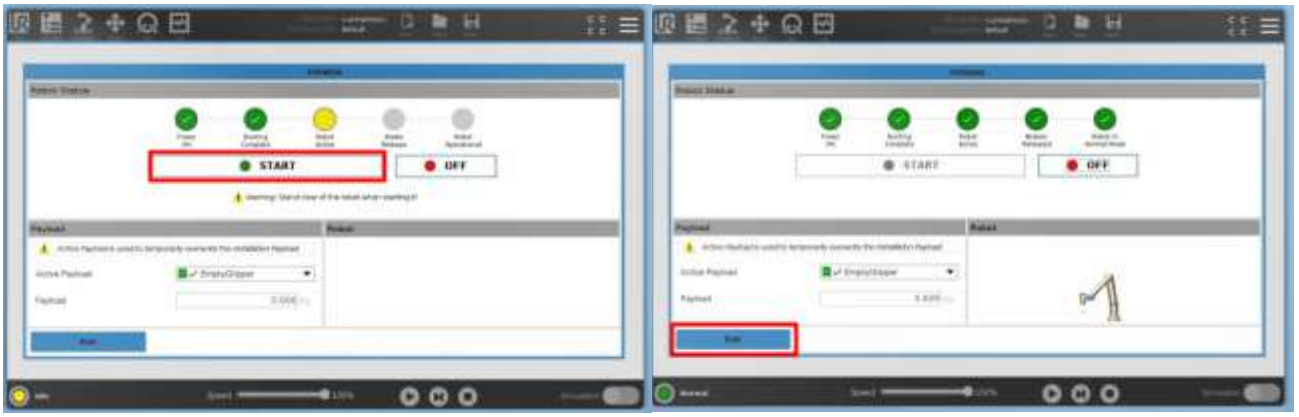
So now, we will only focus on the robot and teach pendant.
The teach pendant is used for controlling the robot directly, with this device you can program the robot to carry out specific tasks and also control signals from peripheral devices such as grippers and sensors.
For the beginning you have to turn it on by pushing on the power button next to the emergency stop (red button).

- To initiate the robot, click on the red circle icon in the lower left corner and click the "**on**" button to enable power for the joints :





- Now click the “**start**” to release the mechanical brakes, the robot is now ready to operate.



If you encounter any problems while using the robot, press the emergency stop button. To restart the robot, turn this button clockwise and pull it toward you to unlock it.

- Click the “exit” button to return to the start screen, lets have a look at the user interface when programming the robot :



On the top left you can see 5 different menus :

- Run
- **Program**
- **Installation**
- **Move**
- I/O (in and out)
- Log

The main menus you will use are :

- “**Installation**”, which allows you to define the robot configuration
- “**Program**”, that is used to create a sequence of automated actions
- “**Move**” in which you can move the robot as you want

The “Run” menu will be just used to run the program, and the “I/O” and “Log” menus will be used for visualisation of the robot characteristics and debugging.

B. Verify the robot installation

We will first make sure the robot options are set for a robot without any tool, as your robot should be.

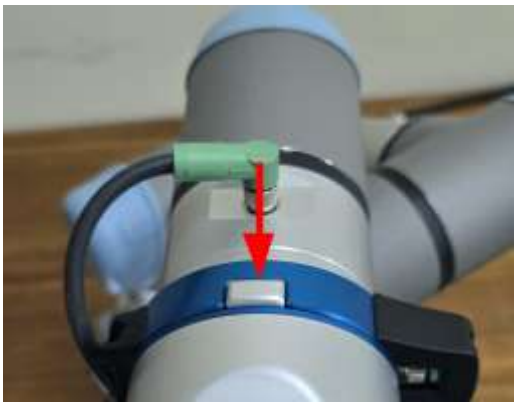




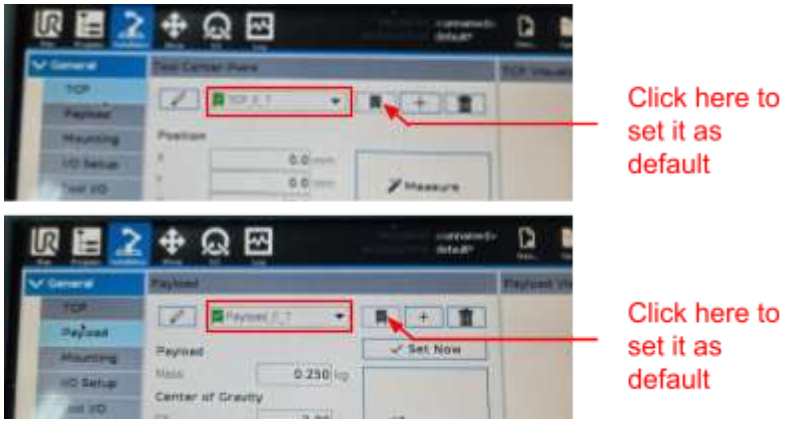




If a tool is mounted on the robot, just detach the tool by pressing the aluminium button at the base of the tool :



- Go to the “**installation”** menu, in the “General” section and in “TCP” : verify that the TCP is set on “TCP_E_T”. If it is not, select it in the drop down menu and set it as default by clicking on the banner next to the drop down menu.
- In the same way, go to “Payload” just below “TCP” and select “Payload_E_T”.



The “Installation” menu presentation will be detailed in the next exercises.
Next we will learn how to move the robot as we want.

II. Move menu

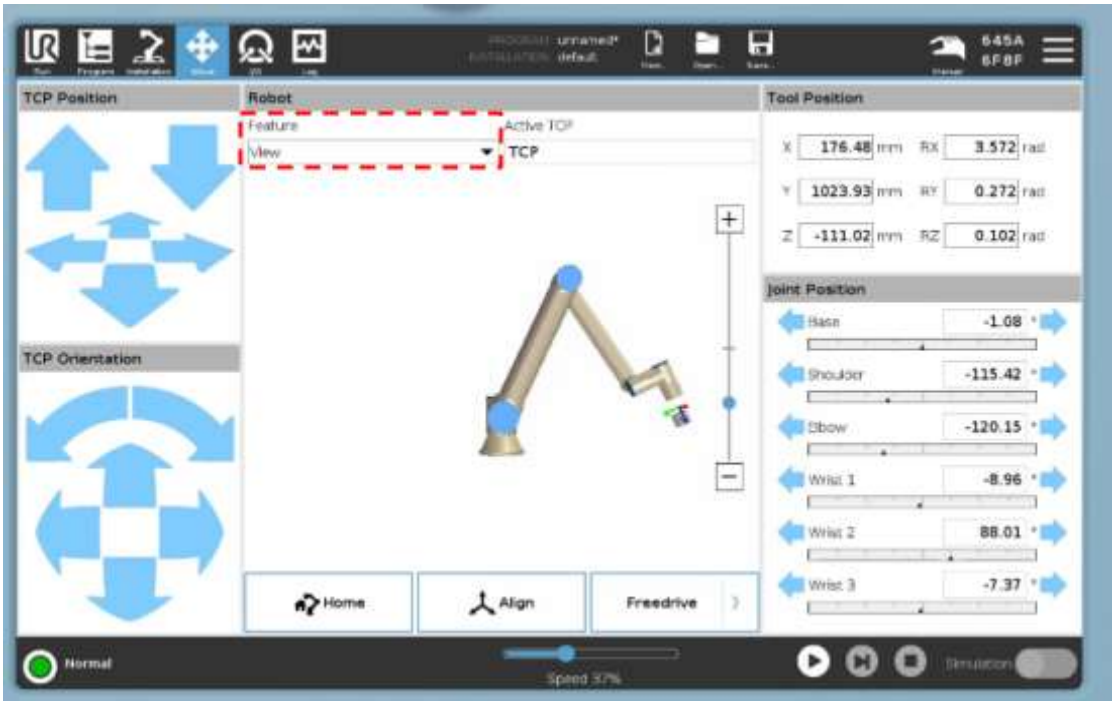
Click on the “**Move**” menu. There are two main different ways to move the robot : with the arrows on the left (TCP Position/Orientation) to move the TCP, and with the sliders on the right (Joint Position) to move each joint independently.
All the options to move the robot are dependent of the general speed, that you can change with the slider at the bottom of the teach pendant screen :



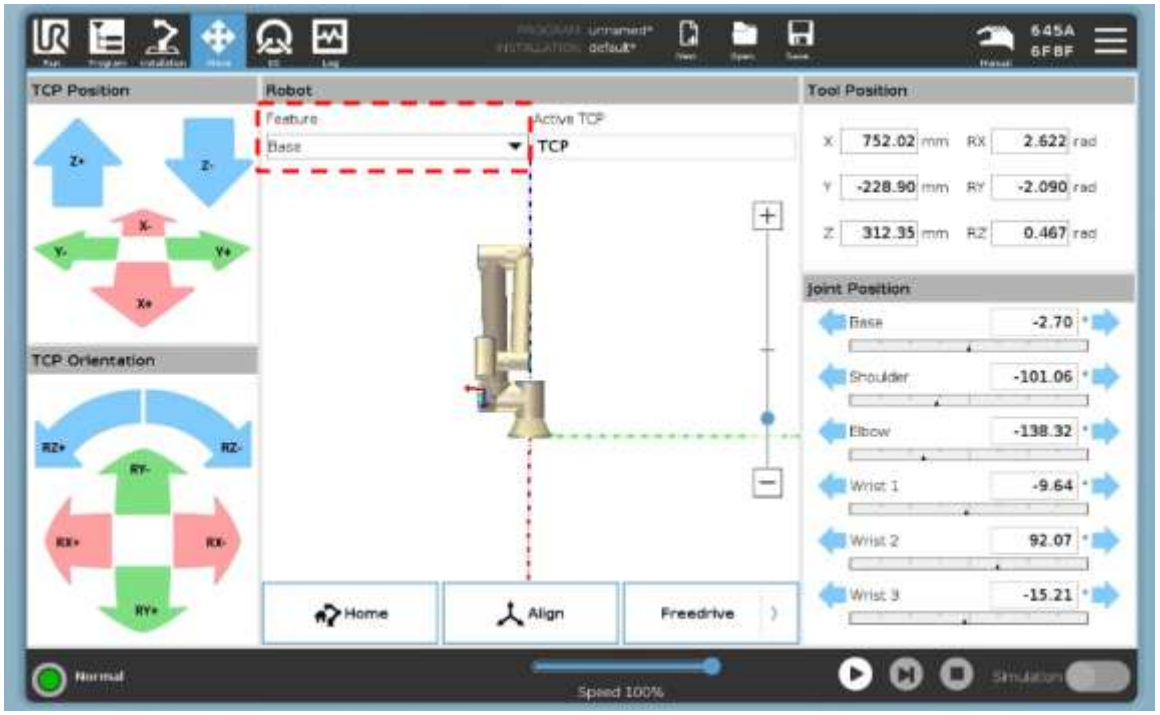
A. Move with TCP Position and Orientation

The way the robot will move using the arrows on the left side of the screen depends on which reference you use. This reference can be changed in the “Feature” dropdown menu :

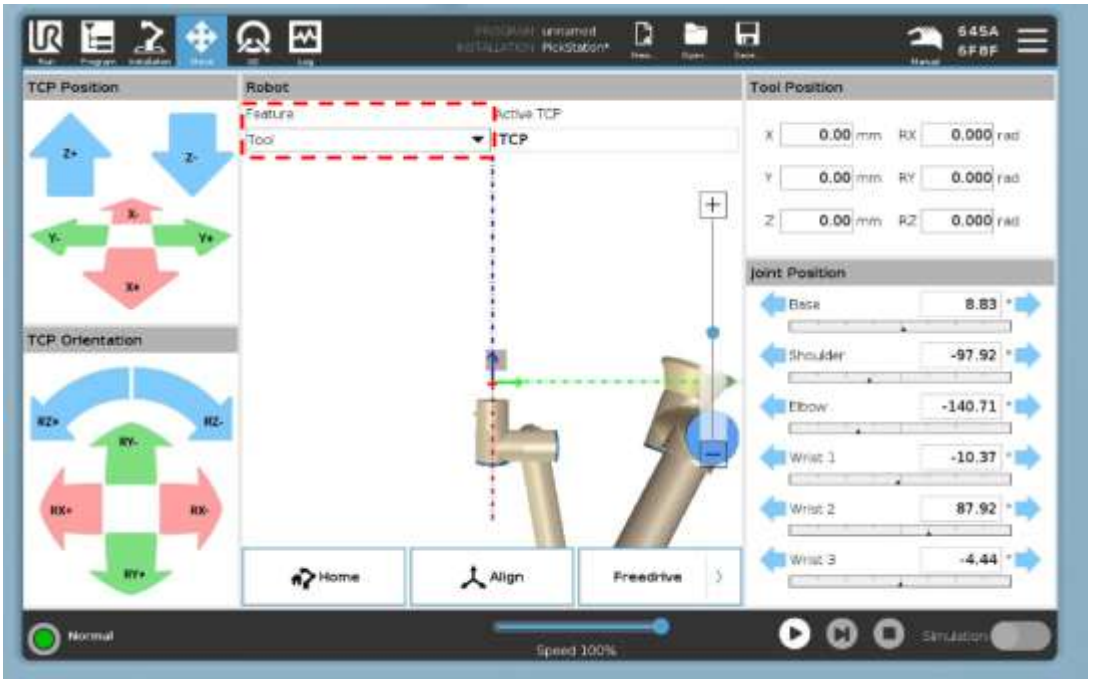
- The first one is “**View**” :
  It means, you are controlling the robot TCP (*Tool Reference Point*) with the orientation of the view you have on your screen, so it can be easily changed by rotating the view with your hand on the tach pendant.
  The key is to orient the view with your perspective, so you can easily move the head of the robot in the direction you want.



- The second one is “**Base**” :
  It means, you move the TCP (tool reference point) along the XYZ axes based on the “Base” reference position.

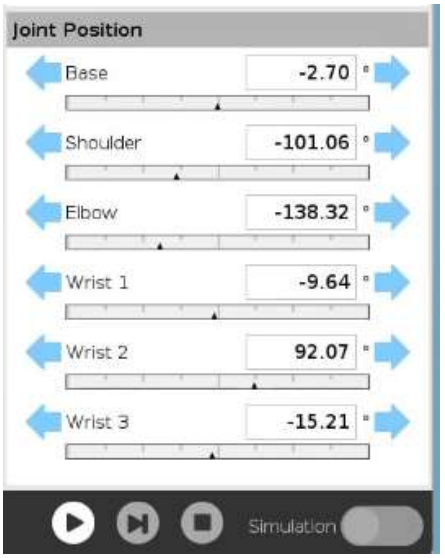


- The Third one is “**Tool**” :
  It means, you move the TCP (tool reference point) along the XYZ axes based on the “Tool” reference position.



B. Move with Joint Position

In the “Joint Position” window on the right side of the screen, you can move axes by axes the robot (Base/Shoulder/Elbow/Wrists) with the arrows on each side of each sliders :

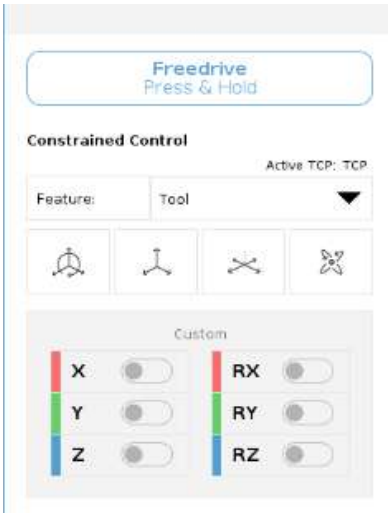


C. Other features

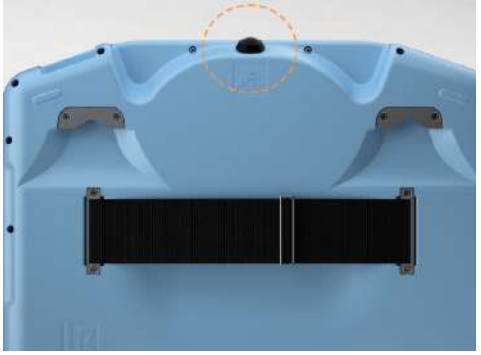
We also have an “**Align**” option 

, it can be used to align the TCP with one axe like Z for example, to align the tool for going down.

On the same place on the screen you can see the “**Freedrive**”, it means you can move the robot with your hand freely to go to the position you need.
In this panel you have the ability to restrict movement along specific linear axes or rotations, which can be really handy. By default, all axes and rotations are free to move.
Imagine you want to move the robot BUT you want to keep its orientation fixed.



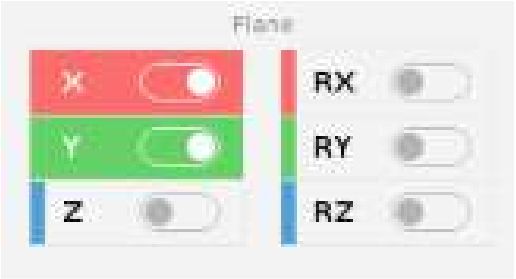
As an alternative to the freedrive with all translations and rotations free, you can also press the freedrive button at the back of the teach pendant. You will notice it is easier to move the robot in this way, by pressing the freedrive button with your hand that holds the teach pendant, and by moving the robot with the other hand.

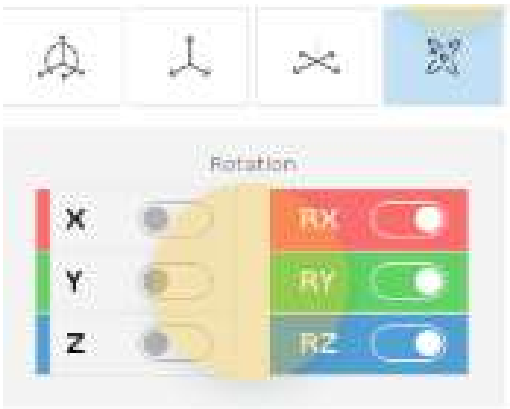


- You can switch to **translation mode** : 

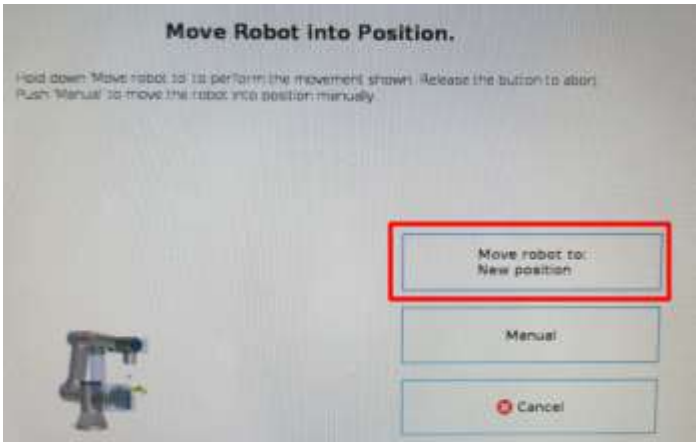
so you are free to move along the XYZ axes.

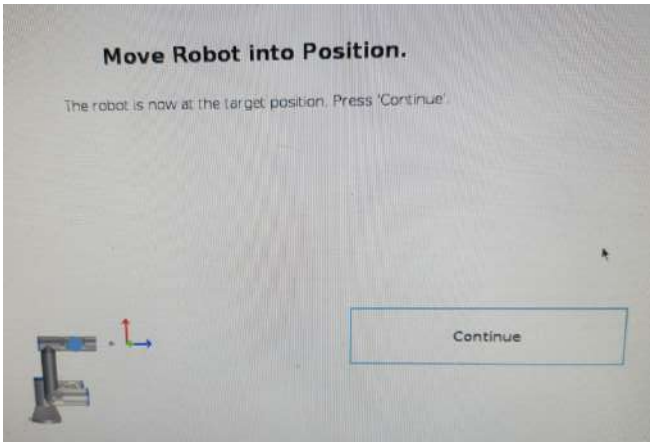
- When you're welding or gluing something you can switch to **plane mode** :

it restricts all the rotations and the linear Z axe movement, it lets you keep its orientation.


- You also have the "**rotation mode**" where all translations are kept in place and the TCP stays in the same position, it's really useful for quality inspection or welding,


And finally, you got the "**Home**" button, and this is for the security position, you just have to maintain the Button, and then the robot will automatically go back to this safe position,at the end of the movement click on continue.

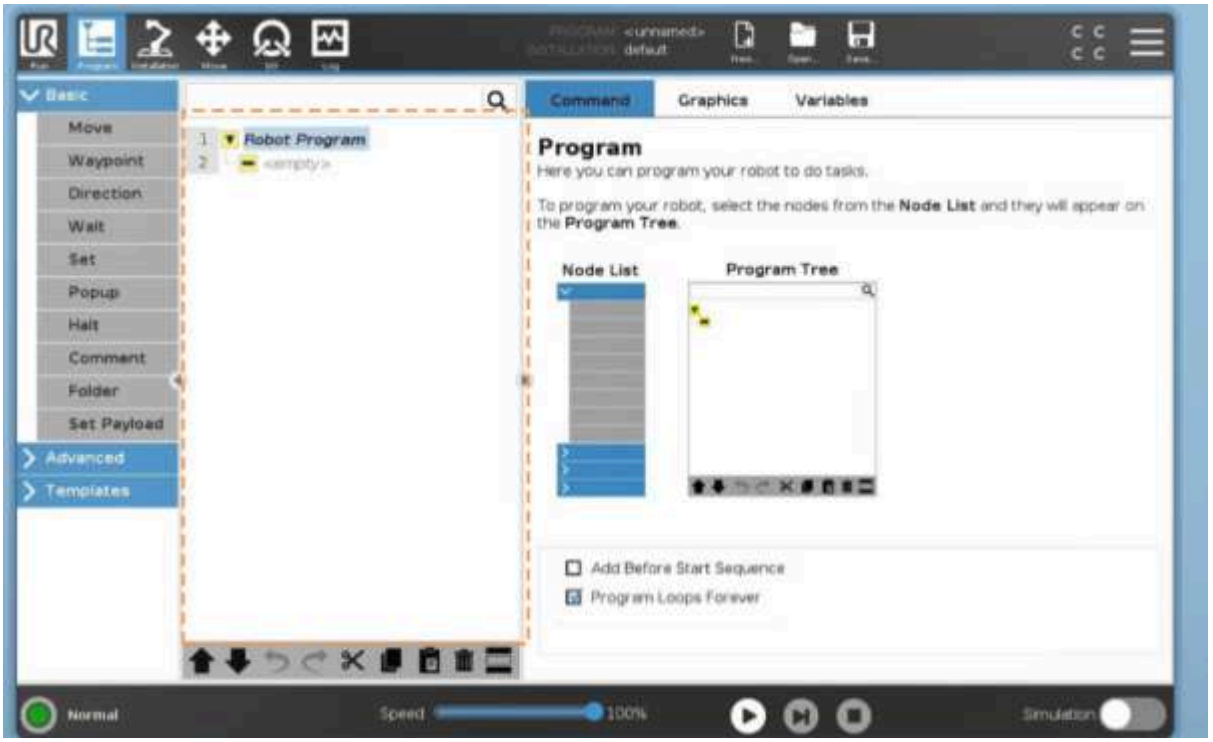




III. Program Menu

- Now click on the “**program**” menu

In the middle pane, the robot program is displayed - this is called the program tree



On the right of the screen, you will see some information for each node you will select. For the “Robot Program” node that you have at the start of a program, you can define with the checkboxes at the bottom if you want robot actions before the main sequence or if you want the main sequence loop forever.

At the bottom of your program tree, you will find some useful features :

 : move the selected node up or down

 : undo or redo the last action

 : cut or copy the selected node

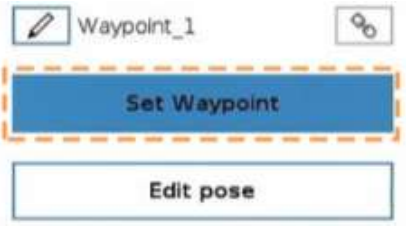
 : paste the last memorised node

 : delete the selected node

 : disable the selected node. This one won’t be active but will be always visible in the program tree in gray.

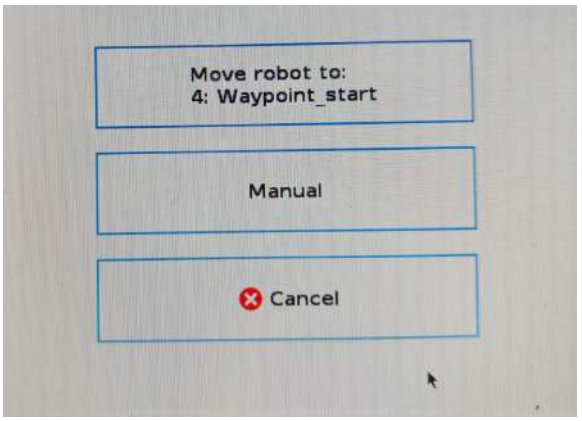
We will try to program the robot, and first to make it move :

- Click on the "**Robot Program**" node in the program tree
- Go in the "**Basic**" menu on the left
- Click on "**Move**"
- On the program click on "***Waypoint_1"***
- Click on "**Set Waypoint**" on the right part of the screen
- It makes you go to the "**Move**" menu. Move the robot to a position that you want the robot to have at the start of the program.



- Click "**OK**" 

to save the position as *"**Waypoint_1**"*
- You have now saved the robot's position as waypoint 1. You can also rename it if you want to something more descriptive (like "Waypoint_start" for example), by clicking on the pen on the right of the waypoint name (see the image above).
- In the same way, define another waypoint (waypoint 2) by clicking on "**Waypoint**" on the left menu when the node of the first waypoint is selected. Define its position, that has to be different but not too distant than the position of the first waypoint.
- Run the program by clicking on the start button in the bottom right corner  . A pop-up will appear, you must choose the play button labeled *"**Play from beginning Robot Program**"* to move the robot to the beginning of the program. This screen will appear :



To move the robot to the right position, press and hold the button which says *"**Move robot to 4: Waypoint_start**"*. Then press *"**Play from beginning Robot Program**"* in the next screen to start the program.

If you select the checkbox to make your program loop in the "Robot Program" node, you will see your robot alternate between the two waypoints you defined in the program.

Exercise 1 : Creation of a new tool in the robot system and movement types

Objectives :

In this exercise you are gonna learn the basics to code with a Universal Robot and to use a new tool on these cobots.

This exercise divides itself in three different parts :

1. Define a new tool ;
2. Create a plan to work in it ;
3. Compare the different types of movement for a UR cobot.

I. Definition of a new tool

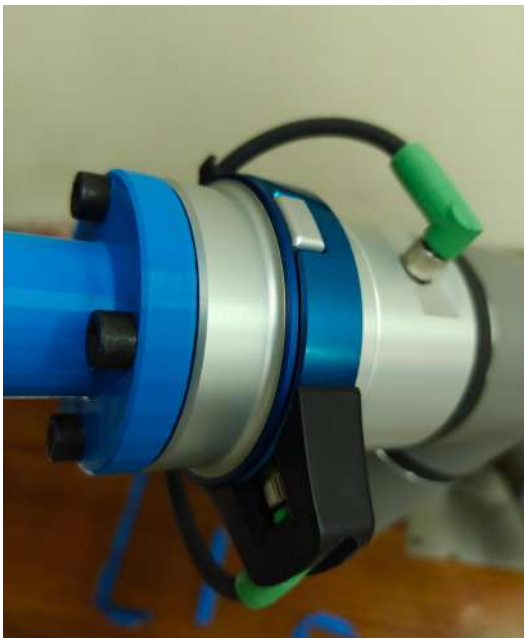
1. Turn on the robot and start it like in the first exercise (grey button on the teach pendant then the power on button on the screen).

2. You will make the robot use a pen, by using the tool in the photo below. After making sure the pen is secured on the tool provided, you can mount it on the robot arm.



To mount it you must first grab the bar on the robot arm and then clip the tool on it. See the following photos for more details :





The first thing that needs to be done with a new tool is to teach the robot how to use it.

For this, the robot must know three major things :

- the **Tool Center Point** (TCP) which will be in this the point of the pen
- the center of gravity of the tool
- the **payloads**, which is the mass of the tool

1. Implementation of the TCP

To create the TCP of this new tool, we are gonna use two different methods :

- a more physical method
- a more numerical method

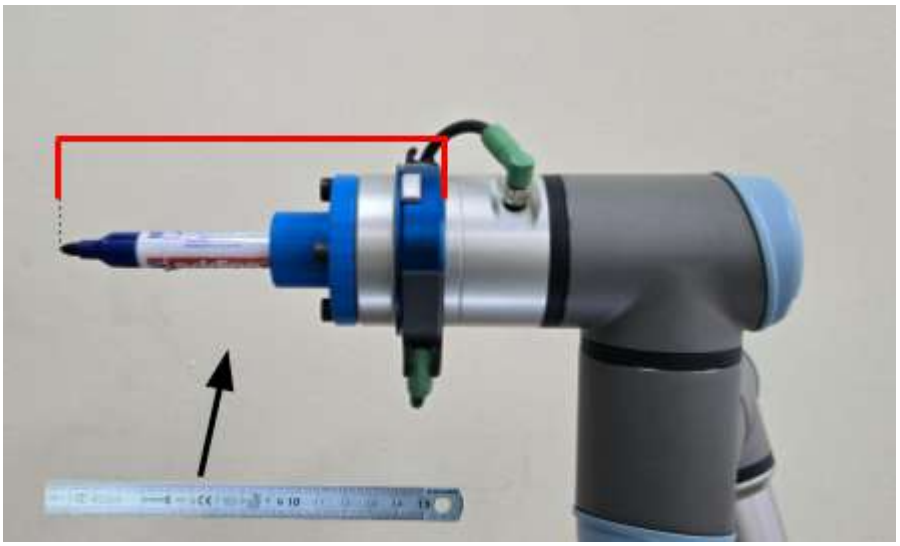
a. First method : physical measure

This first method is really dependent on how the tool is looking, it can be much quicker than the second but it's less precise.

This method is mainly used on a simple looking tool like the one we use in this exercise because the determination of the TCP is done by simply measuring it on the real tool or using the technical documentation we have.

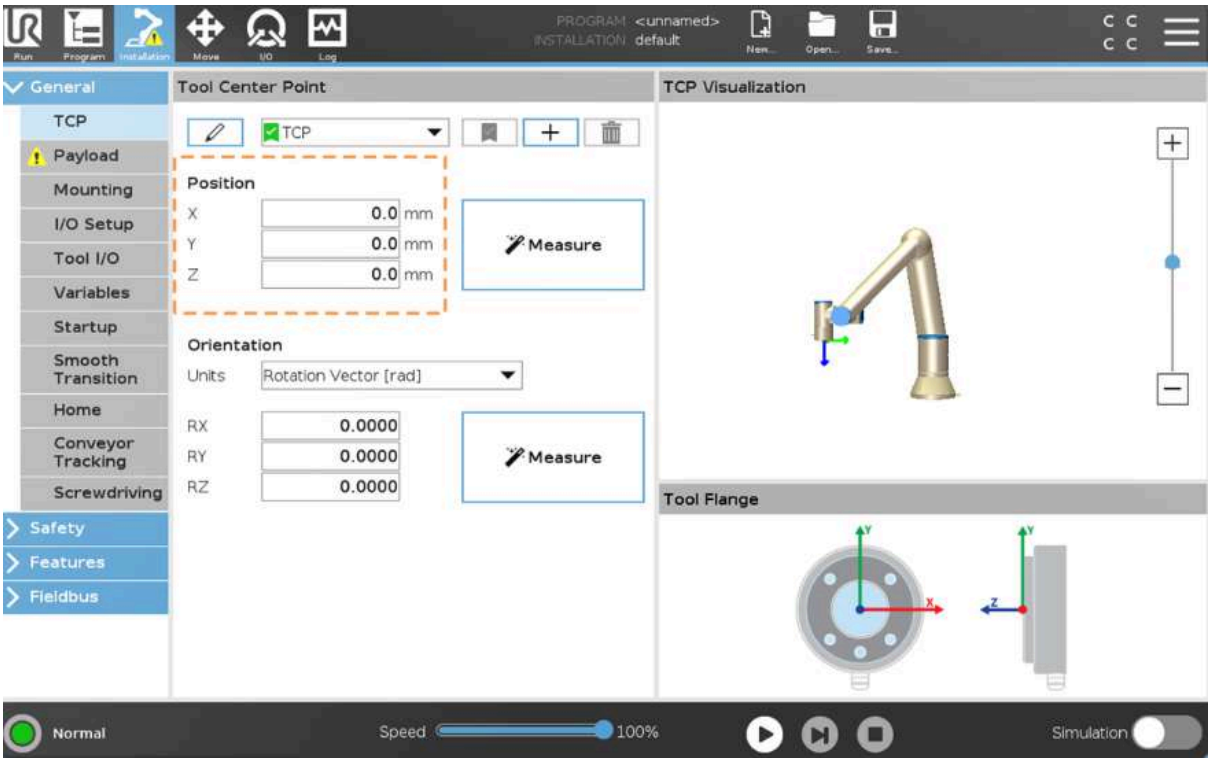
In this exercise we are simply gonna measure the TCP with a ruler.

As we said, here the TCP is at the end of the pen, so we need to measure from the connection between the robot and the Quick Changer (metallic blue part at the end of the robot) to the tip of the pen. See the following photo for more details :



When you have this number go in the installation window, in the *"**TCP**"* menu and enter the value you found in these boxes using the tool flange represented on the right to know which box to fill.

Keep in mind that the pen goes straight out of the middle of the robot arm.



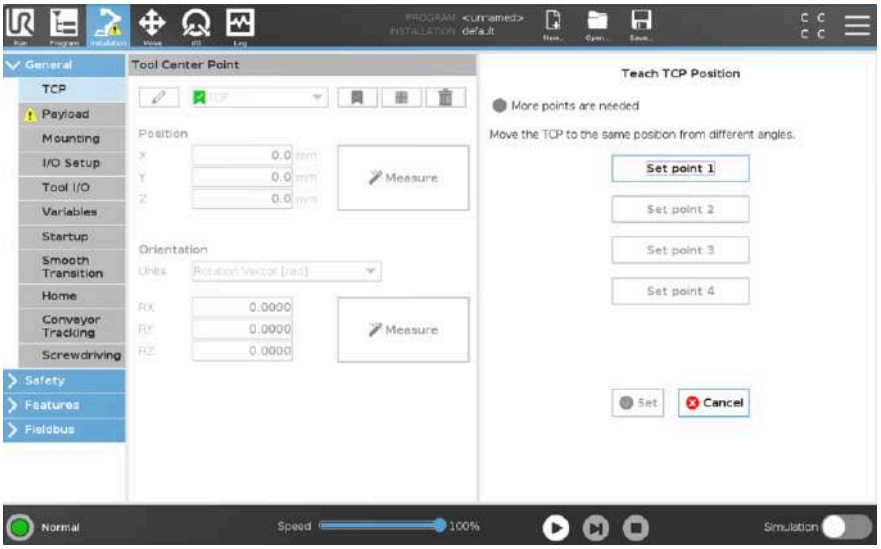
When this value is entered you can say that you correctly defined the TCP.

b. Second method : TCP position wizard

If the TCP is hard to measure or you want to be more precise, you can use the teach pendant to let the robot calculate itself the TCP.

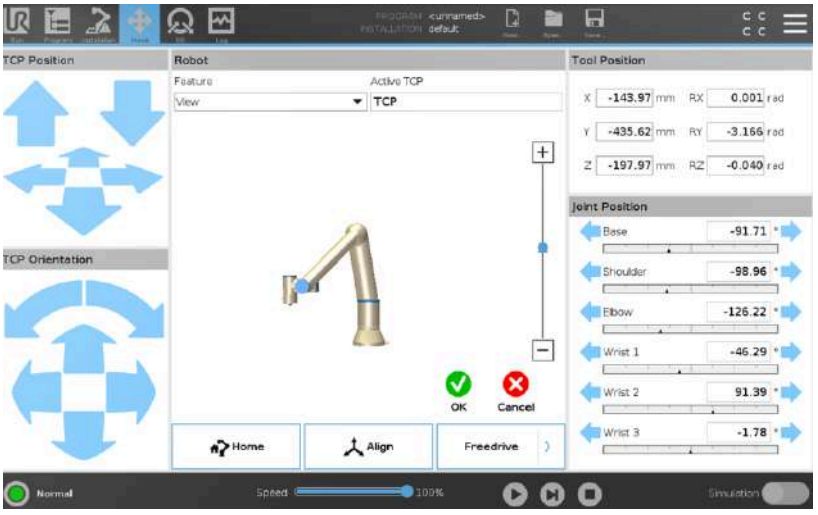
Here we are gonna use the little spike on the table. The extremity of the spike will be used as a reference point.

Start by clicking the *"**Measure**"* button next to the boxes you filled before in the installation window and TCP menu. A window will appear to the right :



From there, choose to **set the point 1**, then move the tip of the pen to the tip of the spike, for this you can use free movement or the window that you are on.

When you are satisfied with the placement of the two tip, click the green *"**OK**"* button to validate the point 1 :



Repeat this process three other times by choosing a different position each time for the robot in which the two tips are still touching each other.

After this process, the boxes you filled before will be filled automatically and the TCP will be set.

You can compare the two methods to see if there is a difference in value between them. In the following, we will use the TCP value calculated using the first method. Depending on how deeply the felt is deep into the tool, the measured value should be between 153 and 175 mm.

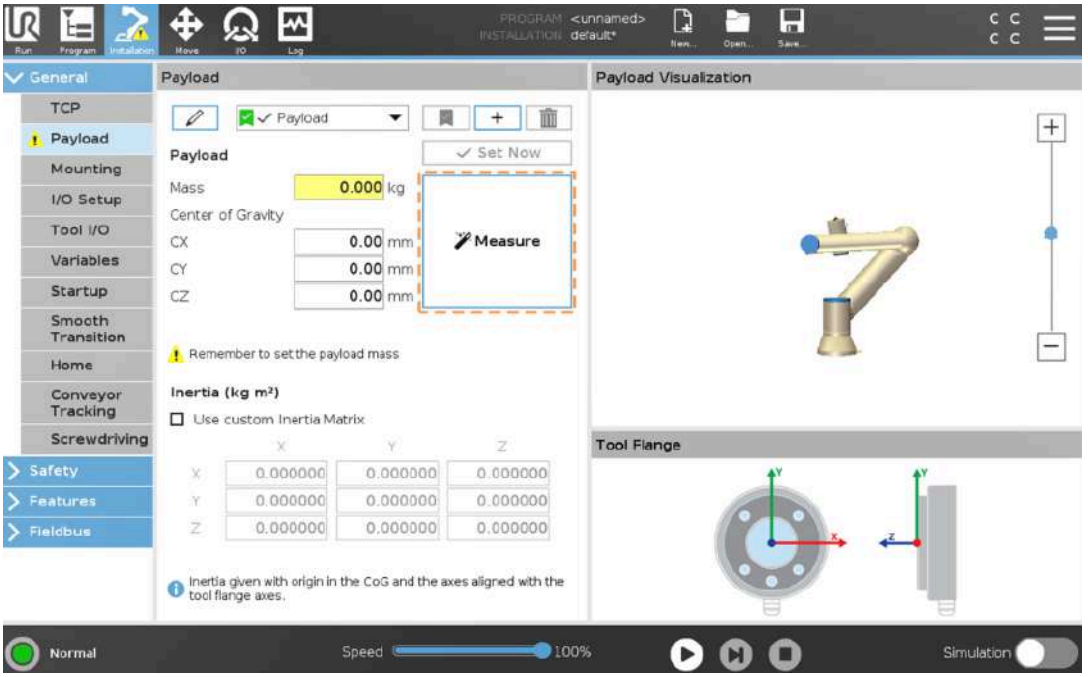
2. Determination of the payload and the center of gravity

The Payload is the parameter representing the mass of the tool. Like the TCP there are two possibilities to enter this parameter in the robot memory. The first is rarely used because it requires to know exactly the mass of the tool and where is the center of gravity to enter it manually.

What we are gonna do instead is use the Payload wizard. This second method is similar to the TCP wizard, we are gonna let the cobot calculate the payload and the center of gravity via 4 different positions.

Start by clicking the *"**Payload**"* menu in the *"**Installation**"* window.

Then click the *"**Measure**"* button in the middle.



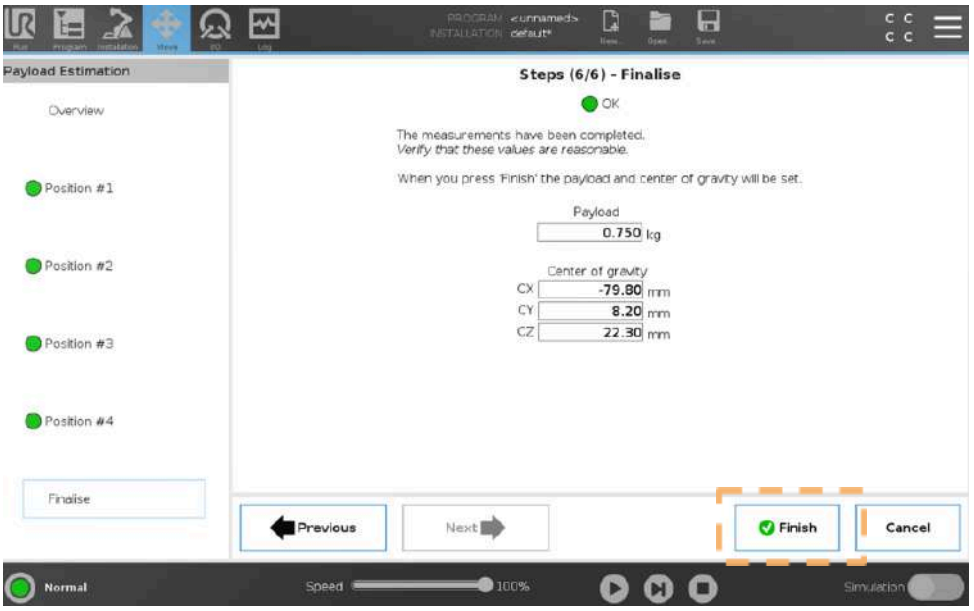
Click *"**Next**"*, then *"Set **position**"*.

You can now move the robot in a position of your choice by using the arrow on the screen or the freedrive mode.

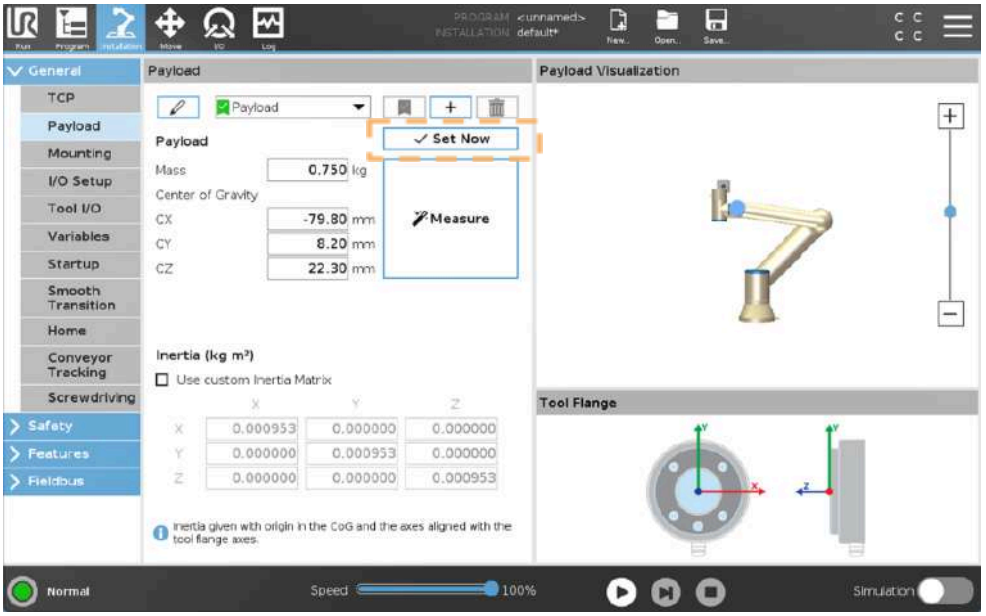
Important : If you use the freedrive method you must not touch the tool while moving the robot or the calculation could be wrongfully calculated, you can grab the robot arm only.

When you are satisfied with the position, click *"OK"* to save the position, then click *"Next"* and repeat the process three more times with a different position each time.

When you finished saving all four position you can click the *"**Finished**"* button on this screen :



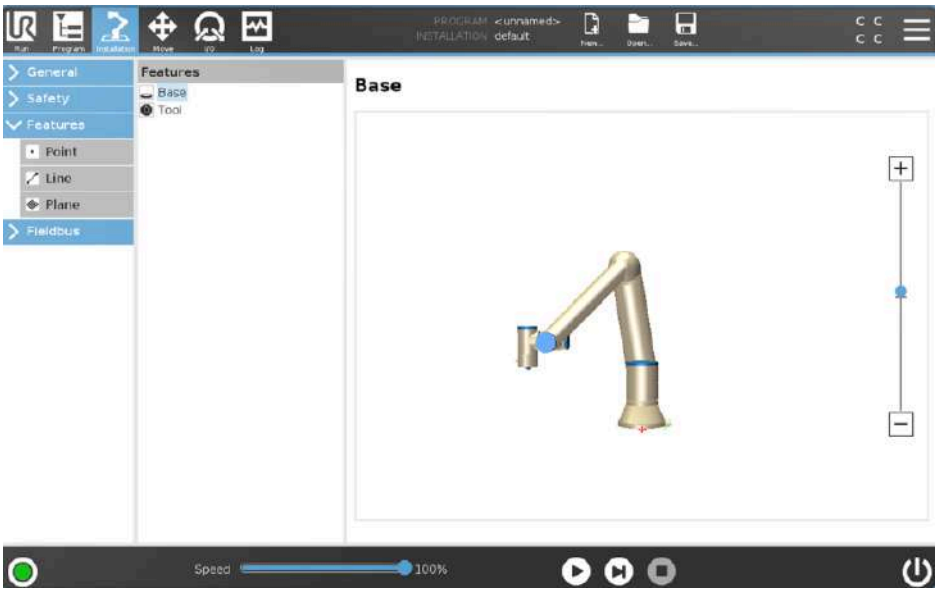
Click *"**Set now**"* on this screen to save the payload parameter :



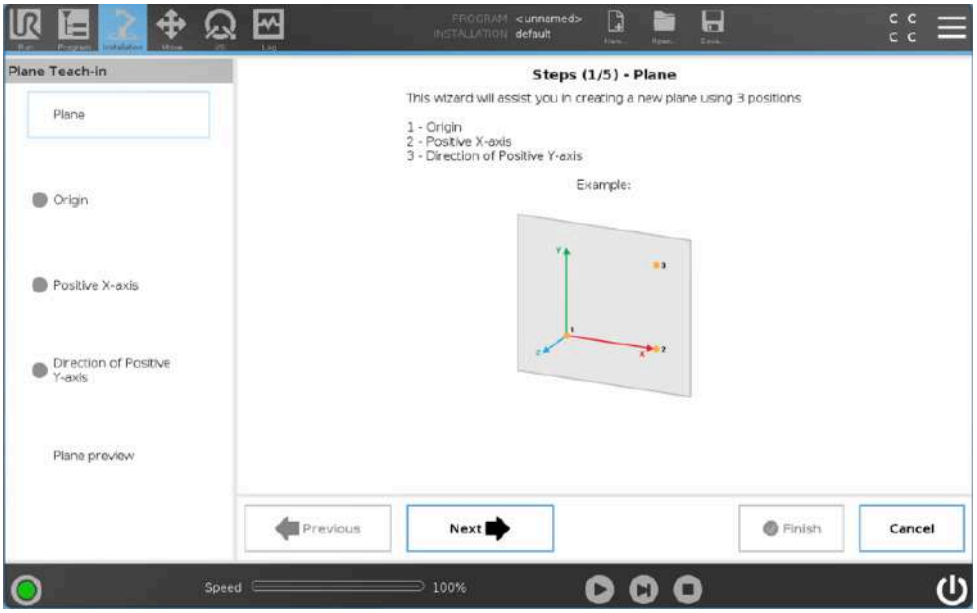
II. Set up a working plan

The working plan will help us set our waypoint for the programmation in a more precise way.

Start by going to the *"**Installation**"* window and choose the *"**Features**"* menu :



Click *"**Plane**"* to add a plane to the features, then on "**Teach this plane**" at the bottom right corner. You will normally be presented this screen :



Click *"**Next**"* to start creating the plane.

The first point you will set will be the origin of the plane, in this exercise, the origin is set on the point O on the whiteboard on the table. Therefore, you can click *"**Set position**"* and move the tip of the pen to the point O and click *"**OK**"* to save the point and *"**Next**"* to move to the second point.

The second point will be used to represent the *positive X axis*. Repeat the process you did for the origin but chose the X point on the whiteboard.

The third point will be used to represent the *positive Y axis.* Repeat the process you did for the two points before but chose the Y point on the whiteboard.

On the *"**plan preview**"* screen you can verify that the plan is correctly placed, it should be on the same level as the base, then, you can click *"**Finish**"* to validate the plan position.

III. Start programming

The purpose here is to understand the differences between the different types of movement.

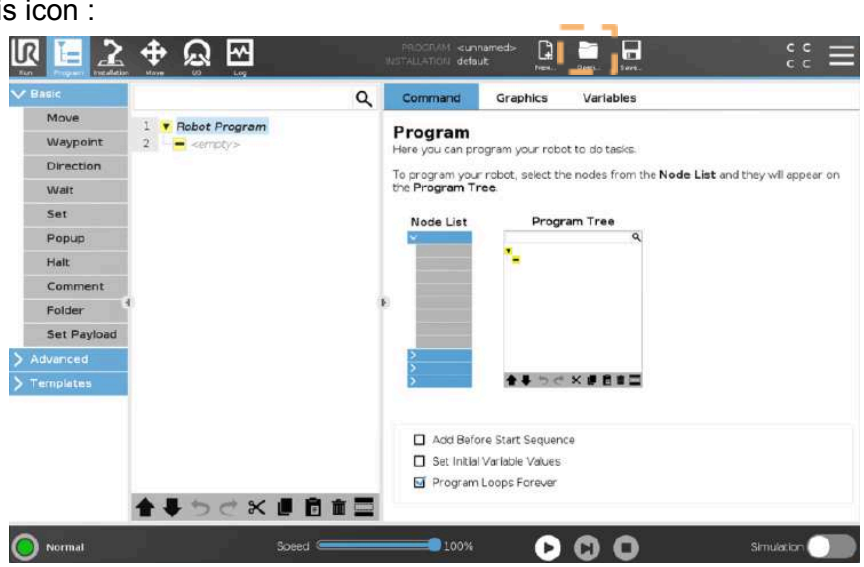
Go to the *"**Program**"* window;

The UR3e robot can have three different types of movement :

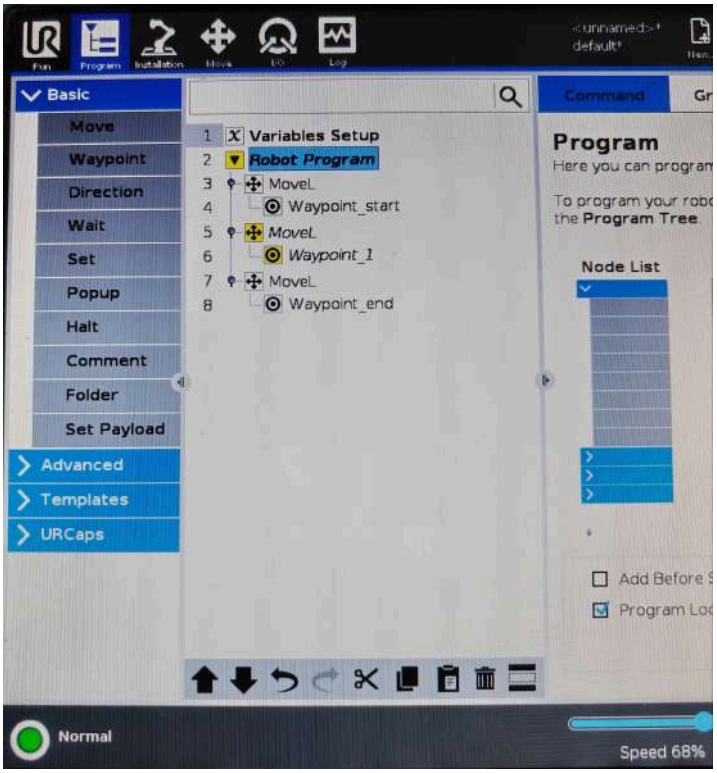
- J : "**joint move**" this type of movement is used when the trajectory of the TCP doesn't matter, for example to return to a rest position after working on something ;
- L : "**linear move**" this type of movement is used when the trajectory of the TCP must be in a straight line between two waypoint, for example for a pick and place task ;
- P : "**process move**" this type of movement is similar to the L move because the TCP will move in a straight line between the waypoint, the major difference is that the TCP will move at a constant speed without stopping at the waypoint, it could be useful to dispense glue or for welding tasks.

1. The program

To start the programme, you are gonna open the file named *"**movement.urp**"* by clicking this icon :



Normally the programme contain the lines show here :

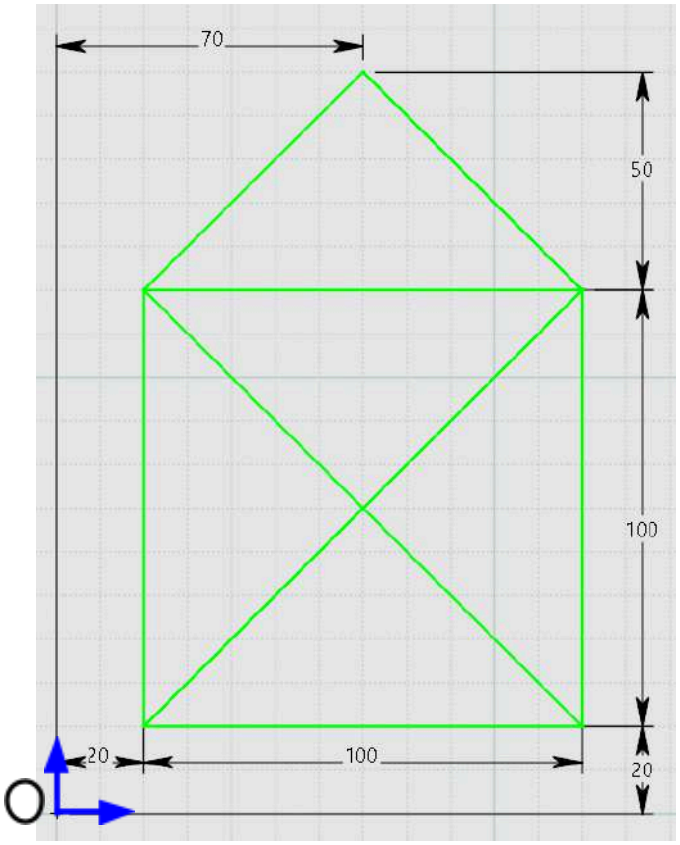


First, uncheck the box named *"Program Loops Forever"* in the screen shown above.

The *"**MoveL**"* on line 3 and the line 7 are the start and the end of the programme, they serve the purpose of putting the pen down on the paper and lifting it from the paper. You don't have to modify them.

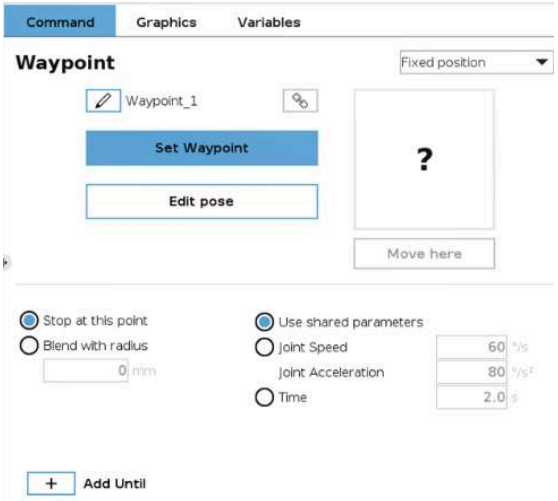
What we will focus on is the second *"**MoveL**"* on line 5.

To understand the differences between the three type of movement we are gonna draw this house on the whiteboard with the same dimensions :

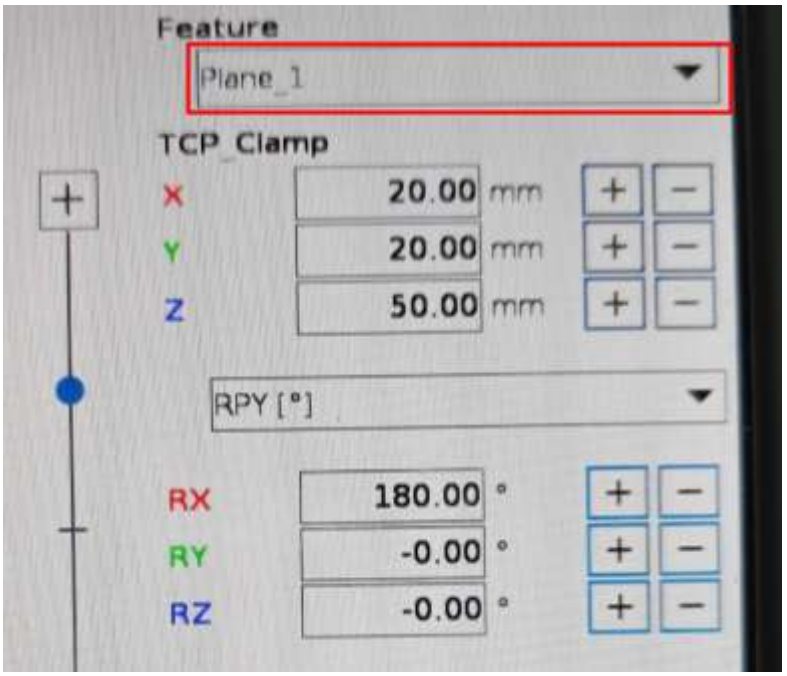


To make the house, the robot needs to know where to go, that's the role of the waypoints.

Start by choosing the waypoint in the second ***"MoveL"*** section, this screen will appear :



Click *"**Edit pose**"*, you will see a screen with this panel on the right :



This screen will be used to enter the following point where the robot must go without moving it.

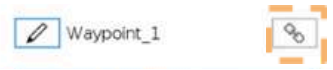
Below is a sheet with all the coordinates of the point to create the house we want to draw. Be careful to have set the "**Feature**" drop down menu to "**Plane_1**" (see the image above) to have the right reference for the coordinates given below.

Because we want to draw on the whiteboard on the table, the Z coordinate will always be 0, that's why it is not shown in the sheet, but don't forget to change it.

points	X	Y
bottom left	20	20
top left	20	120
top	70	170
top right	120	120
bottom right	120	20

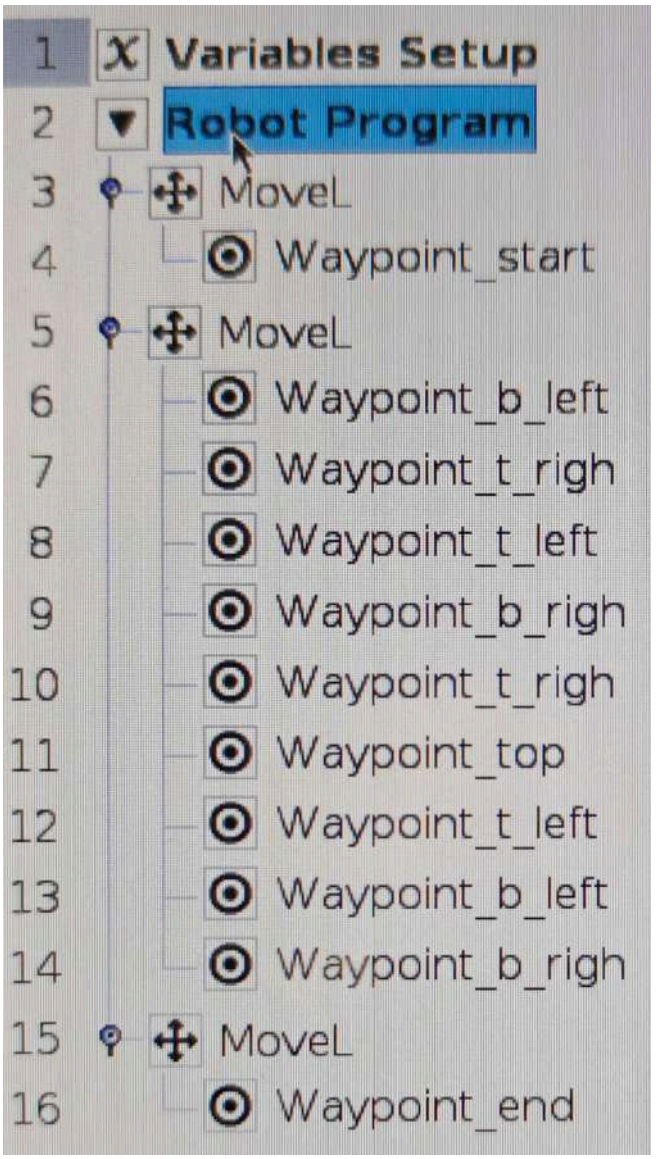
With the help of the sheet above, fill in the X and Y coordinates for the tool position on the screen of the teach pendant for the bottom left point. Then click *"**OK**"* and the waypoint will be saved rightfully. Don't forget to also change the tool orientation to 180° for the x axe (see the image above) to orient the pen perpendicularly to the white board.

Repeat the process with all the points the robot needs to go in order to draw the house.

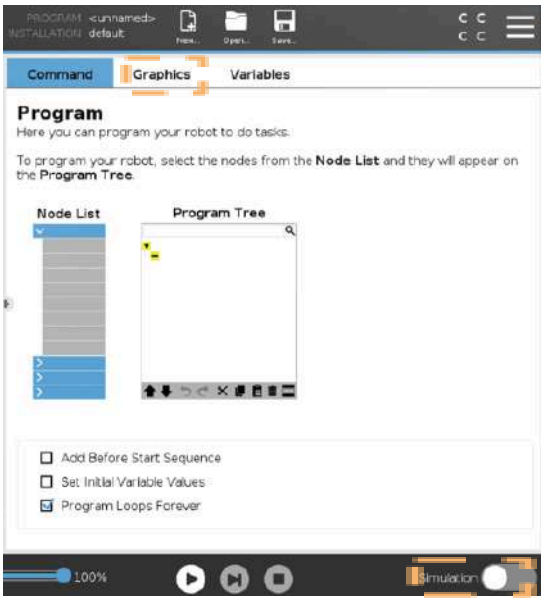
A quick tip, because we are using the same point at different times, is to link the waypoints when they are reappearing. For this, you must click the *"**Link**"* button next to the name you gave to the Waypoint : 

And then select the waypoint name you want to reuse.

When you entered all the different waypoints, the program should look like this :



Let's now test your program, if you want to simulate it first you can click on the *"**Simulate**"* switch on the bottom right of the teach pendant and see the simulation in the *"**Graphics**"* window :



If the simulation works well, you can then adjust the speed of the robot to **25%** with the cursor at the bottom of the screen then run the program by pressing the play button.

If the program runs without any errors and the house looks great, you can clean the whiteboard, then increase the speed to 75% to see how the robot behaves at higher speeds.

Lastly, save your program you made by clicking the *"**Save**"* icon, enter a name for your program then save it.

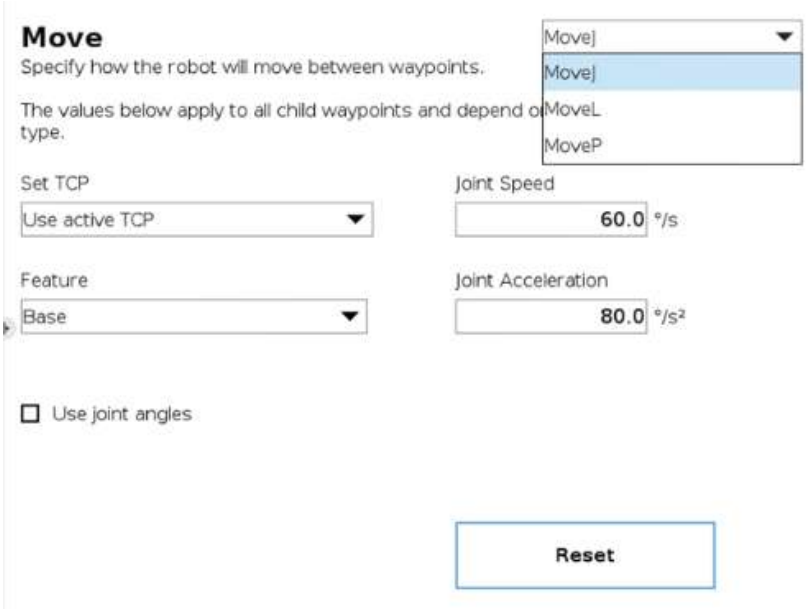
2. Comparison between movement types

Now that you successfully created the program to draw the house we are gonna see the differences between the three types of movement : J, L and P.

First, decrease the speed to 50%.

Then, you can redraw the house with the L movement and don't erase it from the whiteboard, it will be used to compare between the types of movement.

Now change the second *"**MoveL**"* to *"**MoveJ**"* via this dropdown menu :



Restart the program with the MoveJ sequence and see how the house will look compared to the MoveL sequence.

Repeat the process by changing *"**MoveJ**"* to *"**MoveP**"*.

What differences can you clearly see ?
What uses do you think is more accurate to the use of a *"MoveJ"* sequence ? To a *"MoveL"* ? To a *"MoveP"* ?

3. Impact of speed and inertia

You can first clean the whiteboard and restart the program with the *"**MoveP**"* sequence at 30% speed.

Then start the program another time without cleaning the board but with 70% of speed.

How does the speed influence the look of the house ?

How does the speed of the robot influence the path of the robot according to the waypoint in a *"MoveP"* sequence ?

Exercise 2 : Pick and Place objects

This exercise aims to teach you how to use the cobot with a clamp to pick and place some objects.

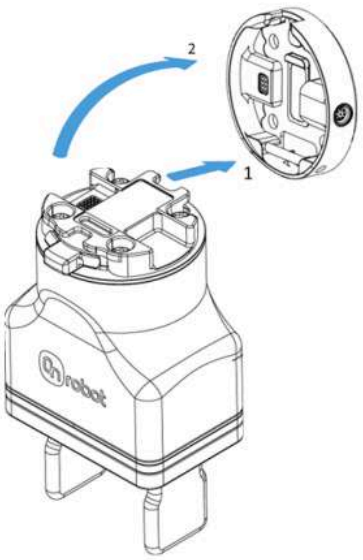
I. Introduction to the clamp

A. Description and installation of the clamp

First, we will focus on the clamp.



The clamp consists of two fingers that can move towards or apart from each other.
It can be attached to the robot :



Step 1:

Move the tool close to the Quick Changer as illustrated.

The hook mechanism (rod and hook tongue) will keep the lower part locked once mounted.

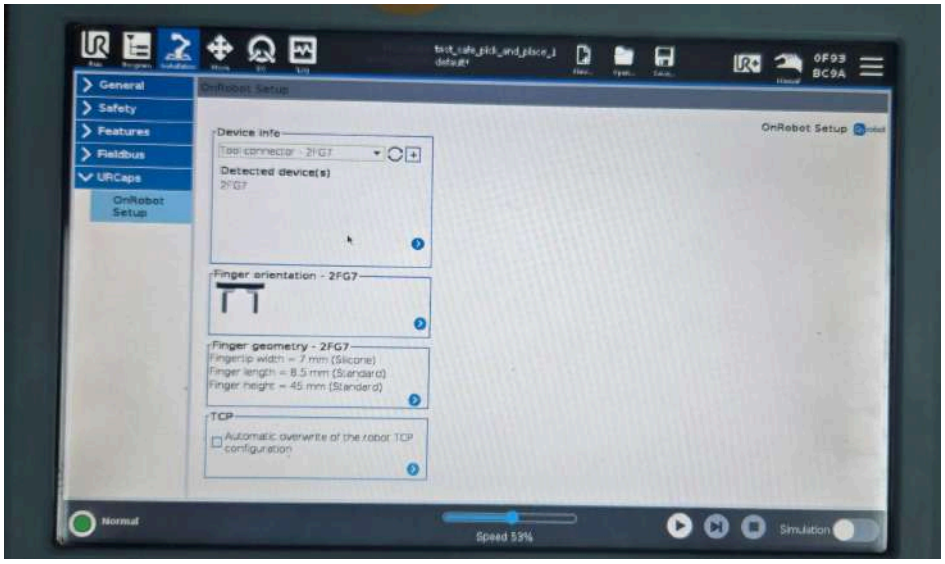
Step 2:

Flip the tool until it is fully mated, and you hear a clicking sound.

To unmount the tool, press the aluminum button on the Quick Changer and repeat the steps in the reverse order.

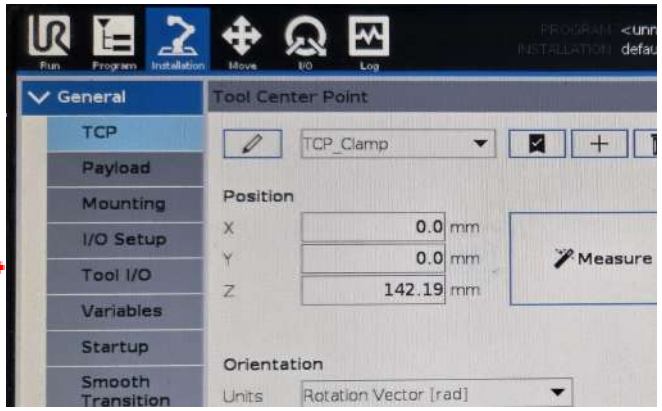
As a new tool is attached to the robot, the TCP and the Payload have to be defined again. To save time, these characteristics are already saved in the Cobot :

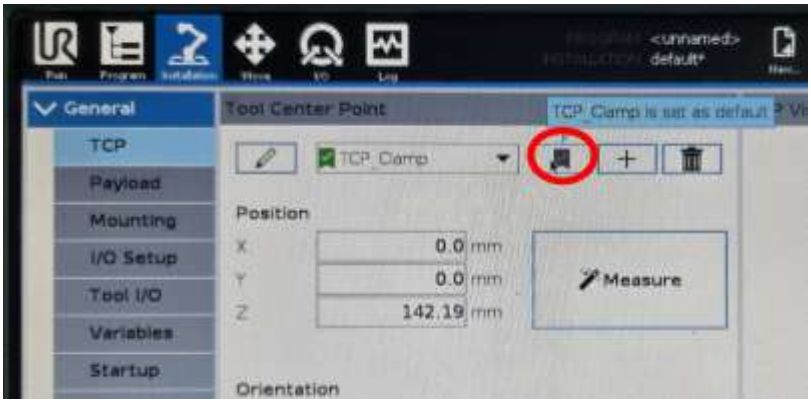
- In the “Instalation” menu, verify the clamp is detected by the robot by clicking on the URCaps dropdown menu:



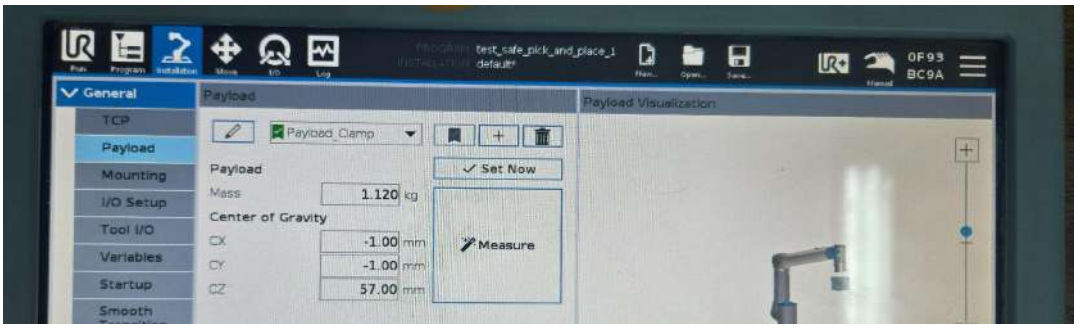
- Then, search in the dropdown menu of TCP the name “TCP_Clamp”. Select it and set it as the default TCP.



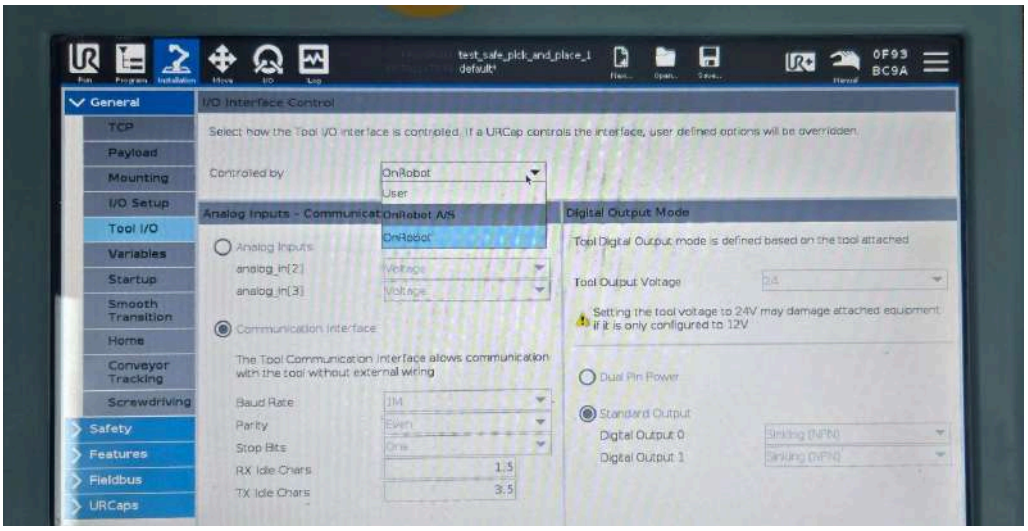




- Do the same for the Payload named “**Payload_Clamp**”. Make sure the different values are the same as in the images.



- In the “**Tool I/O**” setting from “**General**” dropdown menu, verify the dropdown menu “controlled by” is set on “**OnRobot**” :



Now the clamp is correctly installed on the robot. We can see how it works.

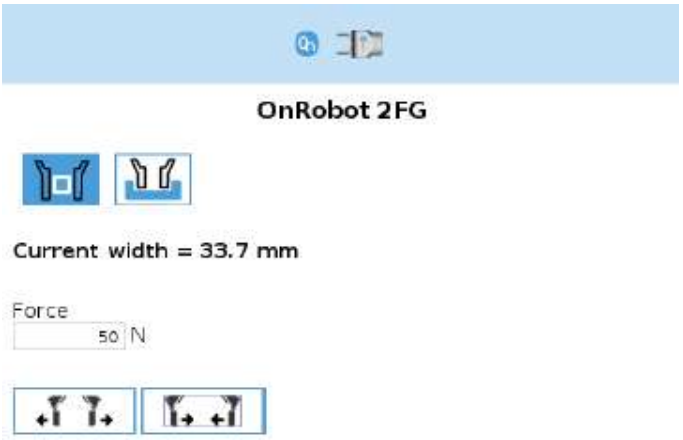
B. Activation of the clamp

Manual activation :

The clamp can be activated manually with the toolbar in the upper right corner…



… and by going on the OnRobot 2FG window :



The buttons  are used to define the type of object we want to grip. For this exercise, we will only use the first : gripping the objects externally.

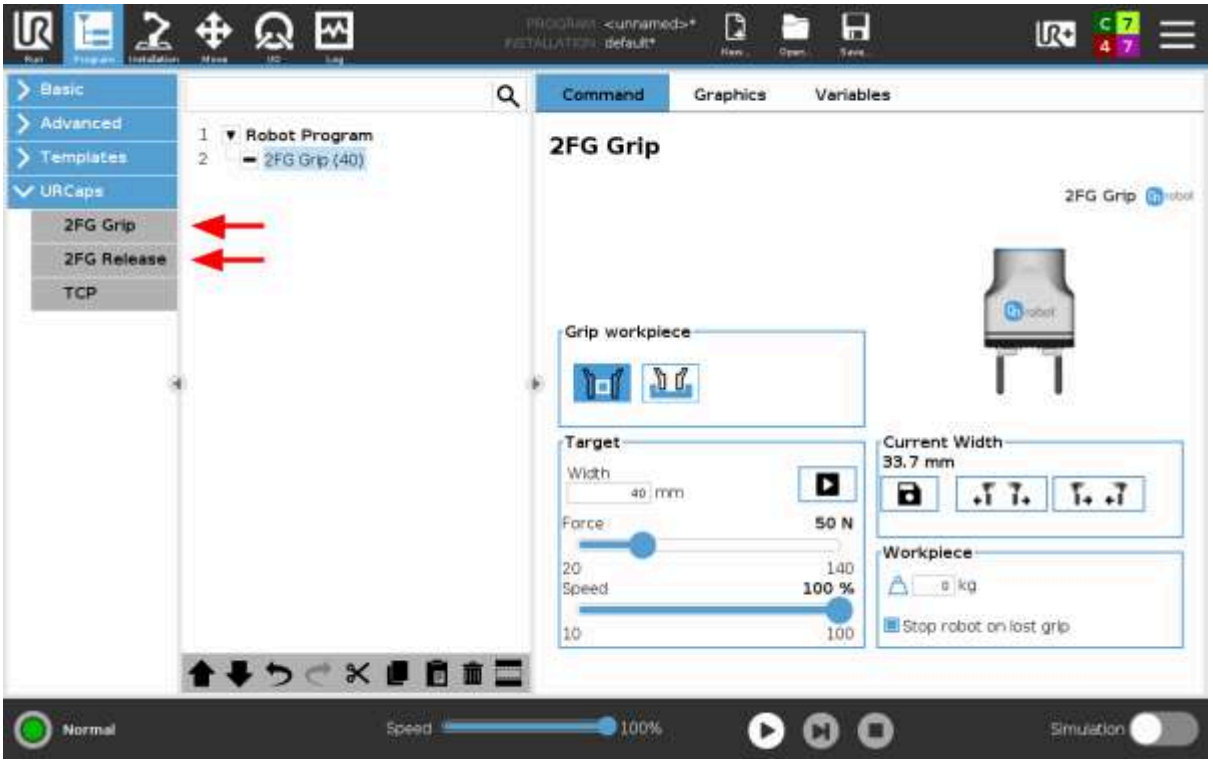
- The current width is measured between the two fingers and its value is between **71 and 34 mm**.
- The force is the maximum effort the clamp can apply between its two fingers, its value can be set between **20 and 140 N.**

You can open the clamp with the button : 

And close it with the button : 

Activation in a program

The activation commands for the clamp are located in the dropdown menu “**URCaps**”, in the “**Program**” menu. These commands are “**2FG Grip**” and “**2FG Release**".



The image above show the “**2FG Grip**” interface, in which there are several windows :

- The buttons to select the type of objects, in the “**Grip workpiece**” window. As same as the manual activation, it will always be set on the external grip option.

- The “**Target**” window allows us to define the width of the object we want to grip, the force with which the clamp will hold the object and the speed of the fingers during their movement.

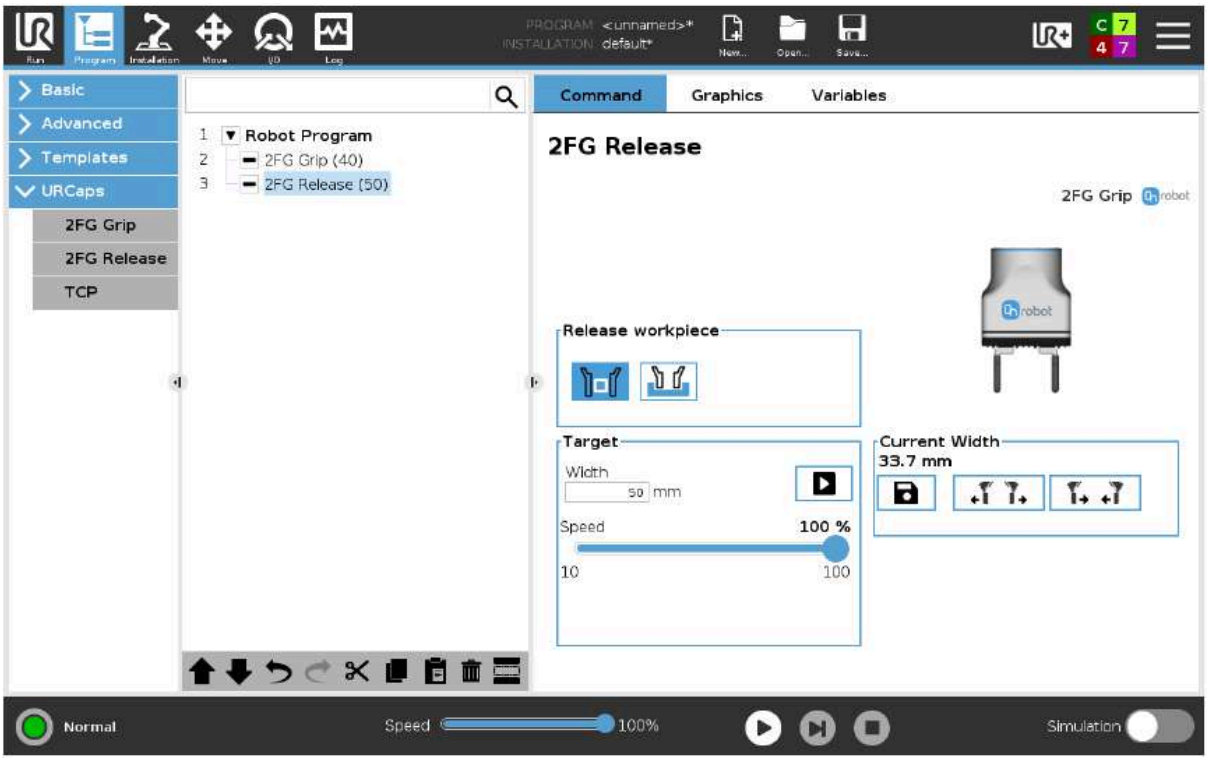
  The button  can be used to test the configuration : the clamp will reach the targeted width at the defined speed with a force that will not exceed the set value.

- The “**Current Width**” window alloys to move manually the fingers and then to register their position as the targeted width by clicking on the button  .

- In the “**Workpiece**” window, we can define the weight of the object we want to pick. It will automatically calculate the new Payload with the object at the end of the command run. The Payload can also be manually defined in the program, as it will be presented later.

If the grip command is executed without an object to grip between the fingers, the robot will stop and send an error message on the screen.

For the “**2FG release**” command, the interface is very similar :



Because its goal is to release the object, its weight is not important and no maximum force has to be defined.

However you can still use it to pick objects, as there is a security maximum force. So if you use the release command to close the clamp and there is an obstacle between the fingers, the clamp will grip this obstacle. This can be useful to avoid the error message of the grip command if you are not certain there will be an object to grip when you close the clamp (see application in the next procedure)

II. First Pick and Place programs

A. First version : using waypoints

We will write a simple program to pick an object at a first position, and place it at a second position. The object will be a 54x54x54 (mm) cube and we will use a support or a marker to positionate it.

Here the different step of the program creation :

1. First, define the initial position of the cube by fixing a support on the table.

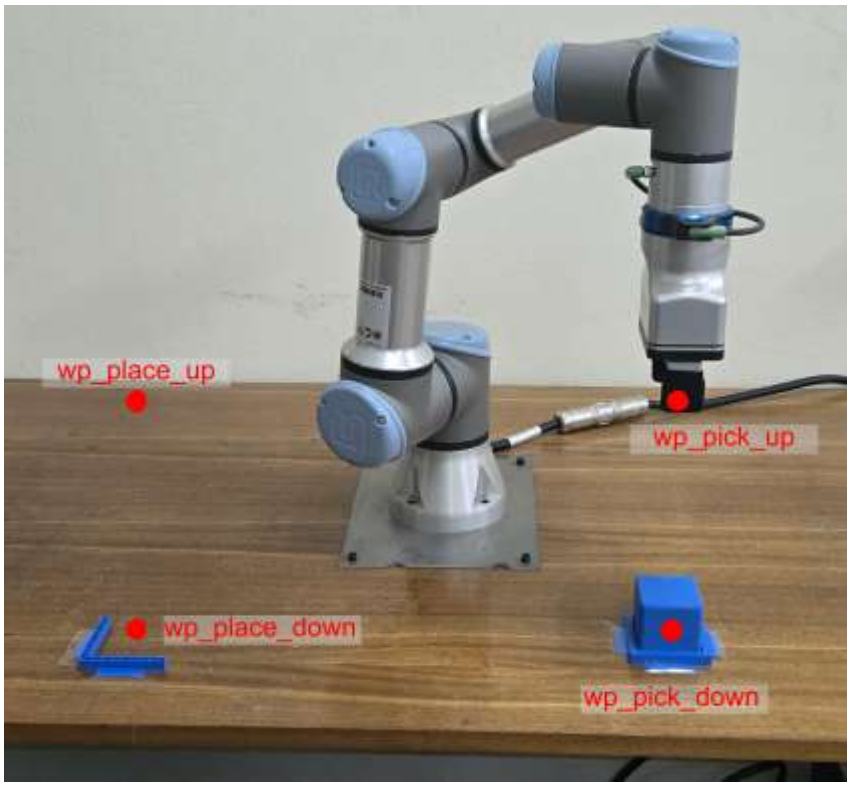


2. Then we will place the clamp vertically, with the “**Align**” feature of the “**Move**” menu. To do this, first place manually the clamp the most vertically you can, to avoid errors from the robot.





3. When the clamp is oriented vertically, you can define the waypoints of your program. Here their names we will use for the explanation:



You will first define the waypoint "***wp_pick_down***", for which we need a precise position. To do this, place the clamp opened above the cube, like to pick it, with the "**TCP position**" arrows and the arrows for the Z rotation of the Move menu, in the **Base** Feature.

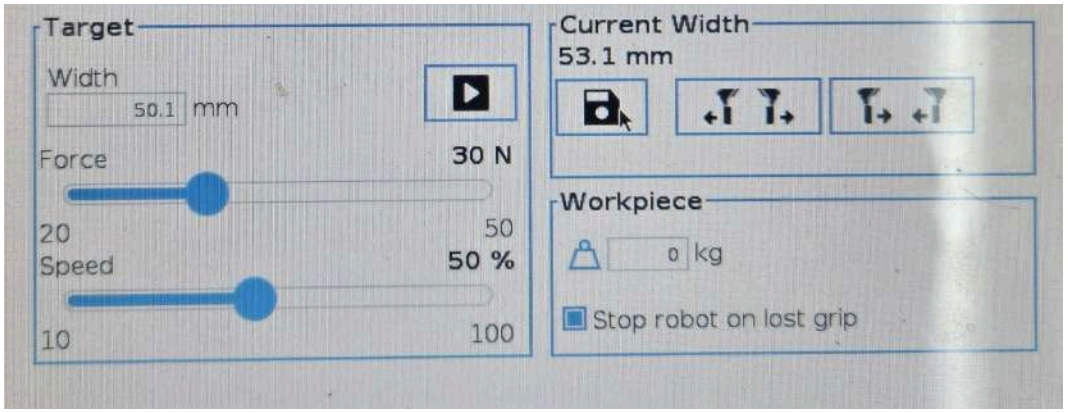




Be careful to place the clamp such that the cube is centered between its fingers.

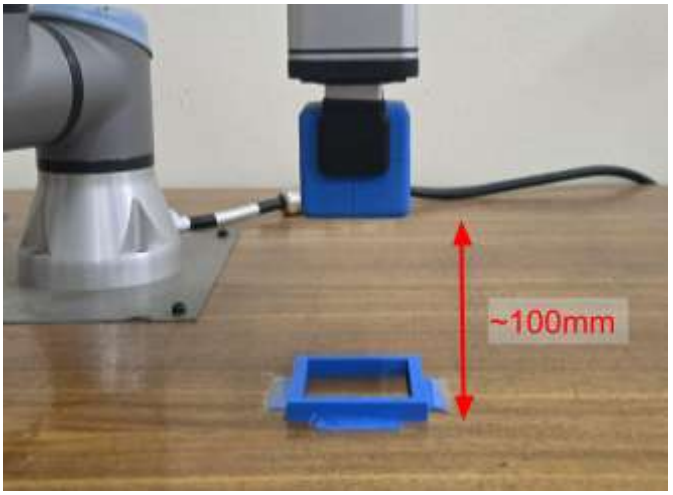
After that, you can create in your program a "**MoveL**" node, set the robot position in the waypoint and rename it as "wp_pick_down".

4. As the clamp is now positioned to pick the cube, you will place in your program a "**2FG Grip**" function. Click on the "MoveL" node you just created and add the grip function, it will place it under the waypoint node. In the grip function, set this parameters : Maximum force : **30N** and Speed : **50%**.
   In the "Current Width" part, close the clamp on the cube until you hear a "click" sound, it informs you the cube is gripped. Then save the current width in the function.



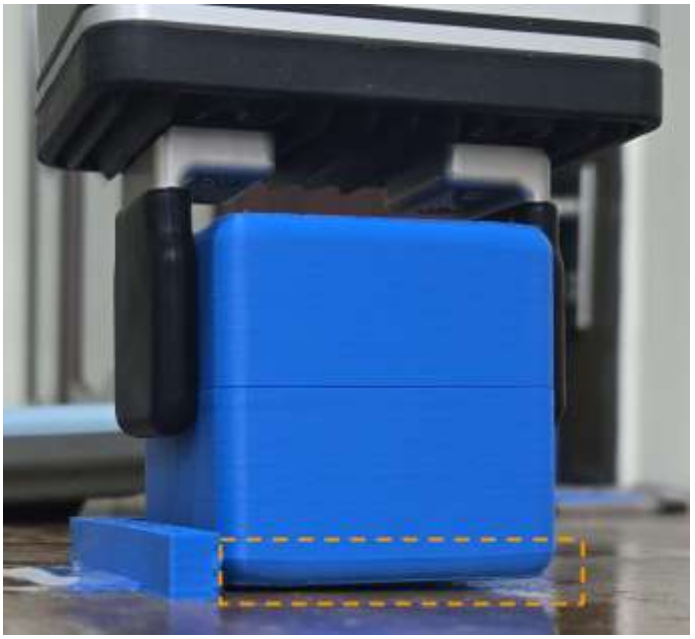
It is important for the next steps to keep the cube gripped in the clamp.

5. After that, you can define the "***wp_pick_up***" waypoint. To do this, click on the "***Robot Program***" node and add a "**MoveJ**" node that should be at the top of your program. When you set the waypoint position, make the robot go up by clicking on the "**Z+**" arrow of the "**TCP position**" part, always in the **Base** Feature. Stop it when the base of the cube is around 10 cm above the table.



6. The "***wp_pick_up***" waypoint will be used two times : one at the start of the program and one after picking the cube. So you need to add a "**MoveL**" node after the grip function and link the created waypoint with "***wp_pick_up***" as you learned in the previous exercise.

7. Then you can define the "***wp_place_down***" waypoint. Add a "**MoveL**" node after the last "**MoveL**" node you created, click on the waypoint it creates, rename it and set its position, by moving the clamp as in step 3 to the wanted position. Pay attention to keeping the clamp vertically. To set the height of the waypoint, place the cube base up to 1 or 2 mm from the table. Make sure they will not be in contact.



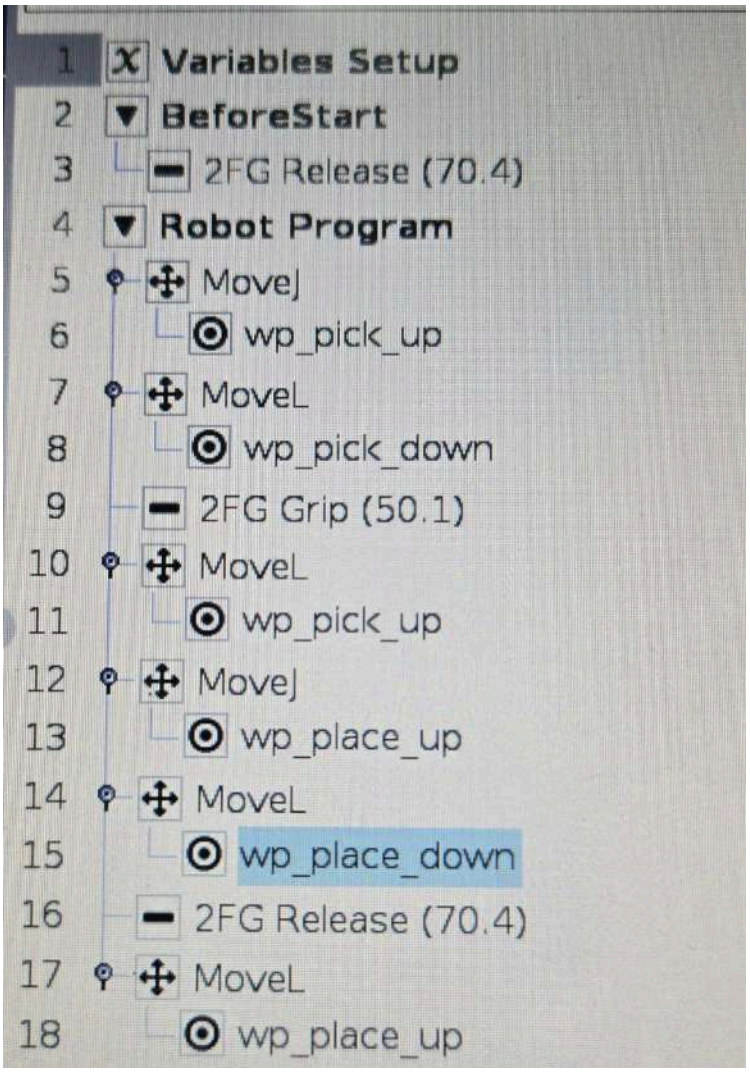
8. When “***wp_place_down***” is set, you can add a “**2FG release**” function. Set the targeted width to the maximum (70 mm) manually to release the cube at the same time. Let the speed to 100%.

9. Then, add a “**MoveJ**” node between the “**MoveL**” nodes of “***wp_pick_up***” and “***wp_place_down***” to add the first “***wp_place_up***” node. Set its position as you made it for “***wp_pick_up***”, with a height of the cube base around 100 mm from the table.

10. At the end of your program, add a “**MoveL**” node and reuse the “***wp_place_up***” waypoint to make the robot go up after placing the cube.

11. Finally, before testing your program, click on “**Add Before Start Sequence**” in the “**Robot Program**” node. In the “**BeforeStart**” node it creates, add a release function, with a targeted width to the maximum (70 mm). In this way, you will be sure the clamp is always open for the start of the program. Also, unselect the checkbox “Program Loops Forever” to not make the program loop.

Your program should now look like this :



Testing your program :

As we set the waypoints by moving the robot to the wanted positions, the simulation test you learned in the previous exercise won't be mandatory here. Instead, you can verify your waypoints are correctly organised. Remove the cube from the robot area, click on each waypoint, in the order they appear in the program and click on "**Move here**" for each one. The window that appears allows you to move the robot to the selected waypoint by clicking on "**Move robot to: New position**". In this way, you will check all positions the robot will take and in which order.

You can also use this method to easily modify the position of one waypoint, if it doesn't fit you, by going to its previous position and then going in "***Set Waypoint***" to change it.

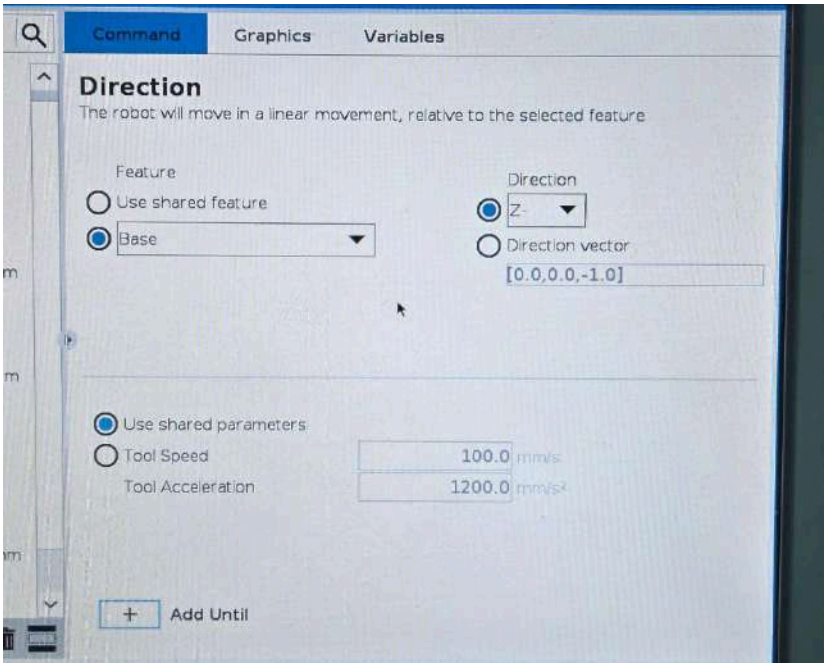
When you see all your waypoints are correctly defined, save your program, set the general speed of the robot to **30%** and run your program. If it is executed with any issues, you can increase the general speed and run it again.

B. Second version : using direction nodes

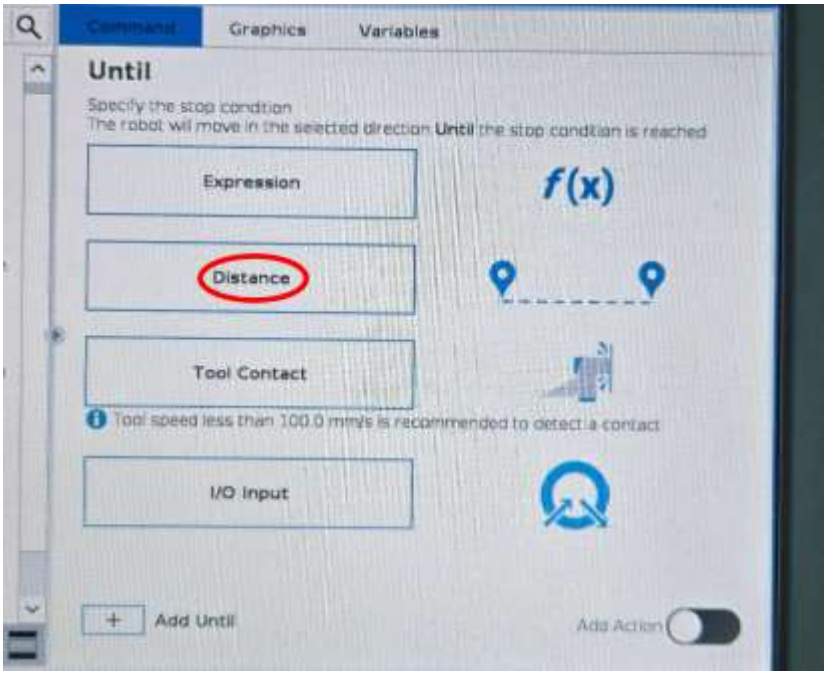
For this part, we will use the same structure as the code of the previous one. The difference will be in the way the robot will go up and down. Instead of using waypoints in the MoveL commands, we will introduce Directions commands in them.

1. For the first "**MoveL**" node of your program, remove the waypoint it contains and add a "Direction" node (that you will find in the "**Basic**" menu, just under "**Waypoint**").

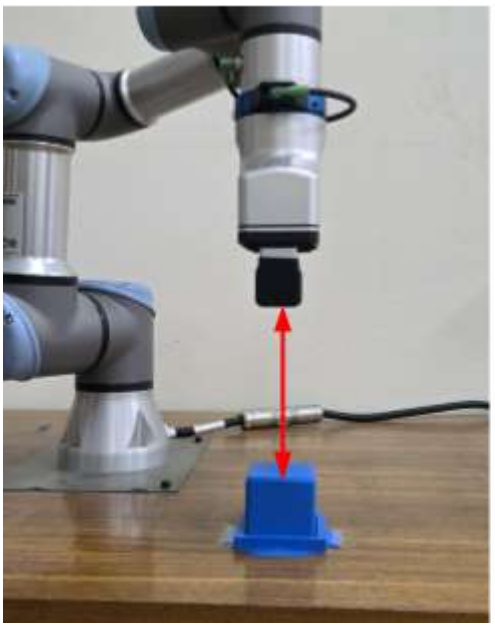
2. The robot will move by following the Z axe of the base (Z- to go down).



3. In the “**until**” node, select the distance option



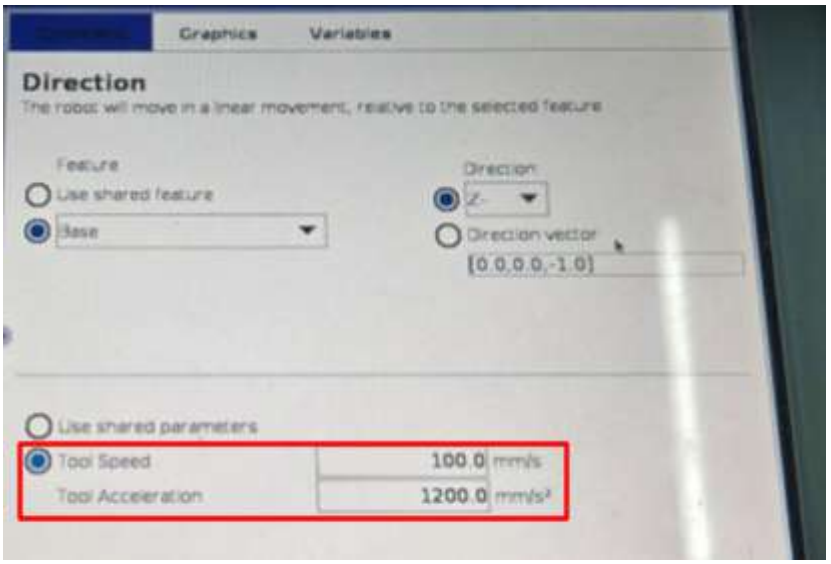
4. To measure the distance you have to define, place the clamp in “***wp_pick_up***”, place the cube in its position to be gripped and measure the distance between the end of the clamp fingers and the top of the cube and add 30 mm that correspond to the vertical length of the gripping surface (see the images below).



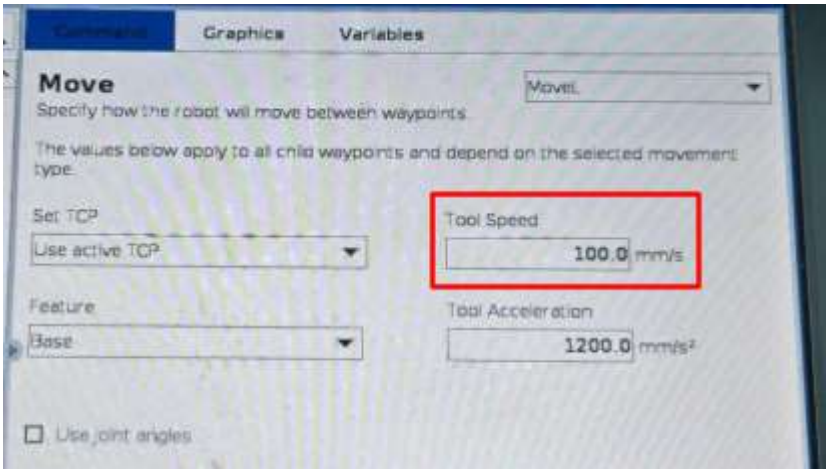


5. For the second “**MoveL**” node, follow the three first steps above and set the same distance.

6. For safety measures, we will modify the speed of the “Direction” nodes. To do this, you have two options :
    - The first is to modify the Tool Speed of the “Direction” node by clicking on the select button “**Tool Speed**”. If you do this, it will unselect the “**Use shared parameters**” feature, and allow you to modify the “**Tool Speed**” and the “**Tool Acceleration**” values. This will modify the speed and acceleration of only this direction node.



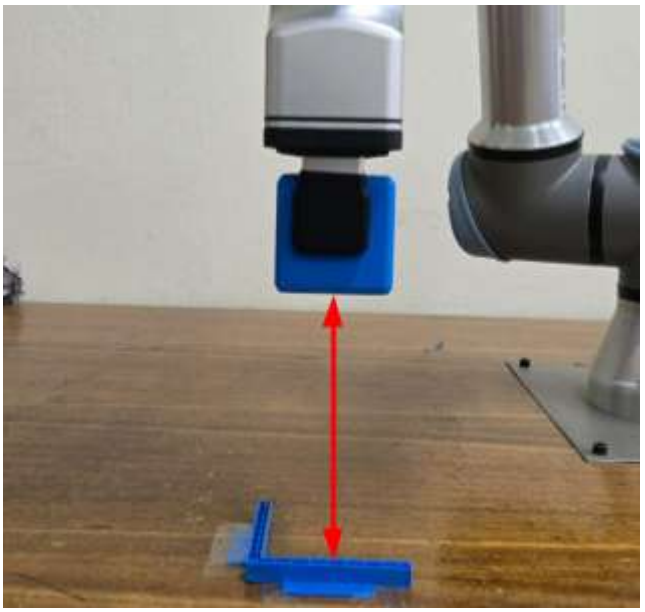
    - The second is to modify the “**Tool Speed**” of the “**MoveL**” node that contains the direction node, with the “**Use shared parameters**” feature selected in the direction node. If you do this, all nodes (waypoints and directions) that are included in this “**MoveL**” node and have “**Use shared parameters**” selected will be run with the speed you set in the “**MoveL**” node. This works with the “**MoveP**” node too, and also with the “**MoveJ**” node but with the “**Joint Speed**” instead of “**Tool Speed**”.



Whatever is the option you choose, set the tool speed to 100 mm/s for the descent and rise of the tool.

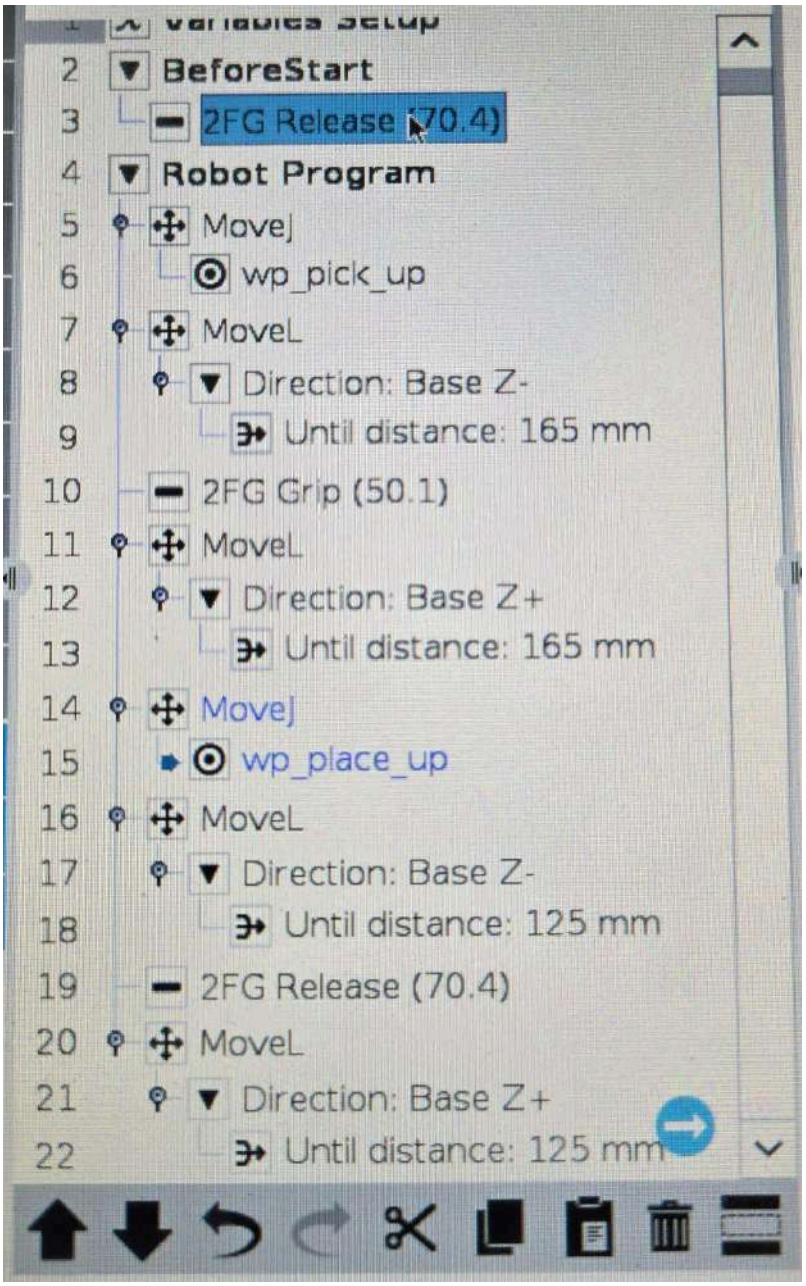
7. To modify the third “**MoveL**” node, you want to have the robot in the “***wp_place_up***” position with the cube gripped. To do this, add a “Halt” node (that is in the “**Basic**” menu) after the “**MoveJ**” -> ”***wp_place_up***” node. This will stop the program at this step when you will run the program.
   So place the cube in its picking position, set the general speed to 30% and run the program from the start, it will make the robot pick the cube and go to the “***wp_place_up***”. If the way the robot picks the cube doesn’t look like the image in the step 4 (the fingers grip the cube to high or the clamp collide with the cube), open manually the gripper (be careful to catch the cube when you do it), modify the length you put in the “Until” nodes, put the cube in its picking placement and test the program again. Do this until the way the robot picks the cube is correct. After that you can remove the “Halt” node.

8. When the first length is correctly set up, move the robot to “***wp_place_up***”, measure the length between the base of the cube and the table (see the image below), subtract it 2 mm to avoid the cube collide with the table and this will be the distance you will set in the "Until" node of the “Direction” nodes in the third and fourth “**MoveL**” nodes of your program.



9. As the second “**MoveL**” node, modify the last by setting the last length you measured in it. Modify the tool speed for the placement in the same way you did it for the picking part.

You should get a similar code as this one :



You can test your program, once with the general **speed at 50%**, then you can increase it. If it is necessary, modify the two lengths you defined in the “Direction” nodes.

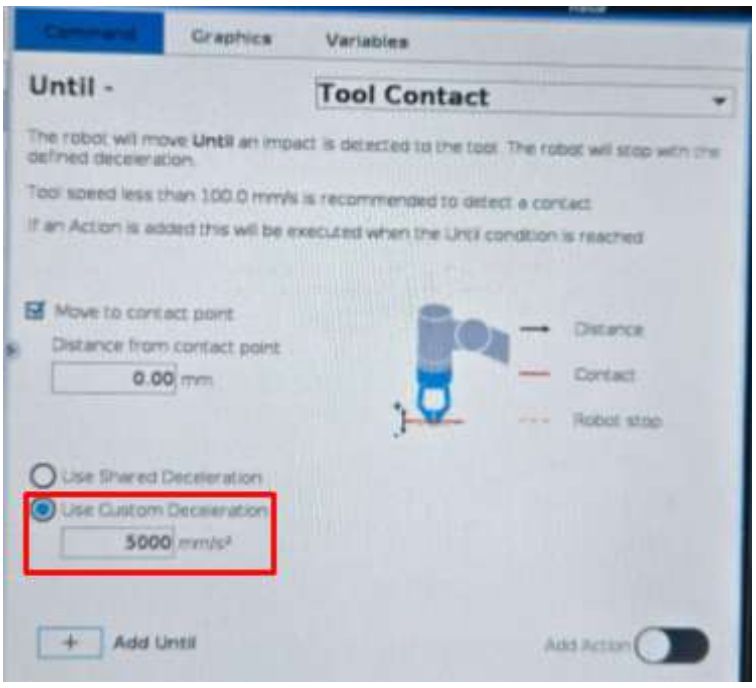
III. Improving the placement

Releasing the cube with a gap between it and the table isn't really satisfying. So we will improve the last program by modifying the release section.

- In the “until” node of the descent movement, after the “***wp_place_up***” point, select the “**contact tool**” option (from the “**Until Distance**” option, you can select it in the upper right drop down menu), it will stop the clamp when the cube touches the table.



- Set the **Deceleration to 5000 mm/s²**. Modify the speed of the Direction command to **50 mm/s**.



Once your program is modified, test it. If the robot forces too much when it places the cube, change the Tool Contact Deceleration value, try values between 3000 and 6000 mm/s², keep the one that works best.

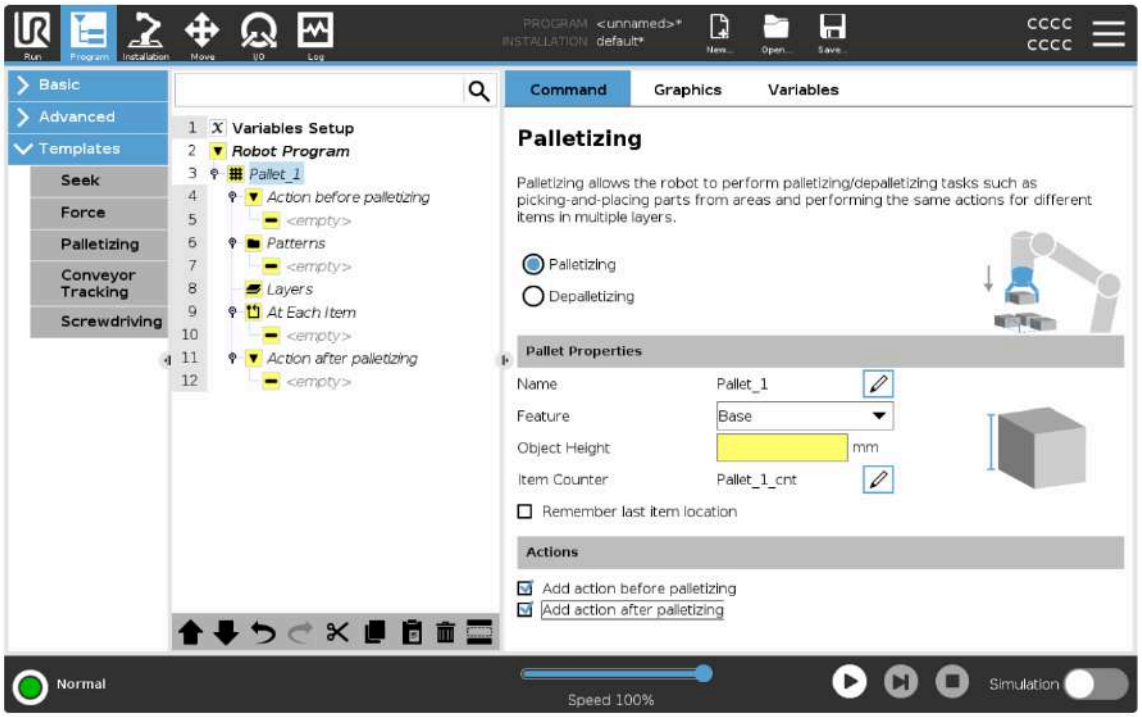
Exercise 3 : Palletizing

In this exercise, we will use as in the previous exercise the gripper to pick and place some cubes, but with a more advanced method.

I. Presentation of the Palletizing function

One useful function already implemented in the robot is palletizing. You can use it to pick or place several objects and organize them following different patterns.

You can find this function in the Templates menu :



The Palletizing option will be selected to place the objects, contrary to the Depalletizing button that will be used to pick objects.

In the first node (Pallet_1), you define the object heights and if you want to palletize or depalletize.

In the Patterns node, you specify with which pattern the objects are organised : Grid, Line or another specific pattern (Irregular).



Grid

To teach the positions, select each item in the Program Tree:

- Corner_Item_1
- Corner_Item_2
- Corner_Item_3
- Corner_Item_4

Insert the number of rows and columns in the appropriate text boxes to set the dimensions of the pattern.

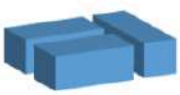


Line

To teach the positions, select each item in the Program Tree:

- Start_Item_1
- End_Item_1

Insert the number of items in your sequence using the **Items** text box at the bottom of the screen.



Irregular

To teach the positions, select each item in the Program Tree:

- Item_1
- Item_2
- Item_3

Tap **Add Item** to add and identify a new item in the sequence.

You can define several patterns in one palletizing function. Then, you can add layers, and specify for each one the patterns they will use.

At each item node : The At Each Item Wizard assists in defining the actions performed at each item on a pallet, such as the ReferencePoint, the Approach Waypoint, ToolActionPoint Waypoint, and Exit Waypoint (described in the table below). The Approach and Exit Waypoints for each item remain in the same orientation and direction regardless of the different items's orientation. All these waypoints will be defined for the first object (Corner_item_1, Start_item_1 or item_1, depending on the chosen pattern).

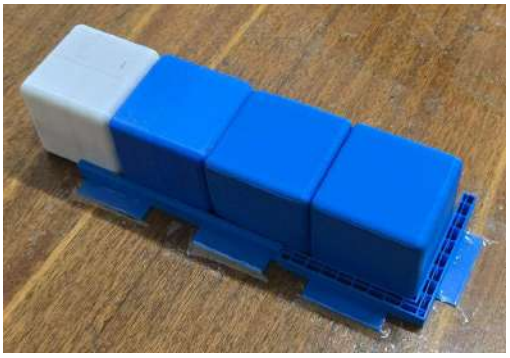
Tool Action Point		The location and position you want the robot to be in when conducting an action for each item in a layer. The ToolActionPoint Waypoint is the ReferencePoint by default, but it can be edited in the Program Tree by tapping the ToolActionPoint Waypoint node. When using the wizard, the ReferencePoint is the first position in the first defined layer on the pallet. The ReferencePoint is used to teach the robot the Approach Waypoint, ToolActionPoint Waypoint, and Exit Waypoint for each item in a layer.
Approach		The collision-free position and direction you want the robot to take when approaching an item in a layer.
Tool Action		The action you want the robot attachment to perform for each item.
Exit Waypoint		The position and direction you want the robot to take when moving away from an item in a layer.

After following this wizard, you can define the actions the robot has to do in the ToolActionPoint.

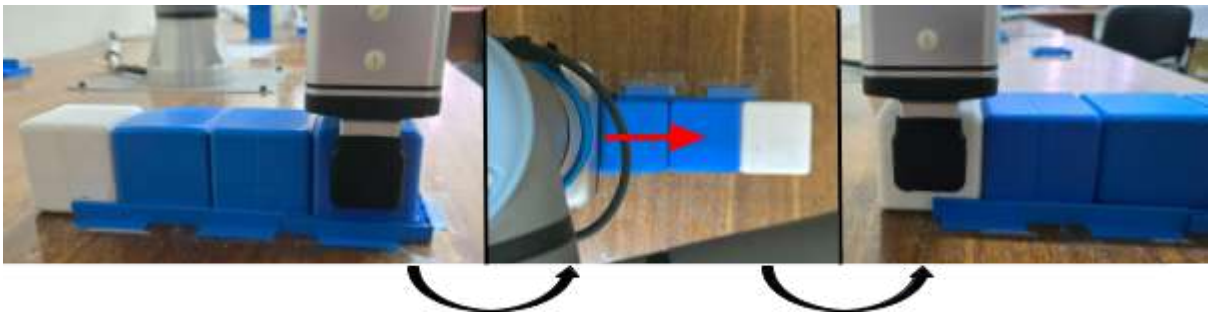
You can also add actions the robot has to perform before reaching the approach point and after being at the exit point. To add these actions, click on the check boxes at the bottom of the Pallet_1 node windows (see the first image of the previous page).

II. First program : Depalletizing

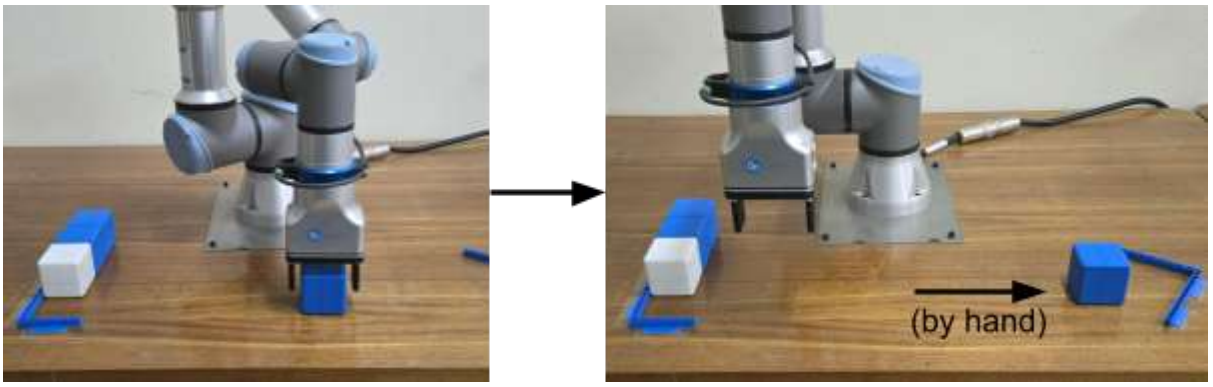
We will focus on a depalletizing program that is easier to set up than palletizing. First, you will use 4 cubes that you will arrange in one line (see the image below). Save their positions with supports or with a marker before programming. In this way, use the line pattern for depalletizing, and apply it to just one layer.



To define the *StartItem_1* and the *EndItem_1* positions, the simplest way is to put the clamp opened just above the first cube, set the Start_item position, move the clamp by using the Y axe of the **Tool Feature** to the last cube and set the *End_item* position.

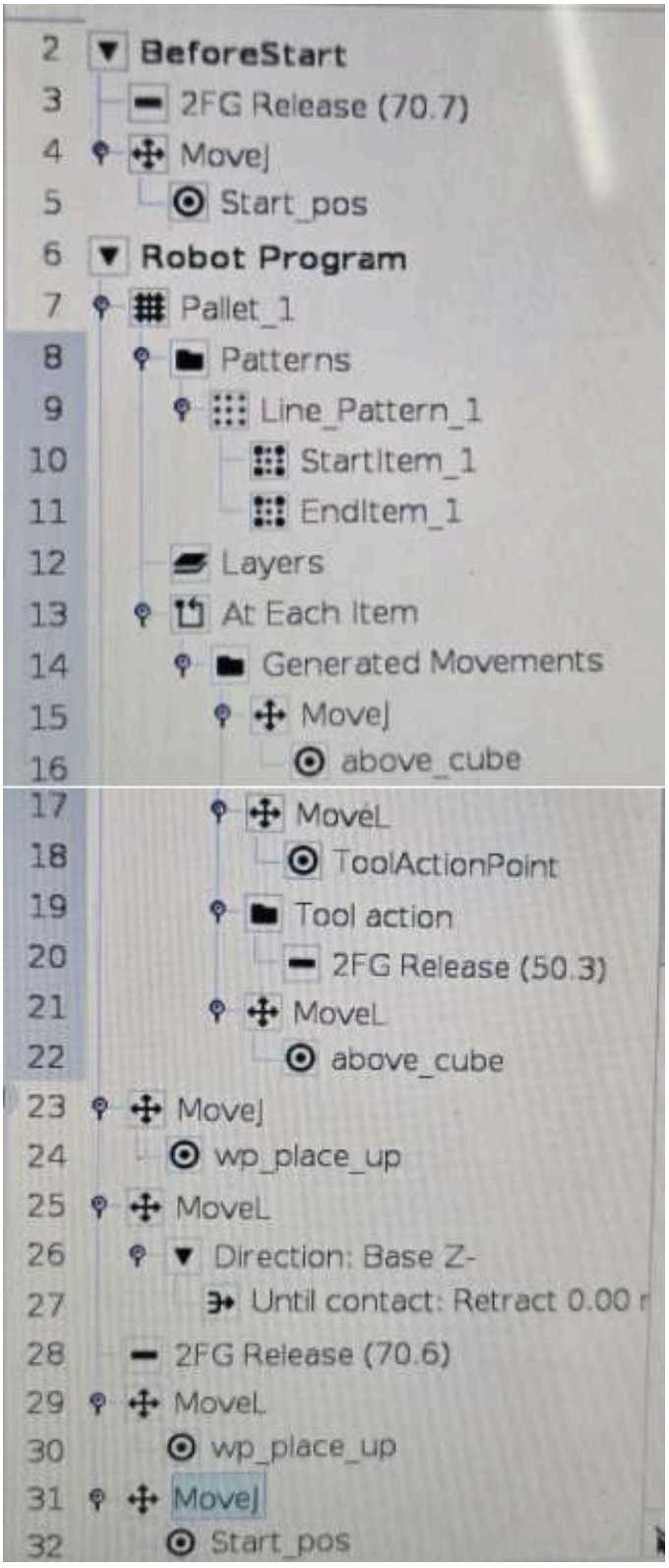


To release the cubes you can use one of the “Place” programs of the previous exercise, and remove them by hand when they are on the table to avoid colliding with each other.



Create your program, follow the instruction of the palletizing function, add in the **Tool action** node a "**2FG Grip**" function and make the robot place the cubes in another place. To make the robot depalletize all the cube, check the "Program Loops Forever" option. Then you can test your program, set the general **speed at 30%** to have time to move away the cubes by hand and don't forget to stop the program when all cubes are moved.

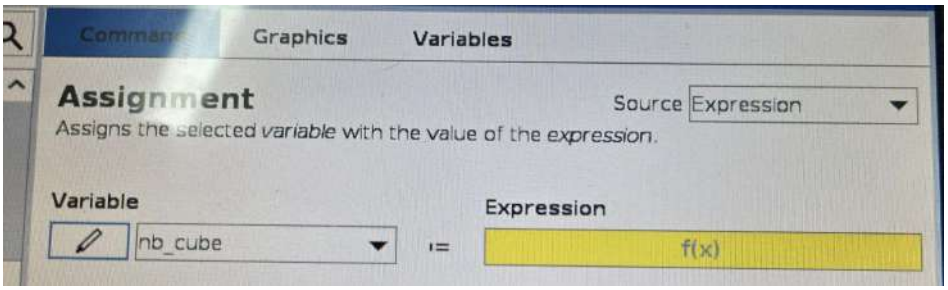
Your program should look like this one :



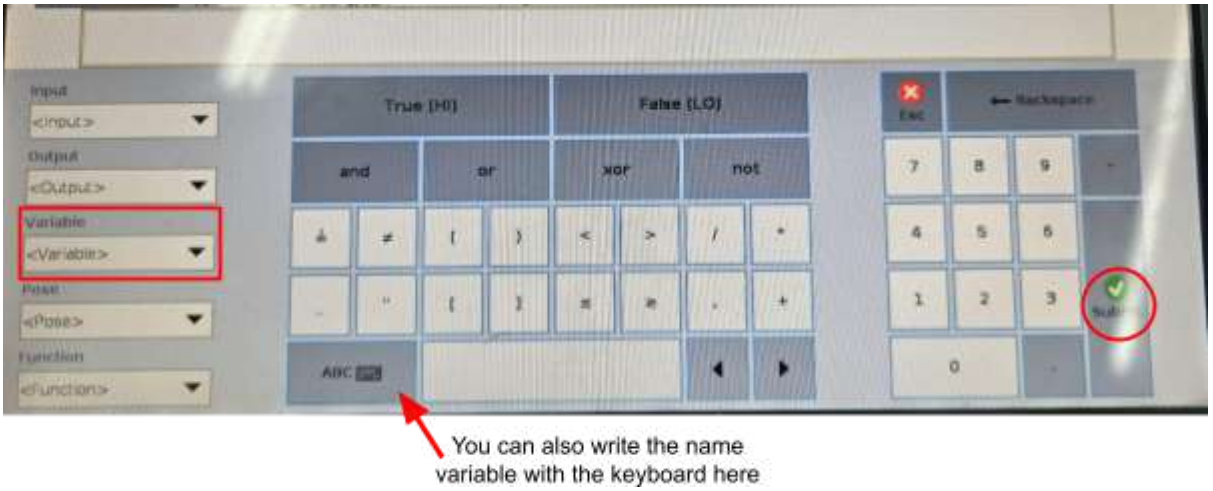
Introduce the variables :

Stopping the robot by pressing the stop button isn't very satisfying, we would like to automatically stop the program after a certain number of loops.

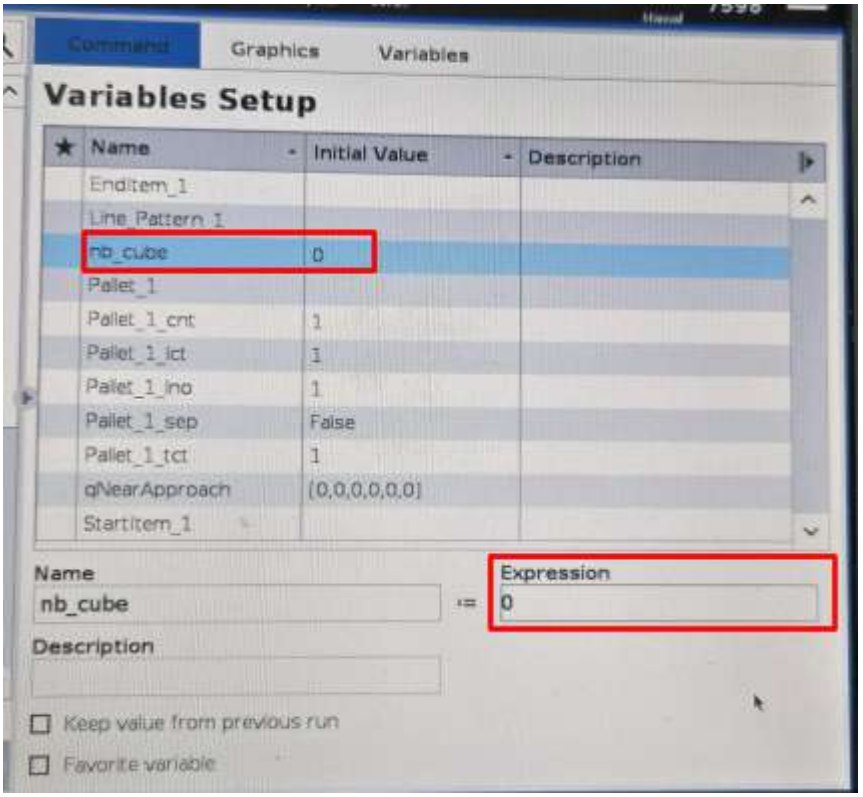
To do this we will create a variable that will keep the number of picked cubes while the program is running. In the Tool action node, add after the grip function an "Assignment" node that you will find in the "Advanced" menu on the left. In this node, rename the variable with something like "***nb_cube***" and click on the Expression space (the yellow rectangle).



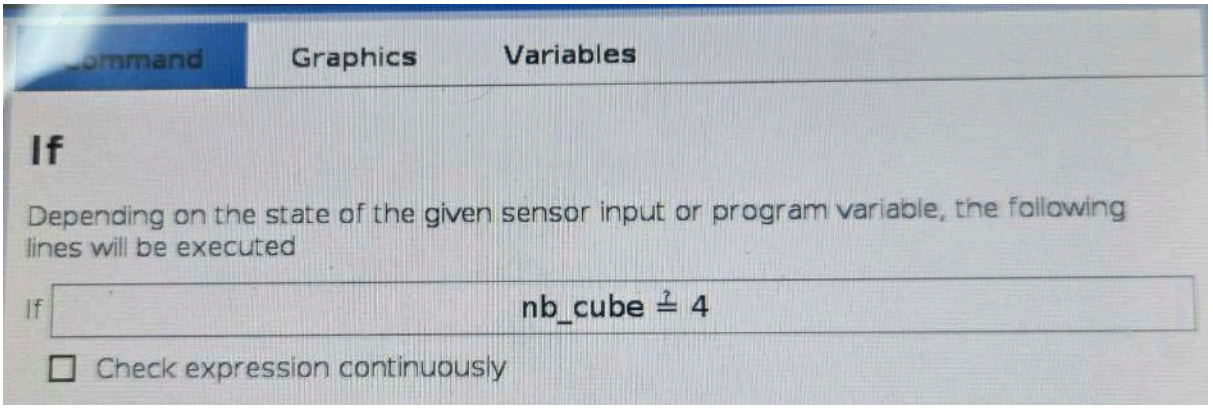
In this space, as the variable has to increase by one each loop, you will write "**nb_cube + 1**" (you can find the name of your variable in the "<Variable>" dropdown menu on the left). Click on "Submit" on the bottom right when it is done.



Once you define the variable expression, you have to define the initial value of your variable. To do this, go in the “Variables Setup” node at the top of your program, find your variable, click on it , set in the Expression space it appears “0”, to set the initial value to zero and submit.

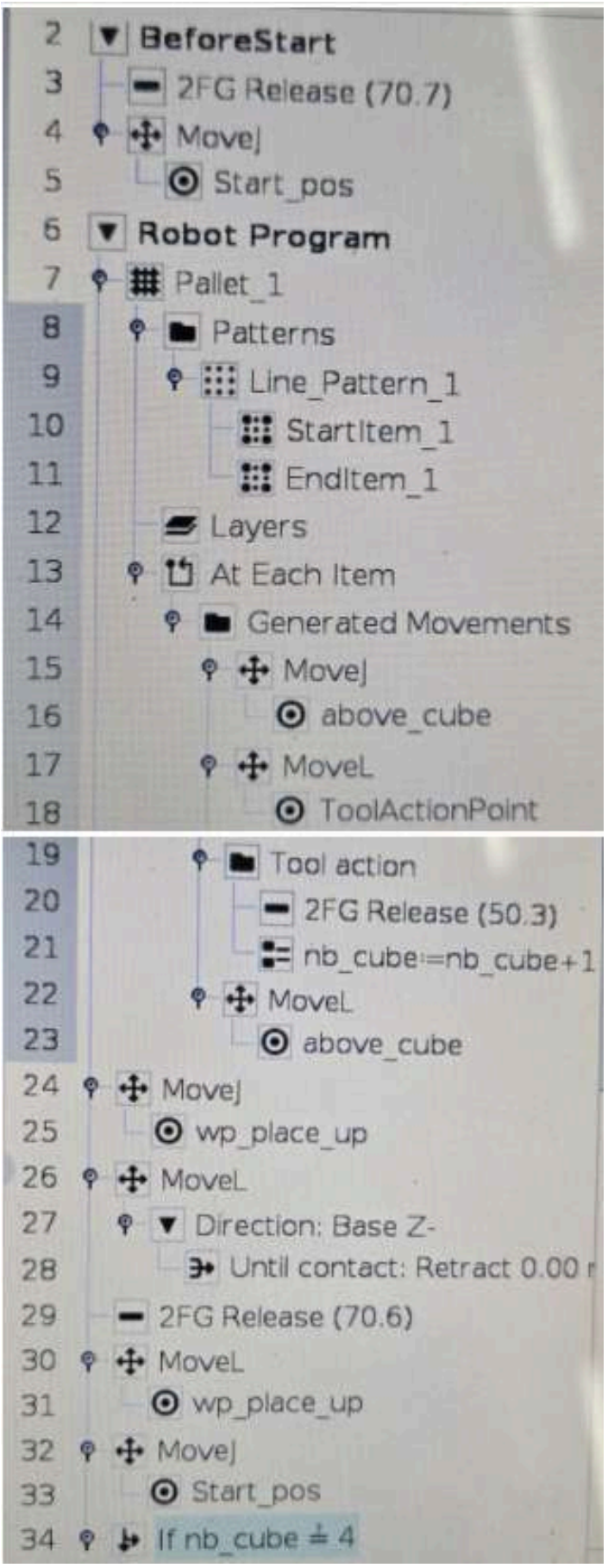


Now to stop the program when all the cubes have been taken, which is when the variable “*nb_cube*” equals 4, add an “If” node from the “Advanced” menu to the end of your program. In it, define the Expression as “**nb_cube ≟ 4**”, submit and add in the node below the If a “***Halt***” function. This will stop the program after the robot grips 4 cubes.

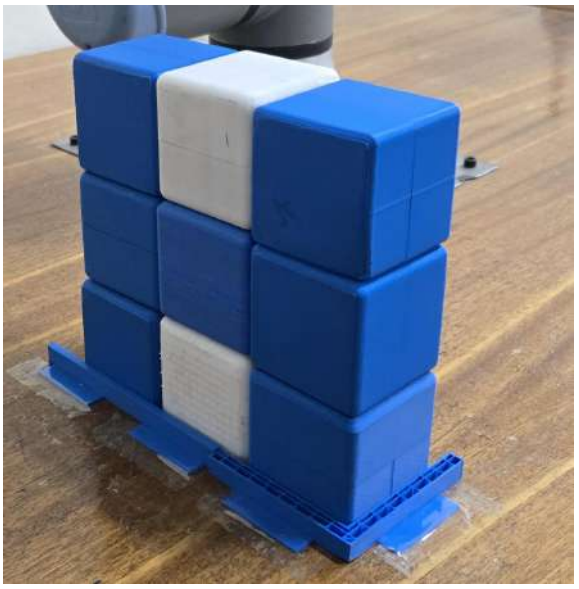


Test again your program. To verify the variable is correctly implemented, set in the If expression “**nb_cube ≟ 3**” to see what happens.

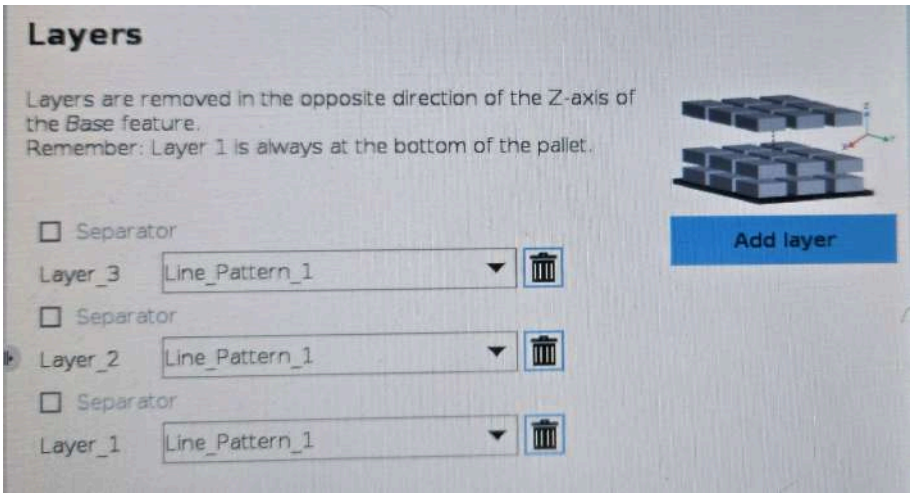
Your program should be similar to this one :



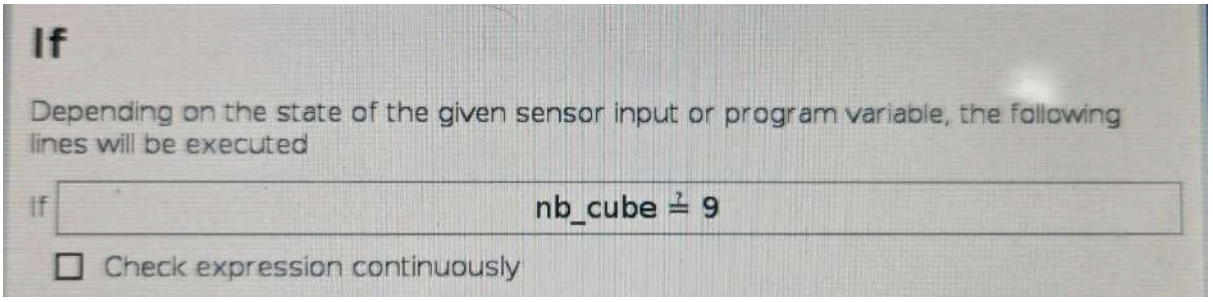
To go further with the Palletizing function, you will depalletize 9 cubes with the robot, organised in 3 layers of 3 cubes.



You can modify your previous code. Modify the EndItem_1 position, and if it is necessary the StartItem_1 position. Add in the “Layers” node 2 more layers, with the same line pattern than the first one.

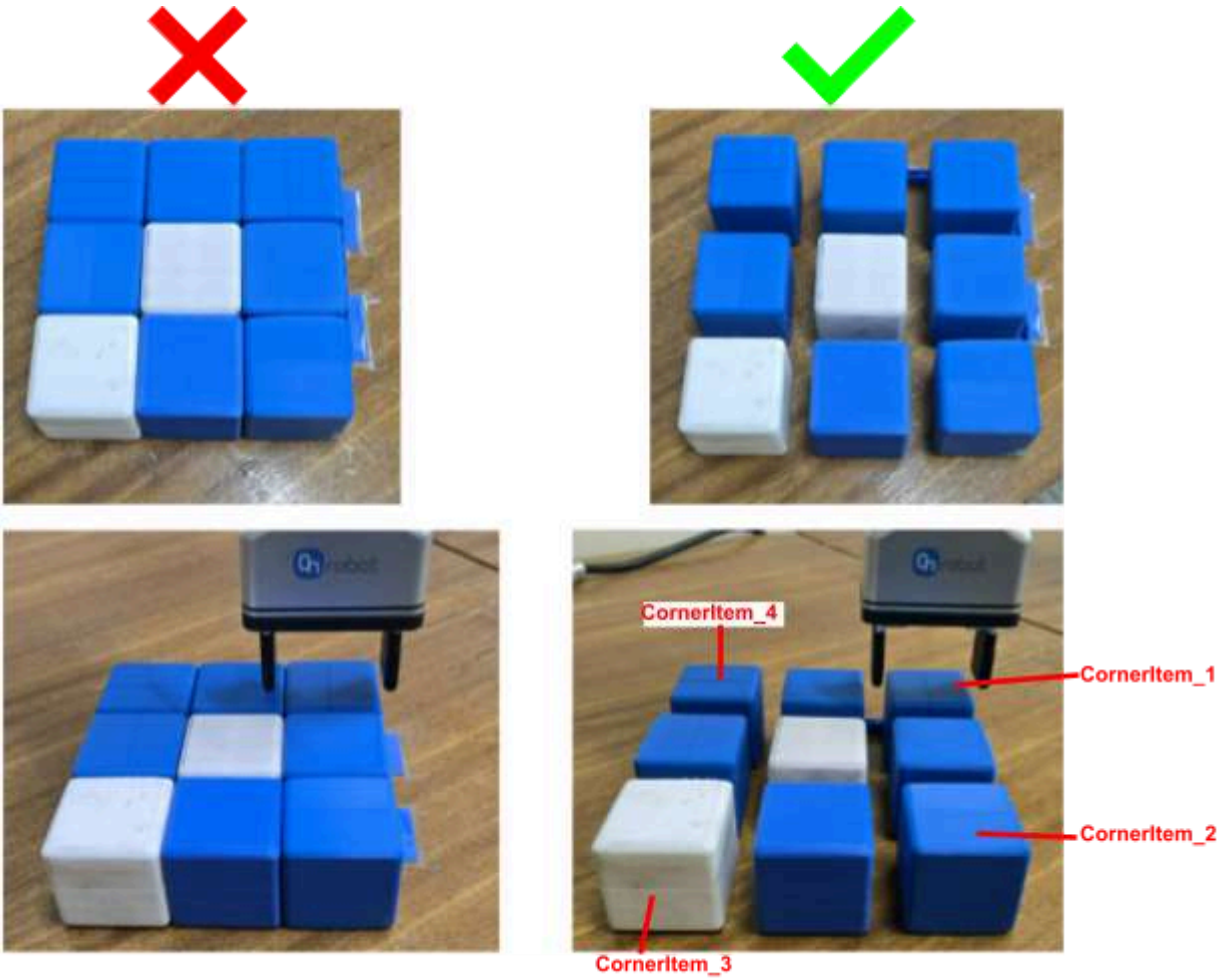


Don’t forget to also change the expression of the “If” function :



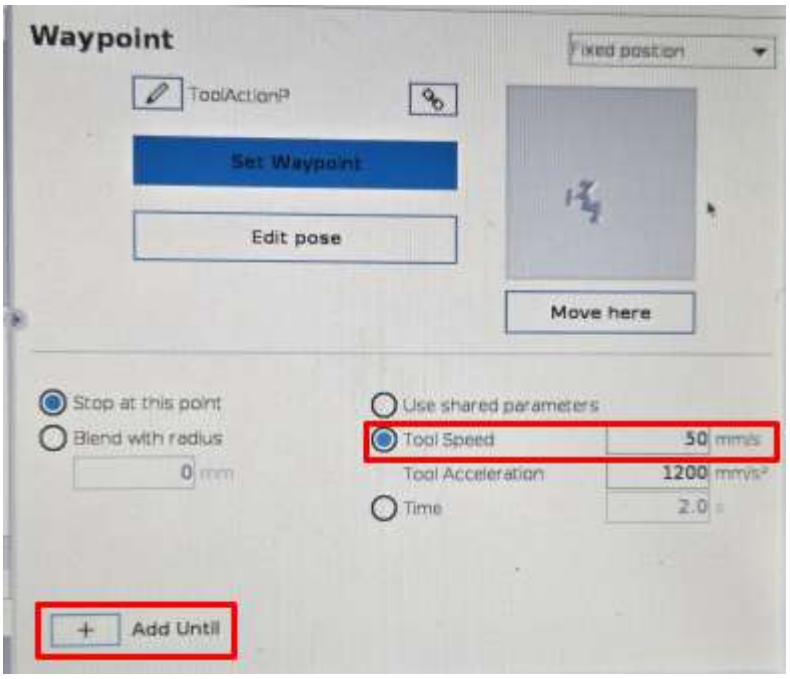
III. Placing with palletizing

Instead of placing the cubes in a simple way, we can palletize them. To be more familiar with the different palletizing pattern, we will use a 3x3 grid pattern to place the 9 cubes. To set the 4 corners positions, set the cubes in their final positions. Note that **the cubes must not touch each other**, so that the gripper can place them properly.

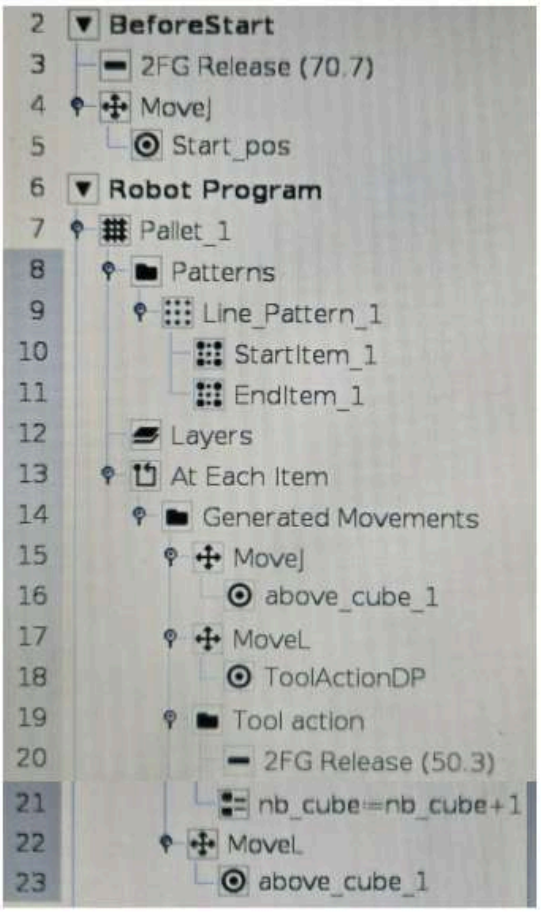


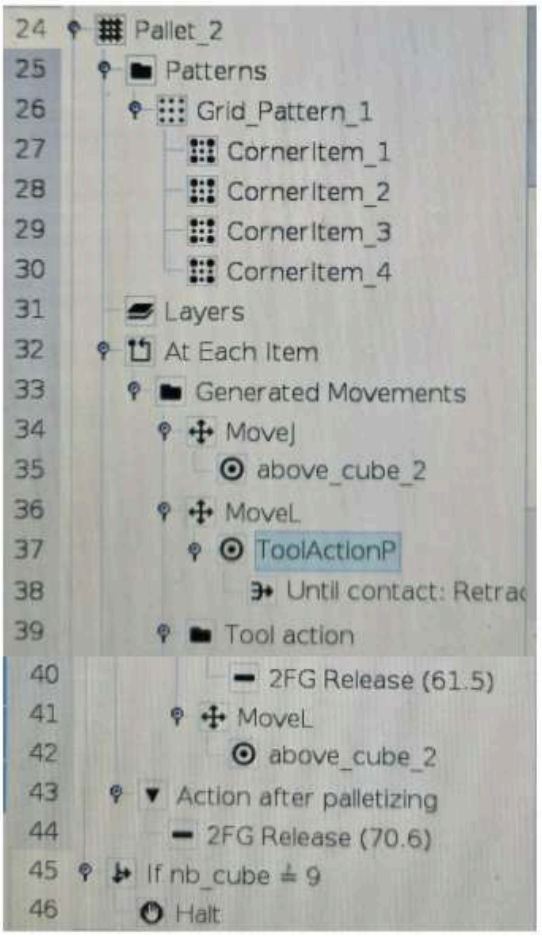
Modify your previous program, by replacing the placing part by a palletizing command. As the depalletizing function, follow the procedure to set all the waypoints. In the Tool Action node, add a “2FG Release” function, with a targeted width around **60 mm** (to avoid the clamp colliding with the already placed cubes) and in the “Pallet_2” node, select “Add action after palletizing” to add at the end of the palletizing function another release function. Set in it the targeted width to **70 mm** to open the gripper completely.

For security measure, set the speed of the Tool Action Point node to **50 mm/s**, add an “Until” node to it, and select “Tool contact” option.



Your program should look something like this :

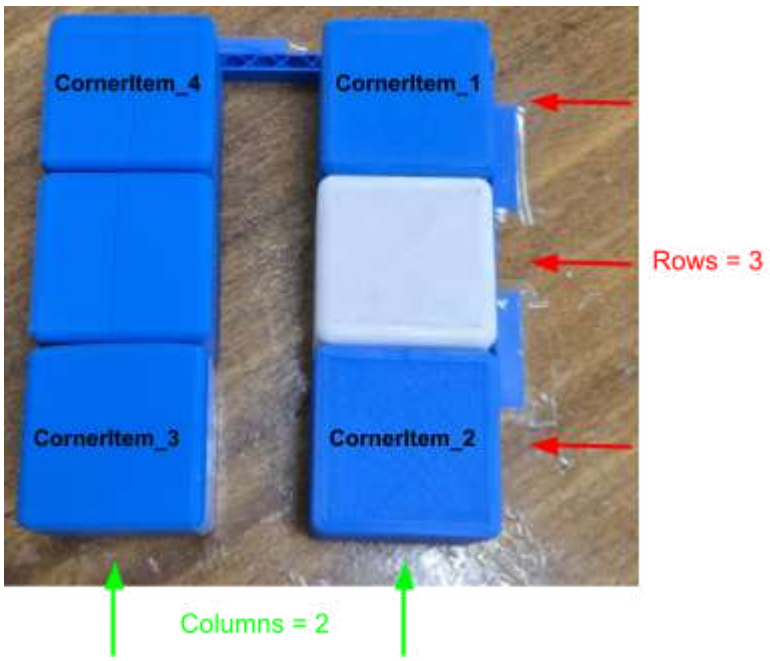




Test your program first at a general **speed of 50%**, then you can increase it.

Some tests you can try :

- Modify the “Pallet_2” function to place the cubes in a 2x3 pattern like in the image below. Define only one layer. What happens when you try to palletize more cubes than the pattern can contain ? What is the advantage of adding the tool contact node at the Tool Action Point of the palletizing function ?



What happens when you invert the number of rows and the number of columns ? (Set the general speed to 50% to test this one).

- Return to the initial configuration with the 3x3 single layer for palletizing. For each CornerItem, change the Tool Z-axe orientation (set the angle around 45° from the initial orientation). How does the robot act ? How does this change affect the other cubes' positions / orientations ?

- Returning to the initial configuration, what happens when you invert the CornerItem_2 position and the CornerItem_3 ? If it is necessary, remove by hand the cubes stacked on others when you run your script with this change.

Exercise 4 : Presence Detection by Material

This procedure aims to teach you how to use the cobot to detect pieces, identify their type (by the weight or the dimension) and then to pick and place objects according to their type.

How to make the robot do what you want without seeing his environment :

As you know, we will use the clamps and their capacity for the detection, or the “non detection” for being more precise. It is then possible to detect pieces without sensors.
You will learn two methods to do this :

- the “**no grip**”
- the “**free space**”

I. No Grip (to close in the empty space)

For the no grip, it is simple to understand, you move the gripper to a predefined position so that it can pick up an object (in this case a cube), and if the gripper ‘clamps down’ on empty space, it assumes that the position is empty at that moment.

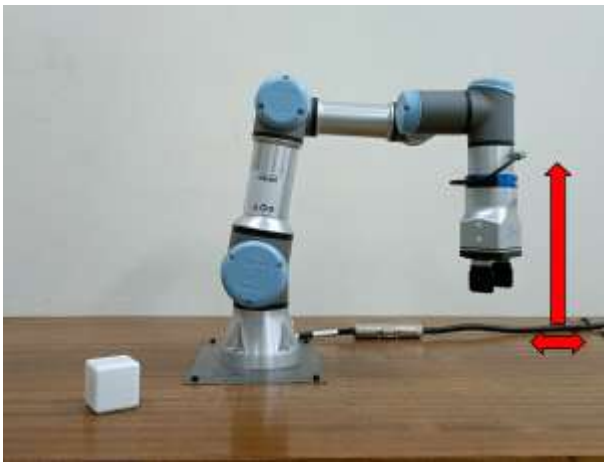
Here is an example of the paths and actions the robot must perform :



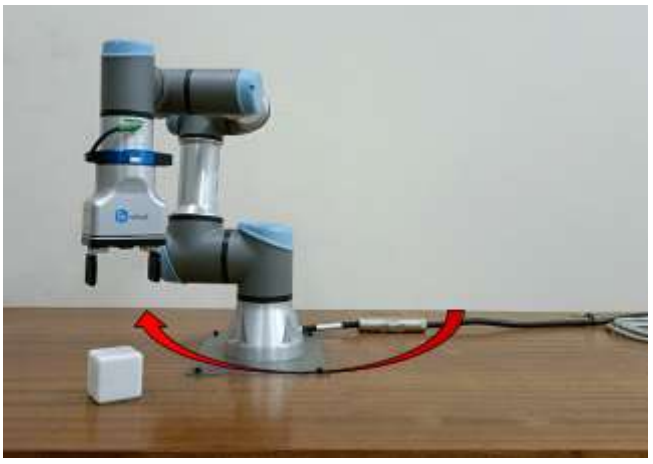
Initiale position



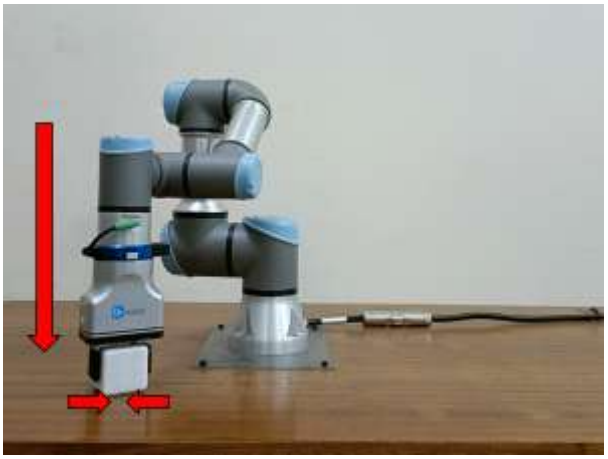
Descent and presence test



Nothing detected, open clamps



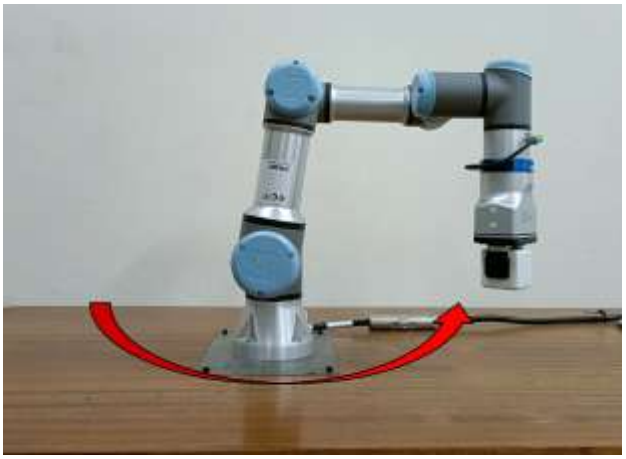
Change location



Pick up the cube



Straight up



Move to “*Waypoint1*”



Place the cube (caution distance)



Come back Initial position



Presence test



Cube detected : open, descend and close



Pick up and leave

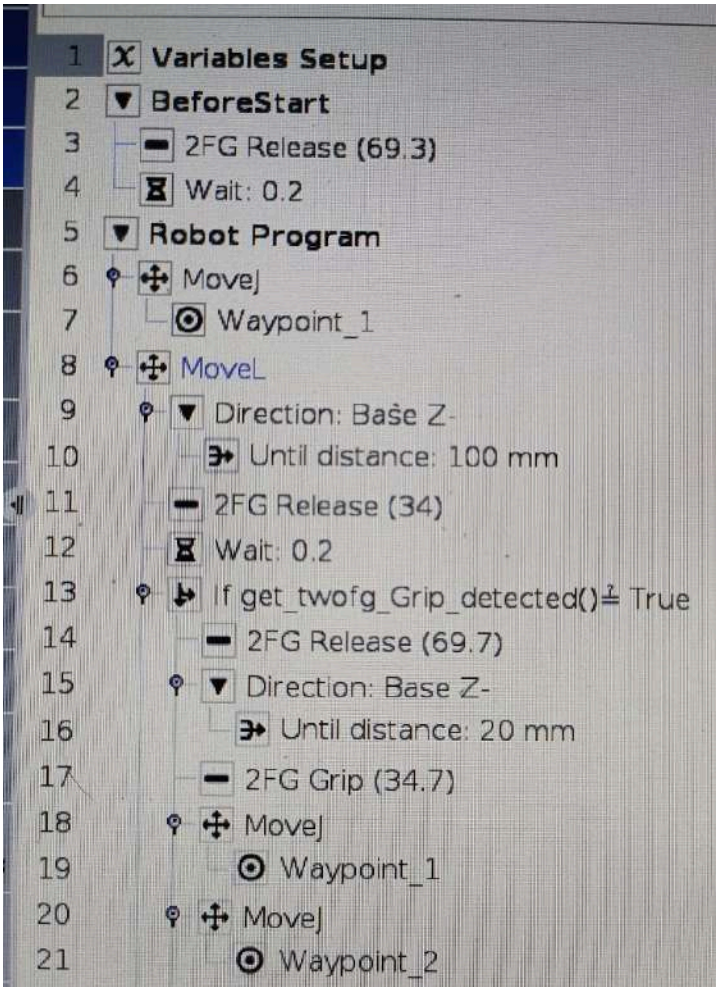


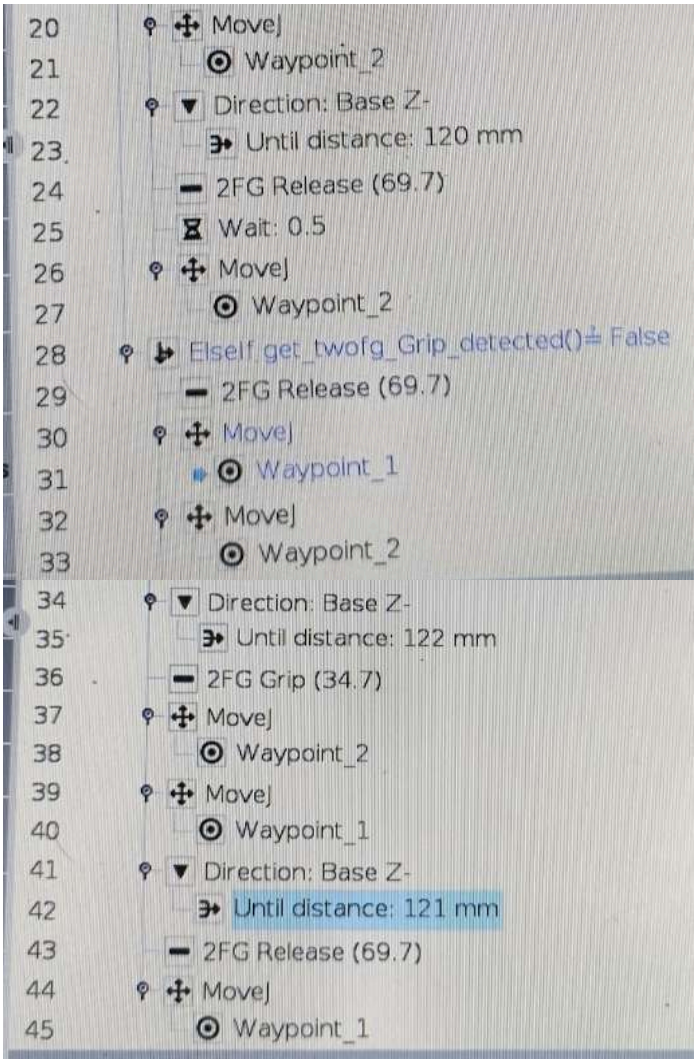
Place the cube again…



…in its initial position

Here is an example of a code that you can have :



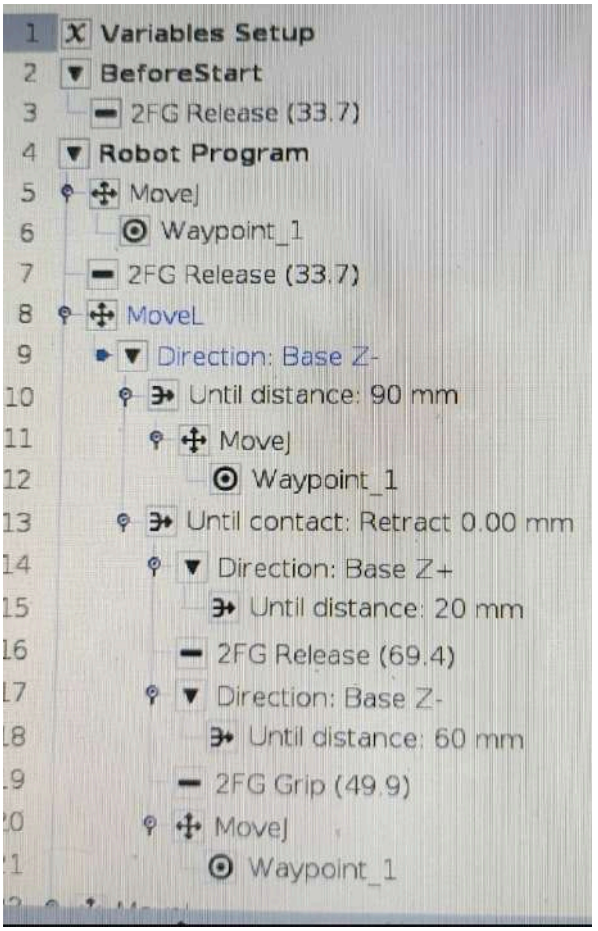


II. Free space (to go down without any resistance)

As for the “no grip”, the principle is simple: the gripper is moved down towards a predefined location so that it can “bump” into an object (in this case a cube), and if the gripper encounters no resistance, we conclude that this location is empty at that moment.

To practise, you can use the code shown here as a guide.

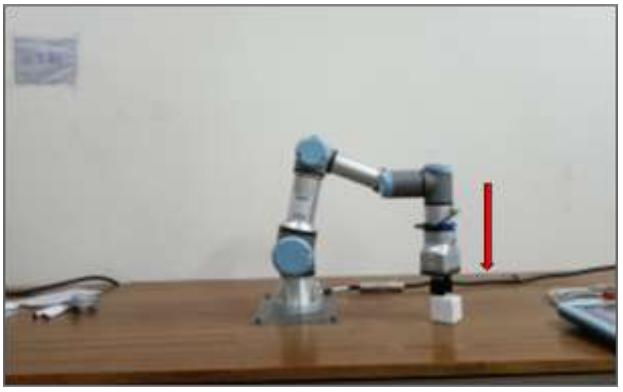
IMPORTANT
Waypoint = security trajectory
Choose between “**moveJ/moveL**”
Advised parameter for MoveL : speed = **90mm/s**
Release can be used for the detection if you invert the initial use (the distance between both fingers is reduced to **33.7 mm**



Here is an example of the paths and actions the robot must perform :



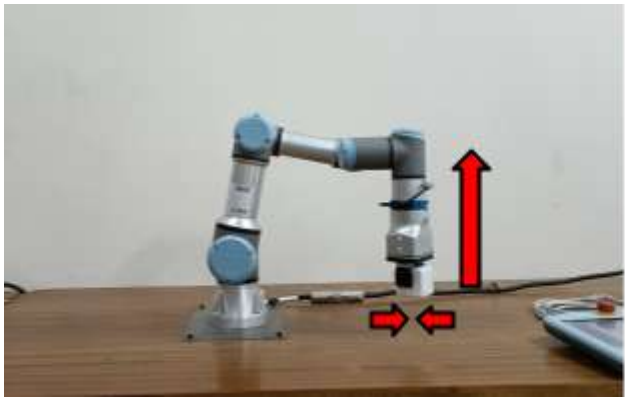
Initial position



Descent



The robot detects that it has come into contact with an object, moves up slightly to open the gripper, and then moves down again to pick up the cube.

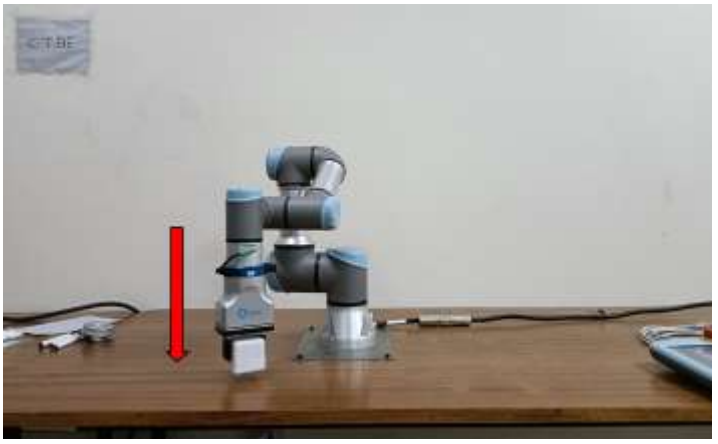


After moving back up, …



…it moves (with the cube) to the second position



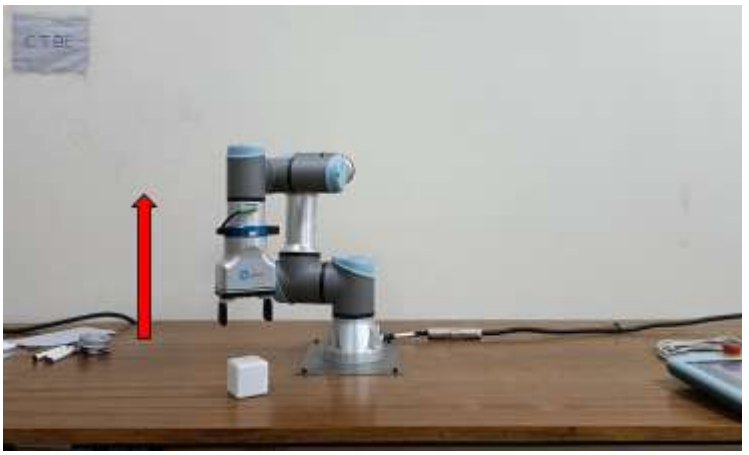


Once you have reached “Waypoint2” …

…you can lower the cube to place it down

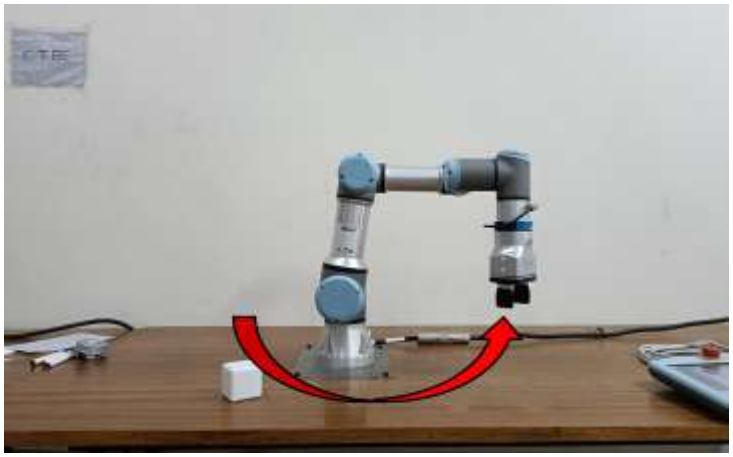
To do this, use as in exercise 2 the “Tool contact” method for placing cubes.

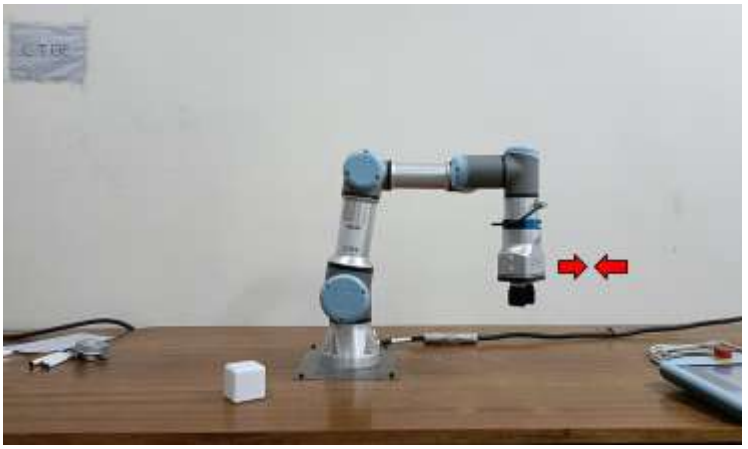




The robot then opens the gripper…

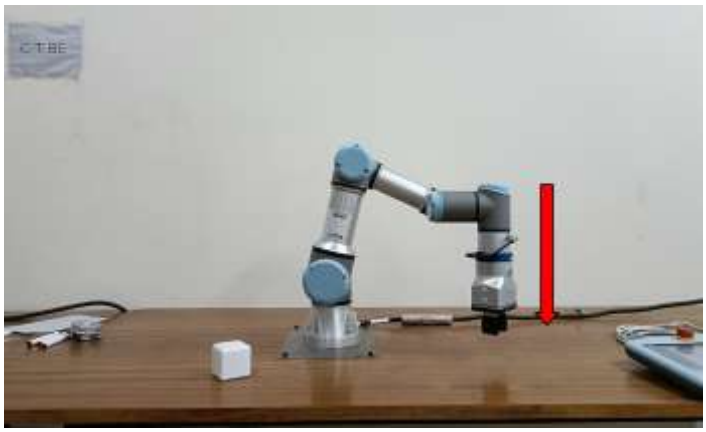
…and moves upwards

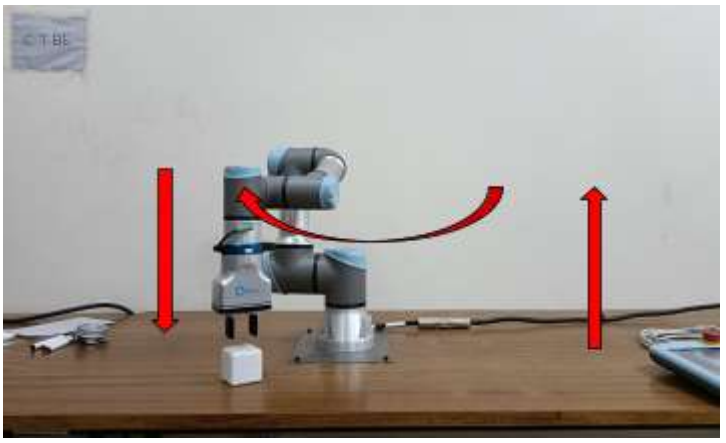




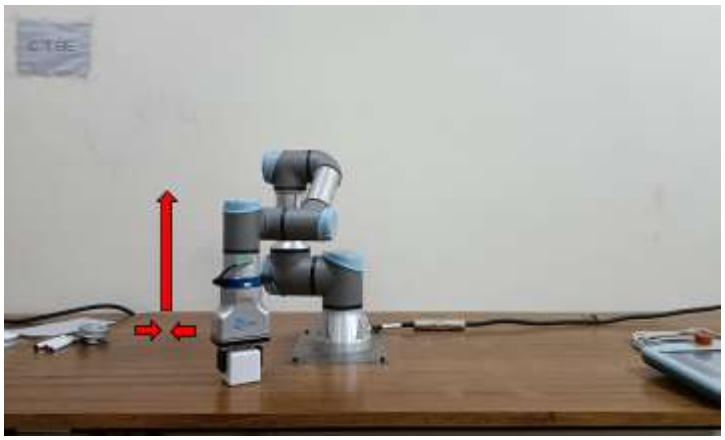
It goes back to “Waypoint 1” …

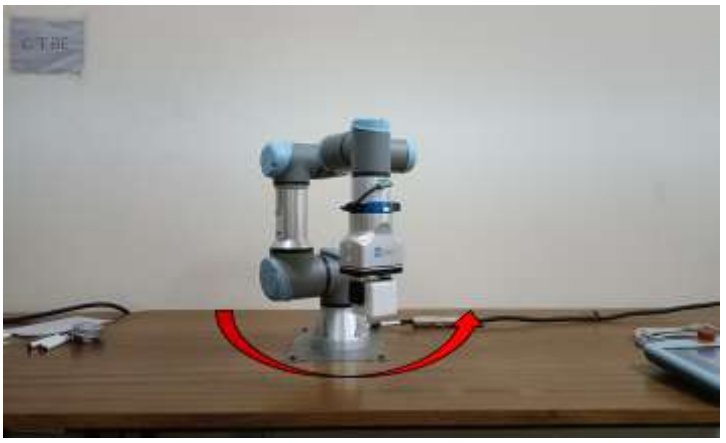
…and closes the clamp



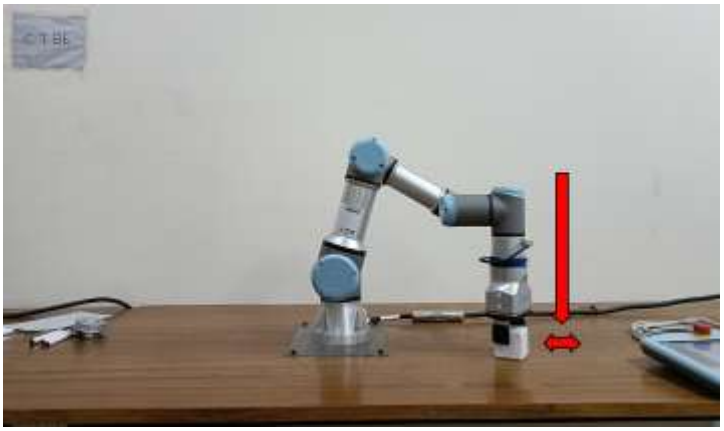


Then it goes down to check for the presence, since it doesn't detect anything, it checks at location 2 to see if the operation is complete and whether it can retrieve the part to return it to location 1.





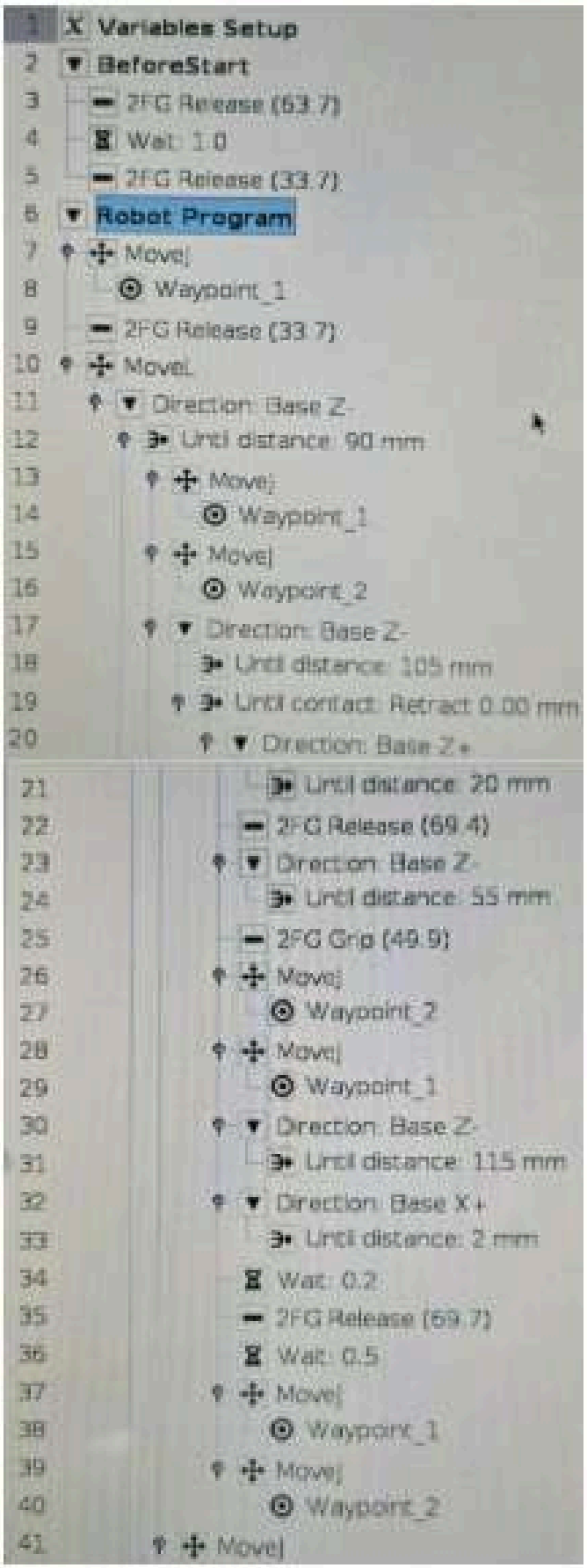


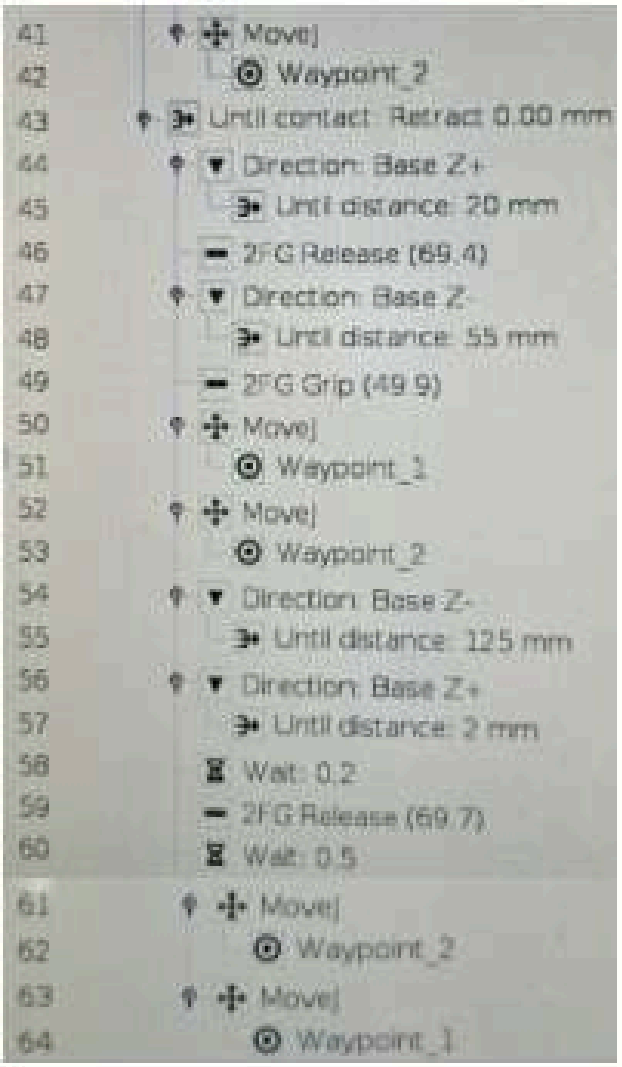


Finally, the robot completes its path by placing the cube down and opening the gripper.

BE CAREFUL with the distances, you can always set up a safety zone when the gripper is holding the cube to prevent collisions
Watch the demonstration on adding security settings :

EXAMPLE OF CODE :





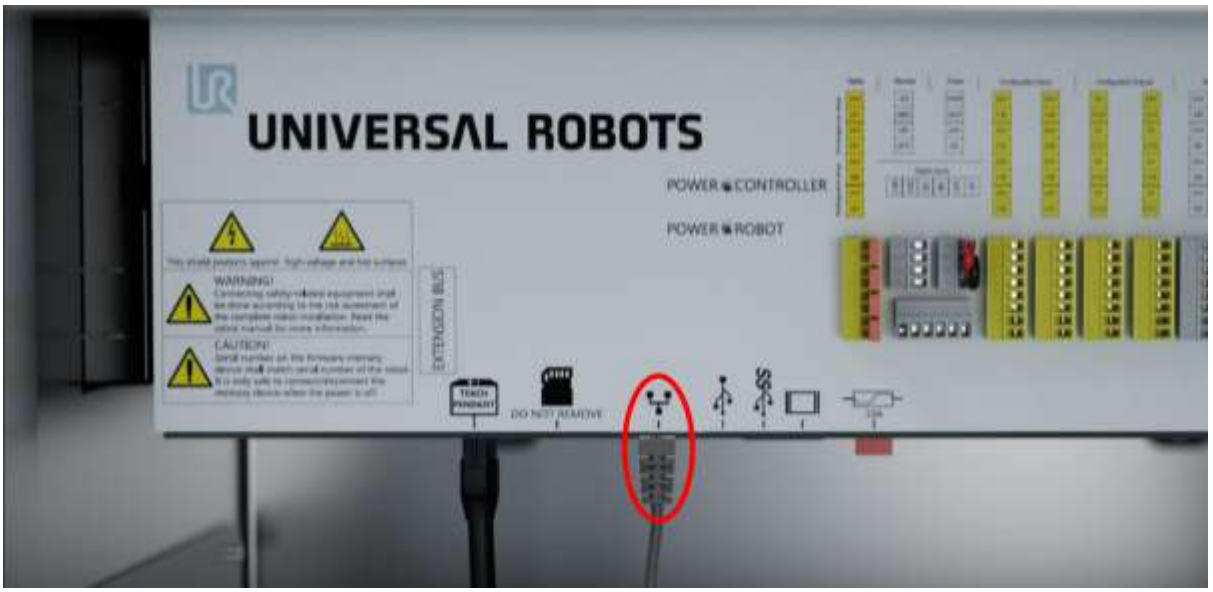
--

Exercise 5 : Communication between 2 cobots

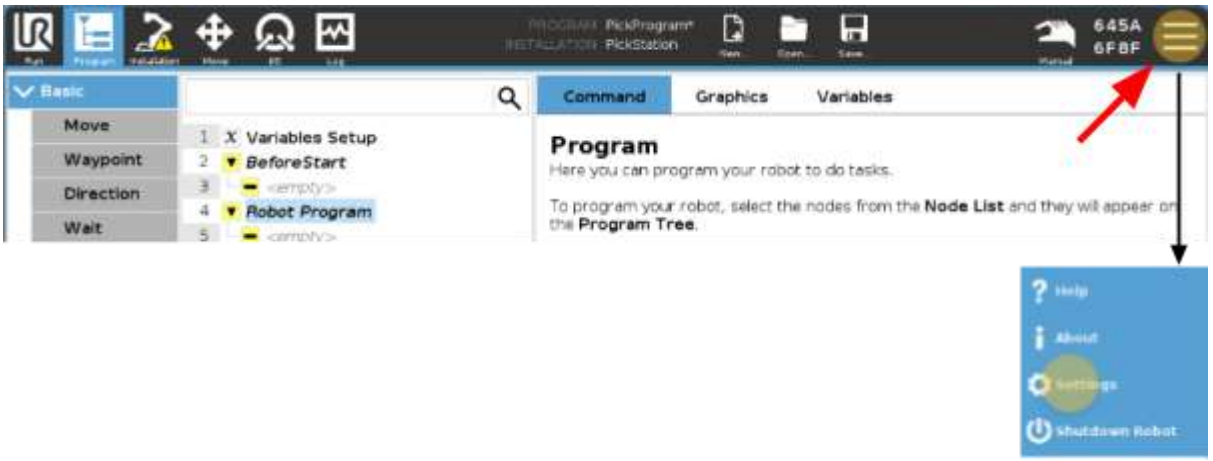
The aim of this exercise is to make use of the collaboration between the two robots to complete a loop with two cubes: a “pusher” cube and an “object” cube. The goal is for each robot to be able to send and receive information via an Ethernet cable (LAN) and react accordingly,before carrying out actions.

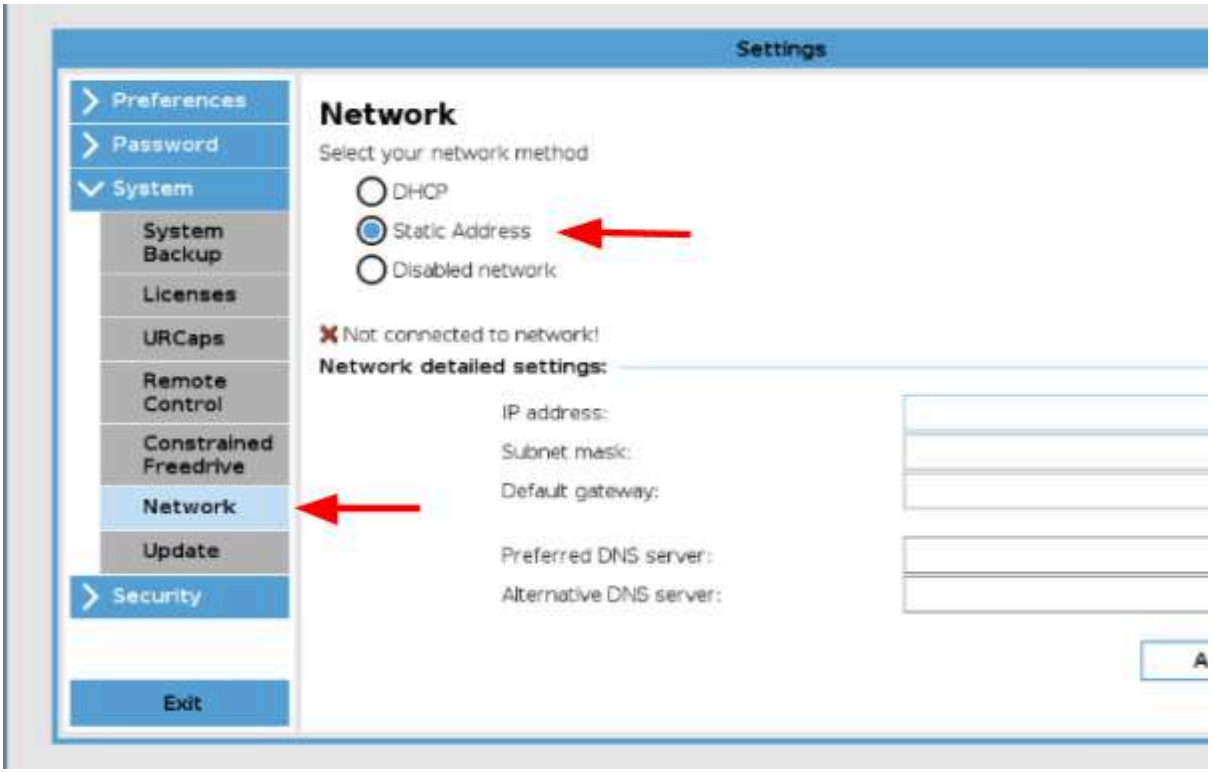
I. Establishing Communication

The communication between the two robots will be set by an ethernet cable that is plugged **in the two control boxes**.



Then the Network settings have to be defined. Check it is done on your robots by going in the Settings, and in the Network section :





For the robot of the right, you should see these settings :

IP adresse : **192.168.1.100**

Subnet masic : **255.255.255.0**

And for the robot of the left, you should see these settings :

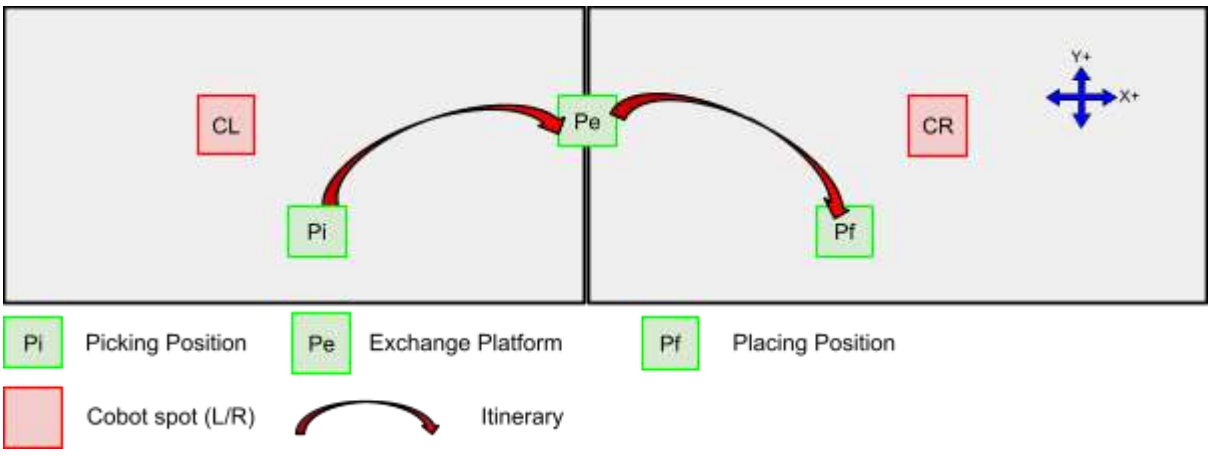
IP adresse : **192.168.1.101**

Subnet masic : **255.255.255.0**

Make sure the IP addresses are different.

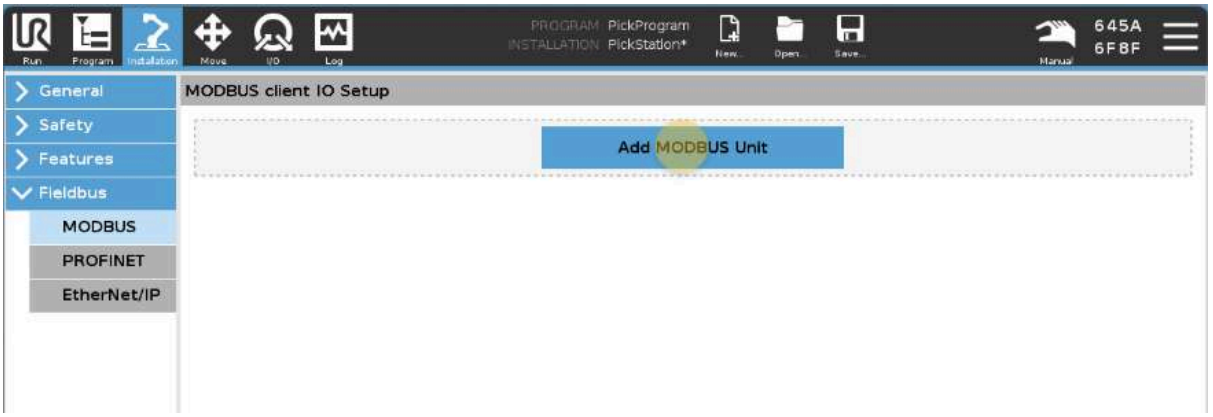
II. One-way Communication

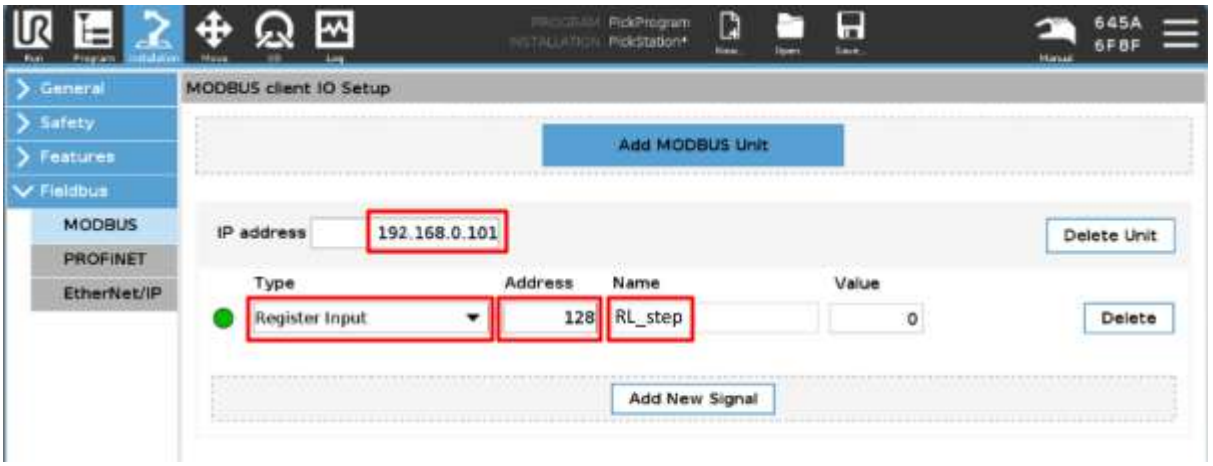
Now your robots are set up to communicate, you can write a first program in which one robot sends information to the other. So the robots will move a cube from one side to another, let's say from left to right, as it is schematized below :



A. Set up the right robot installation

The right robot (slave) will receive information by the left robot (master). To do this, go to the installation menu of the right robot, in the FieldBus section. In the MODBUS menu it opens, check the Modbus Unit has the right set up as the second image below. If there is no Modbus Unit, add one by clicking on “Add MODBUS Unit” and register all the information, as it is shown in the images below.





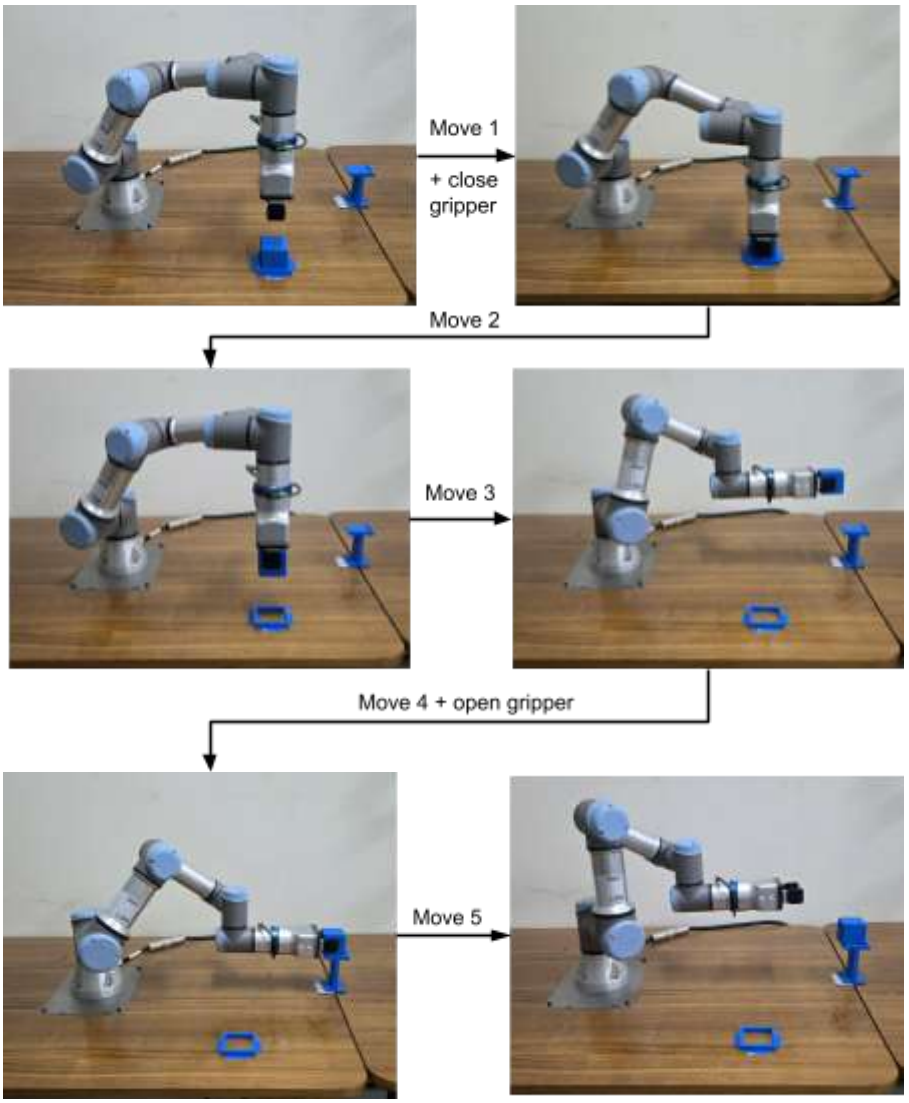
128 is a Modbus address reserved for general purpose information. You can find all the Modbus addresses description in the annexe, at the end of this exercise.

Now the right robot can receive information from the left robot, by reading the variable “RL_step”.

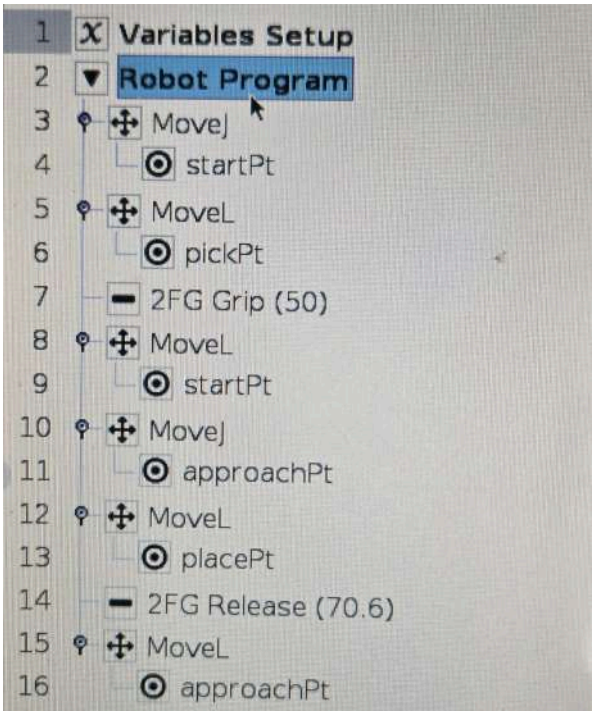
B. Programming the left robot

In this step, you will program a code for the left robot movement by adding commands to set the RL_step variable from the right robot. Here how to do this :

- First write a simple pick and place program with which the left robot moves a cube from an initial position to the exchange platform. See in the following images an example of movement the robot will do :



Your program should be similar as this one :



In this part of the exercise, the only information the right robot has to receive is when the left robot placed the cube on the exchange platform and let the space free around the cube so the right robot can pick it. In this way, **we will use, for the RL_step variable that corresponds to the 128 Modbus address variable, the number 0 to mean the right robot cannot pick the cube, and 1 to mean it can**.

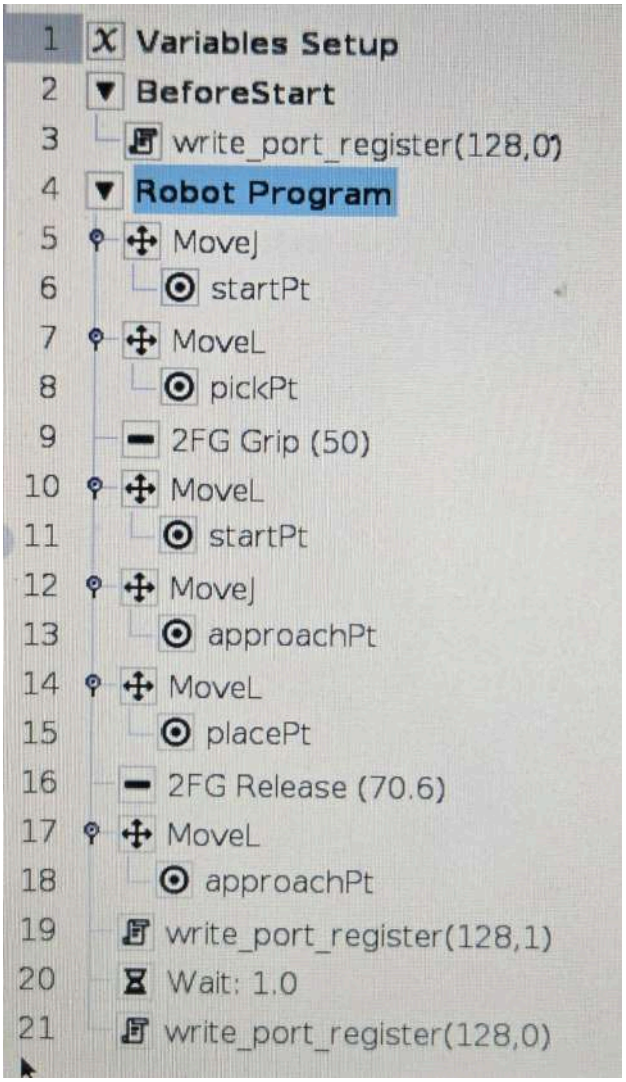
- After the robot returns to the approach point (approachPT, line 16 of the program), it must send information to the right robot to go picking the cube at the exchange platform. To do this, you will add a “Script” node (in Advanced menu) at the end of your program. In the text space, write manually (by clicking on 

at the bottom) this command : “**write_register_port(128,1)**”.
So when the robot arrives at this node, the value **1** will be sent on the ModBus address 128.

- However, in this way, this value will be constantly set at 1. So you have to initialize it at 0 at the beginning of the program by adding a “Script” node with the command : “**write_register_port(128,0)**”.
To do this, you can click on “Add Before Start Sequence” to add the script node before the program.
You can also, to be sure the value returns to zero at the end of the program, add a wait node and set the wait to 1 second, then add another “Script” node with the command : “**write_register_port(128,0)**”. In this way, after sending the value 1 to the right robot, it will return automatically at 0 for the next program execution.

- After creating your program, you can test it once, make sure the cube is well placed on the exchange platform and you can verify the value of the ModBus variable on the right robot teach pendant (see the image of the previous part).

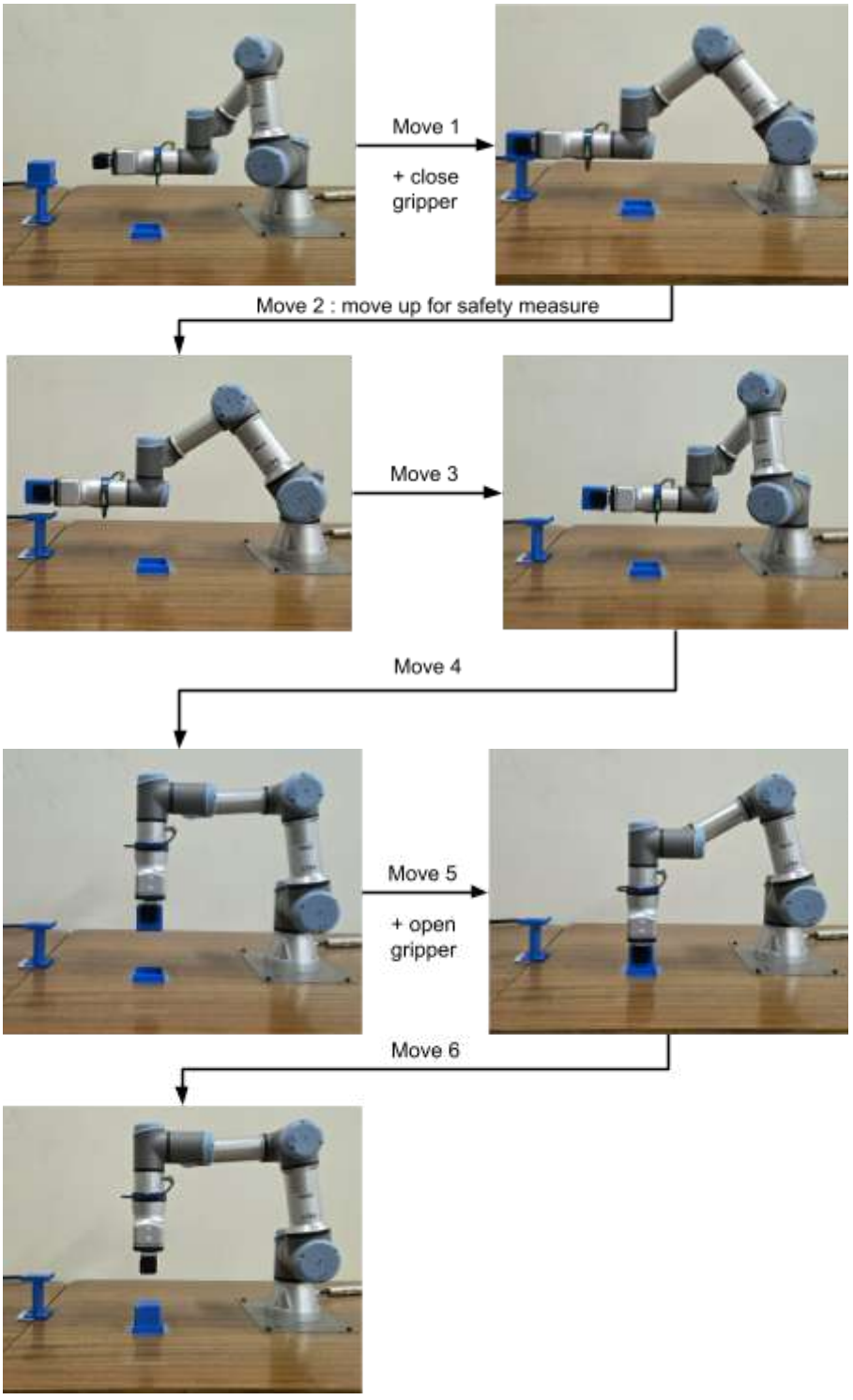
Your final program for the left robot should look like this one :



C. Programming the right robot

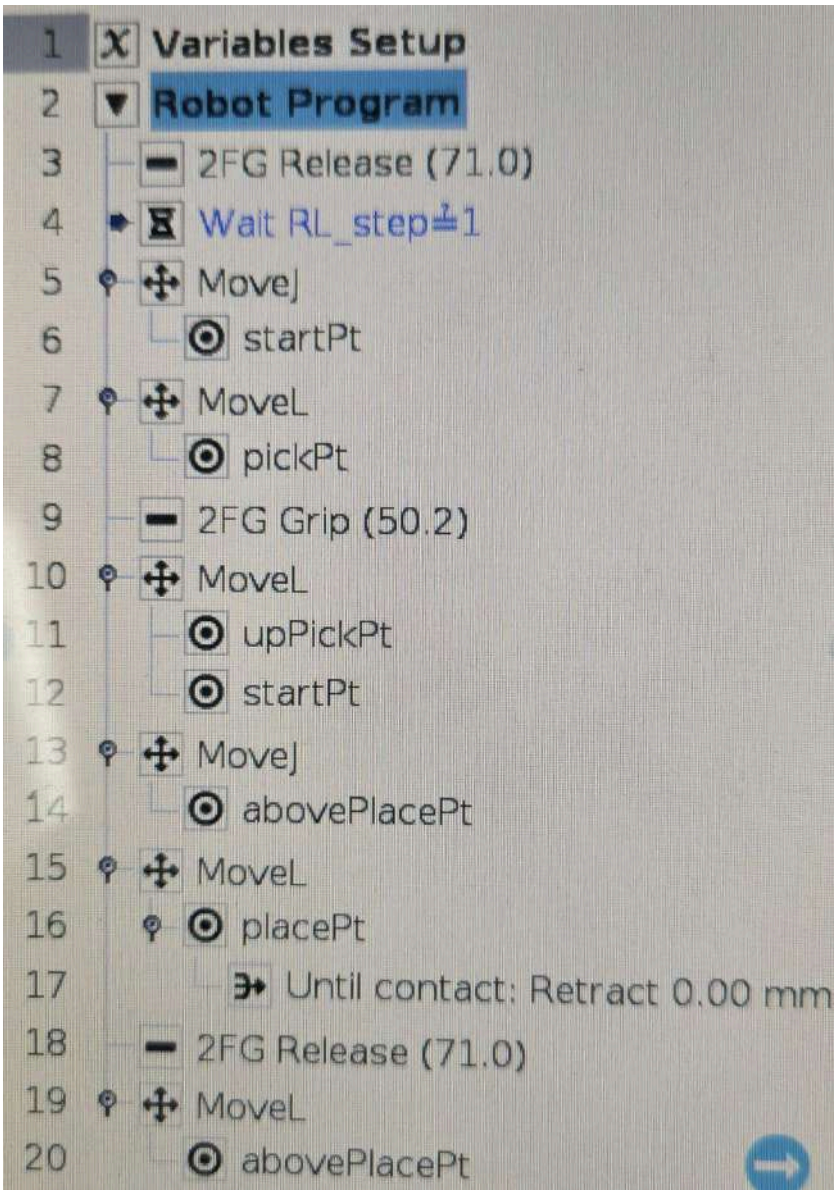
Now you will program the right robot movement. It will consist again in a Pick and Place program, but the robot has to wait until the left robot places the cube on the exchange platform. Here how to do it :

- Write your pick and place program, to make the right robot execute these type of movement :



- Then, at the beginning of your program, add a “Wait” node, in which you will select the expression option (last option) and write in it this expression : “**RL_step≟1**”. You will find the “RR_step” variable in the “Input” dropdown menu, on the left.
  In this way, the right robot will wait, before beginning its movement, that the left robot writes the value 1 in the 128 Modbus address variable, so at the end of the left robot program.

- Your program should look like this :



- Test only your right robot program first, with hiding the wait command by clicking on  . Next you can reactivate the wait node.
- Then test both programs by running **the right robot program first**. Verify the robot doesn’t move and then run the left robot program. Make sure the left robot places the cube on the exchange platform and only then the right robot begins to move.

III. Both-way Communication

Now that you know how to send information from one robot to another, you can make the information be sent in both ways.

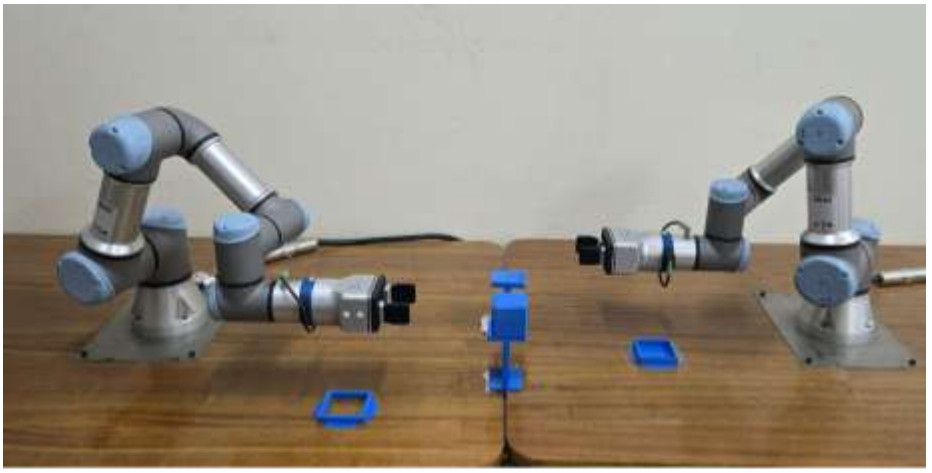
To do this, you just have to add a Modbus unit in the left robot installation menu, as made it in the right robot, but with another register address, let's use the address **129** :



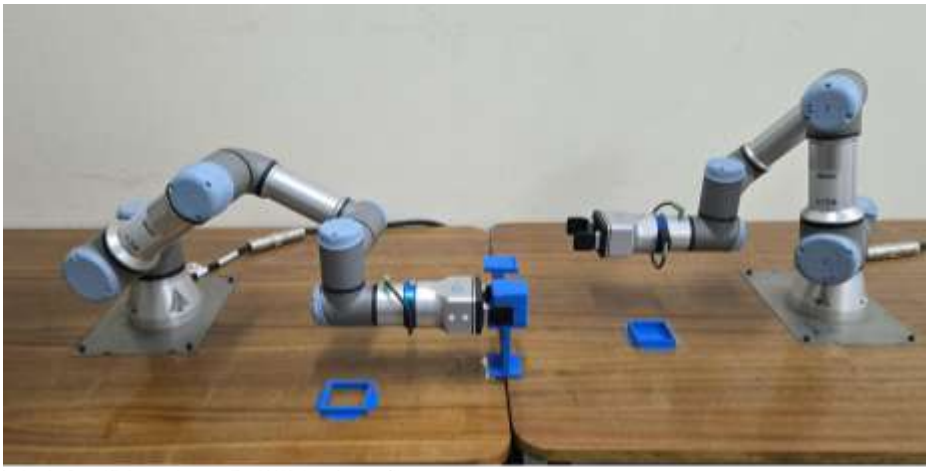
- Set in the left robot the correct IP address, Address, and Name in the Modbus unit.

Now each robot can send information to the other. To learn how to make robots communicate with each other, you will modify your two previous right and left robots programs so they can do these actions :

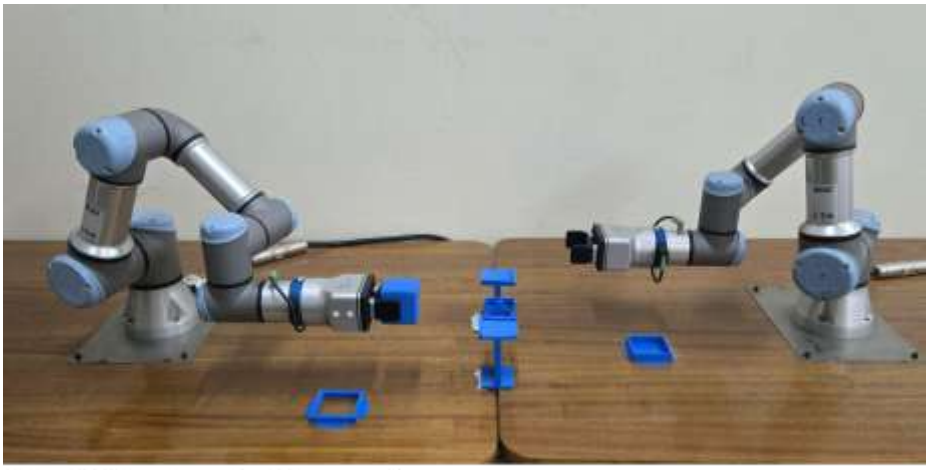
- Both robots will wait for the **other robot variable** to go **to 1** before going to pick the cube, and set their **own variable to 0** when they pick it.
- After placing the cube, both robots will set their **own variable to 1** and then wait for the **other robot variable** to go **to 0**.
- Here are the moves the robots will do. To do that you will need a second exchange platform :



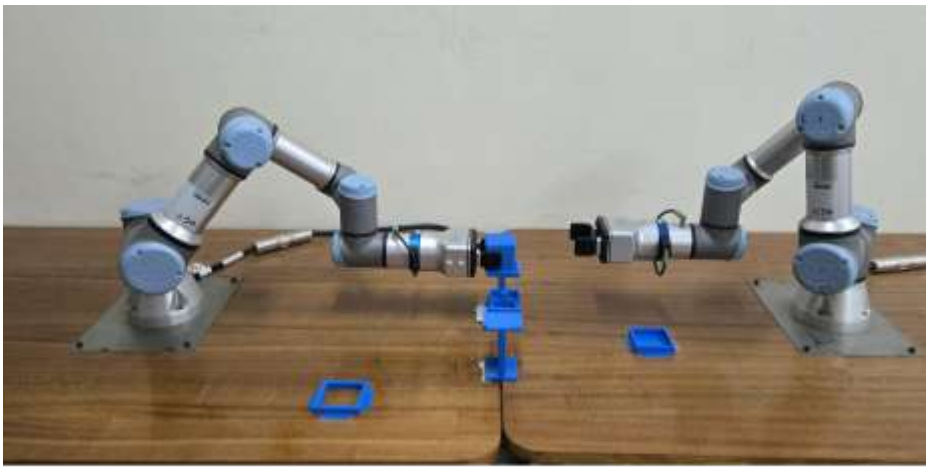
Step 0 : RL_step = 0 , RR_step = 1
Both robots are at their initial position, the right robot wait to RL_step = 1. The left robot wait to RR_step = 1, as it is the case, the robot will move in the next step.



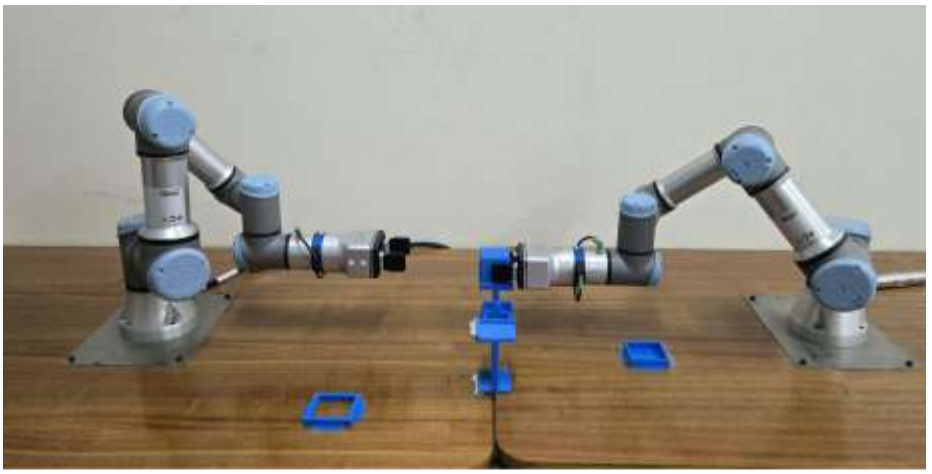
Step 1 : RL_step = 0 , RR_step = 1
The left robot is going to pick the cube on the first exchange station. To return to its initial position, it has to go up to not collide with the exchange platform.



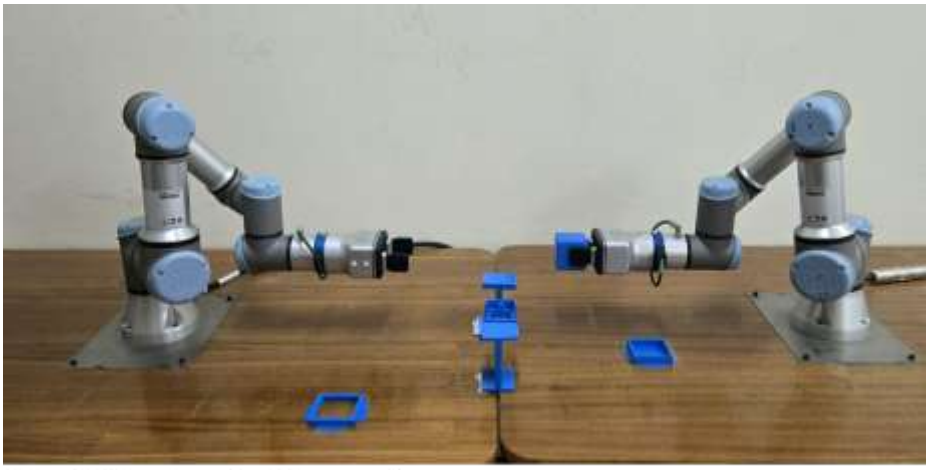
Step 2 : RL_step = 0 , RR_step = 1
The left robot return to its initial position with holding the cube to go to the second exchange station.



Step 3 : RL_step = 0 , RR_step = 1
The left robot goes to place the cube on the second exchange platform and will go back. At the end of this step, it will set the RL_step variable to **1.**



Step 4 : RL_step = 1 , RR_step = 1
The left robot wait for RR_step = **0**. The right robot go to pick the cube.



Step 5 : RL_step = 1 , RR_step = 1
The left robot wait again for RR_step = 0. The right robot go back with the cube and then set the RR_step variable to **0**.



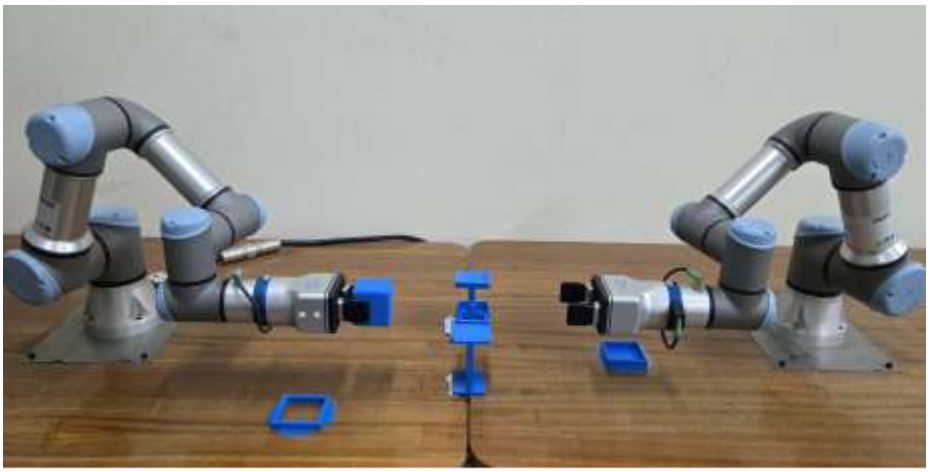
Step 6 : RL_step = 1 , RR_step = 0
Both robots go in front of the first exchange platform.



Step 7 : RL_step = 1 , RR_step = 0
The left robot wait for RR_step = 1. The right robot go to place the cube on the first exchange platform, go back and then set the RR_step variable to **1**.



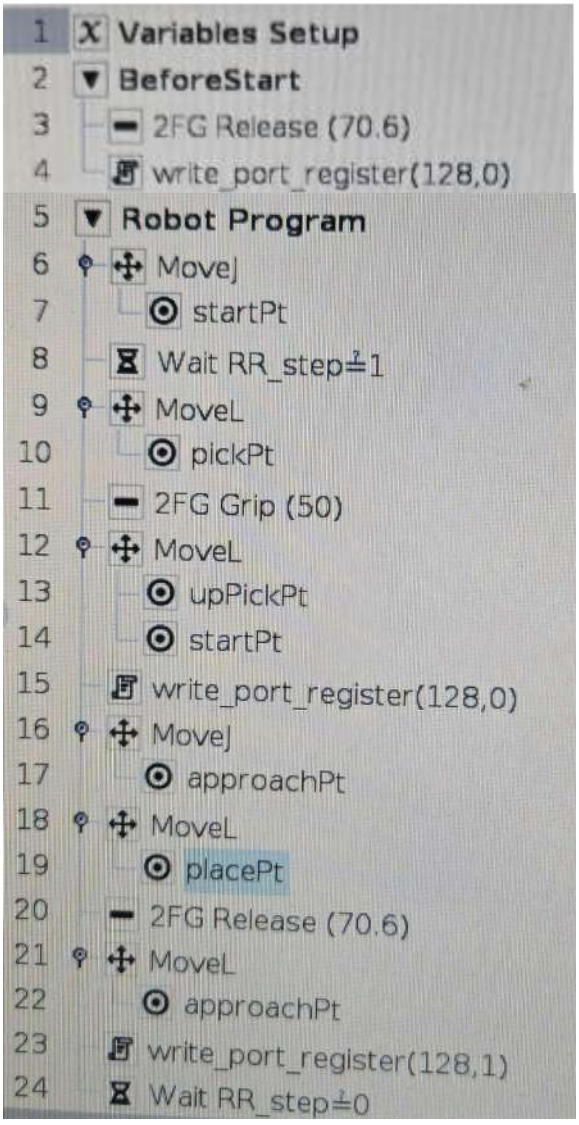
Step 8 : RL_step = 1 , RR_step = 1
The left robot go to pick the cube. The right robot wait for RL_step = 0.



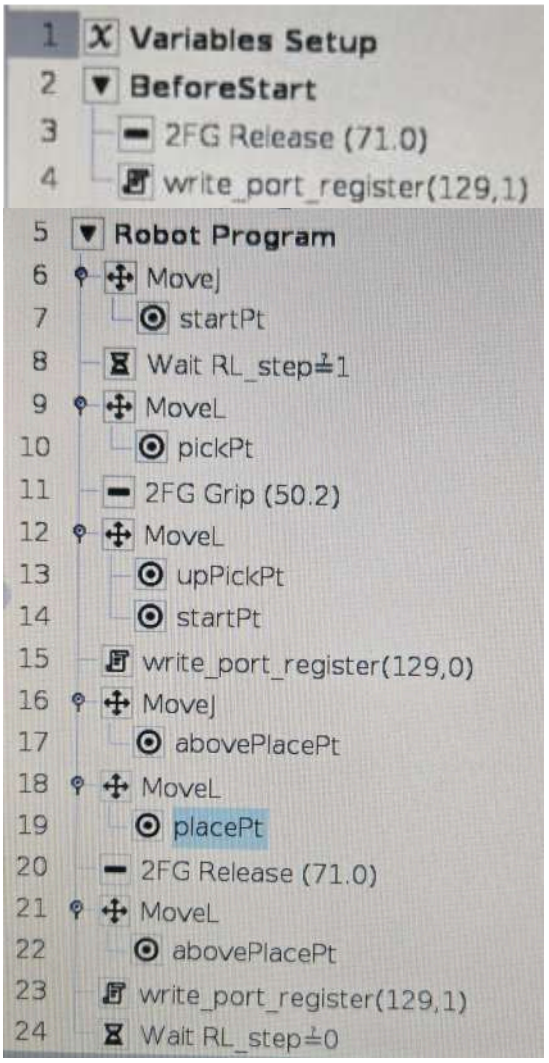
Step 9 : RL_step = 1 , RR_step = 1
The left robot go back, set the RL_step variable to **0**. Then both robots move to the second exchange platform, and another loop can begin

- Your programs should look like these ones :

Left robot program



Right robot program



- Test your programs, by running the right robot program first. Make sure the general speed of both robots are the same, otherwise the step and the variable changes may not be made in the right way.

ANNEXE : Modbus addresses signification:

REGISTER ADDRESS

R=Read, W=Write

Address	R	W	
0	*		Inputs, bits 0-15 [BBBBBBBBTTxxxxxx] x=undef, T=tool, B=box
1	*	*	Outputs, bits 0-15 [BBBBBBBBTTxxxxxx] x=undef, T=tool, B=box
2		*	SetOutputsBitsMask 0-15 [BBBBBBBBTTxxxxxx] x=undef, T=tool, B=box
3		*	ClearOutputsBitsMask 0-15 [BBBBBBBBTTxxxxxx] x=undef, T=tool, B=box
4	*		Analog input 0 (0-65535)
5	*	*	Analog input 0 domain e {0=current[mA], 1=voltage[mV]}
6	*		Analog input 1 (0-65535)
7	*	*	Analog input 1 domain e {0=current[mA], 1=voltage[mV]}
8	*		Analog input 2 (tool) (0-65535)
9	*	*	Analog input 2 (tool) domain e {0=current[mA], 1=voltage[mV]}
10	*		Analog input 3 (tool) (0-65535)
11	*	*	Analog input 3 (tool) range e {0=current[mA], 1=voltage[mV]}
16	*	*	Analog output 0 output (0-65535)
17	*	*	Analog output 0 output domain e {0=current[mA], 1=voltage[mV]}
18	*	*	Analog output 1 output (0-65535)
19	*	*	Analog output 1 output domain {0=current[mA], 1=voltage[mV]}
20	*	*	Tool output voltage (V) e {0V, 12V, 24V}
21	*		Tool digital input bits
22	*	*	Tool digital output bits
24	*		Euromap67 input bits (0-15)
25	*		Euromap67 input bits (16-32)
26	*		Euromap67 output bits (0-15) (read only!)
27	*		Euromap67 output bits (16-32) (read only!)
28	*		Euromap 24V voltage
29	*		Euromap 24V current
30	*		Configurable inputs, bits [BBBBBBBBxxxxxxxx] x=undef, T=tool, B=box
31	*	*	Configurable outputs, bits [BBBBBBBBxxxxxxxx] x=undef, T=tool, B=box
32		*	Bit mask configurable outputs, bits [BBBBBBBBxxxxxxxx] x=undef, T=tool, B=box
33		*	Clear configurable outputs, bits [BBBBBBBBxxxxxxxx] x=undef, T=tool, B=box
34-127			Reserved for future system variables
128-255	*	*	General purpose 16 bit registers
256-	*		Robot state
512-	*		Reserved
768-	*		Tool states
2048-	*		RT Machine control
Robot state			
256	*		Controller version high nuber
257	*		Controller version low number
258	*		Robot mode: CB3 and 3.1: Disconnected=0, Confirm_safety=1, Booting=2, Power_off=3, Power_on=4, Idle=5, Backdrive=6, Running=7 CB2: No_controller=-1, Running=0, Freedrive=1,Ready=2, Initializing=3, Security_stoppde=4, Emergency_stopped=5, Fault=6,
260	*		isPowerOnRobot
261	*		isSecurityStopped
262	*		isEmergencyStopped
263	*		isTeachButtonPressed
264	*		isPowerPuttonPressed
265	*		isSafetySignalSuchThatWeShouldStop
266	*		SafetyMode Refer to Primary Interface manual
270	*		Base joint angle (in mrad)
271	*		Shoulder joint angle (in mrad)
272	*		Elbow joint angle (in mrad)
273	*		Wrist1 joint angle (in mrad)
274	*		Wrist2 joint angle (in mrad)
275	*		Wrist3 joint angle (in mrad)

280	*		Base joint angle velocity (in mrad/s)
281	*		Shoulder joint angle velocity (in mrad/s)
282	*		Elbow joint angle velocity (in mrad/s)
283	*		Wrist1 joint angle velocity (in mrad/s)
284	*		Wrist2 joint angle velocity (in mrad/s)
285	*		Wrist3 joint angle velocity (in mrad/s)
290	*		Base joint current (in mA)
291	*		Shoulder joint current (in mA)
292	*		Elbow joint current (in mA)
293	*		Wrist1 joint current (in mA)
294	*		Wrist2 joint current (in mA)
295	*		Wrist3 joint current (in mA)
300	*		Base joint temperature (in C)
301	*		Shoulder joint temperature (in C)
302	*		Elbow joint temperature (in C)
303	*		Wrist1 joint temperature (in C)
304	*		Wrist2 joint temperature (in C)
305	*		Wrist3 joint temperature (in C)
Joint modes From version 1.7			
310	*		Base joint mode
311	*		Shoulder joint mode
312	*		Elbow joint mode
313	*		Wrist1 joint mode
314	*		Wrist2 joint mode
315	*		Wrist3 joint mode
320	*		Base joint revolution count (number of full turns, typically 0 or 1)
321	*		Shoulder joint revolution count
322	*		Elbow joint revolution count
323	*		Wrist1 joint revolution count
324	*		Wrist2 joint revolution count
325	*		Wrist3 joint revolution count
TCP			
400	*		TCP-x in tenth of mm (in base frame)
401	*		TCP-y in tenth of mm (in base frame)
402	*		TCP-z in tenth of mm (in base frame)
403	*		TCP-rx in mrad (in base frame)
404	*		TCP-ry in mrad (in base frame)
405	*		TCP-rz in mrad (in base frame)
410	*		TCP-x speed in mm/s (in base frame)
411	*		TCP-y speed in mm/s (in base frame)
412	*		TCP-z speed in mm/s (in base frame)
413	*		TCP-rx speed in mrad/s (in base frame)
414	*		TCP-ry speed in mrad/s (in base frame)
415	*		TCP-rz speed in mrad/s (in base frame)
420	*		TCP-x offset in mm (in tool frame)
421	*		TCP-y offset in mm (in tool frame)
422	*		TCP-z offset in mm (in tool frame)
423	*		TCP-rx offset in mrad (in tool frame)
424	*		TCP-ry offset in mrad (in tool frame)
425	*		TCP-rz offset in mrad (in tool frame)
450	*		Robot current (in mA)
451	*		I/O current (in mA)
768	*		Tool Mode Refer to Primary Interface manual
769	*		Tool temperature (in C)
770	*		Tool current (in mA)

Exercise 6 : Practical example : Simulation of a production line

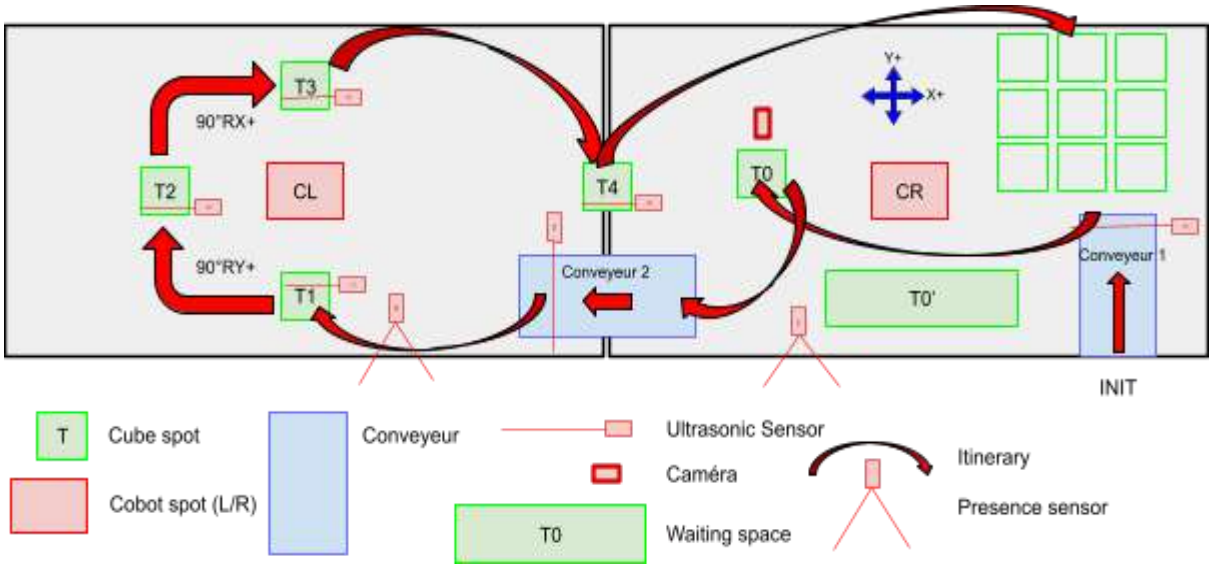
Main simulation : [DÉMO Collaboration et palettisation - Wokwi ESP32, STM32]

https://www.alldatasheet.com/datasheet-pdf/view/1132204/ETC2/HCSR04.html
https://docs.arduino.cc/resources/pinouts/A000067-full-pinout.pdf
https://wokwi.com/projects/373591091126947841
smart camera v2 - Wokwi ESP32, STM32, Arduino Simulator
Guide to NEMA 17 Stepper Motor Wiring - ElectronicsHacks
Arduino Mega seule : New Arduino Mega Project - Wokwi Simulator

Test Moteur Pap : https://wokwi.com/projects/472136968468663297

Concept of line production :

Visualising the process :



T0 : Orientation identification zone (####COLOR#### face) unprocessed Cube
T0' : Temporary storage area if Conveyor 2 is occupied (buffer zone)
T1 : Machining step 1 (duration = 5 sec)
T2 : Machining step 2 (duration = 35 sec)
T3 : Machining step 3 (duration = 15 sec)
T4 : Exchange zone Cube FINISHED (priority)

The objective of this exercise is to utilize the collaboration between the two robots and complete a path to the cubes, represented with one red face on each.
This diagram illustrates an automated parts production and transfer cell, organized into two zones connected by a conveyor, designed to optimize the flow of materials between two robotic stations.

The cycle begins in the INIT position, where a raw cube arrives on the infeed conveyor and is directed to zone T0, dedicated to identification and orientation. Here, a camera verifies that the red face of the cube is correctly oriented upwards; if necessary, this information is used to ensure proper manufacturing subsequently. Once validated, the cube is transferred to conveyor 2, which transports it to the left-hand cell.
The CL cobot then takes charge of the part and initiates a sequential production loop: a three-step manufacturing loop: the cube passes successively through (with controlled orientation changes (90° rotations around the specified axes)) :

- T1 (machining step 1, **5 s**)
- T2 (machining step 2, **45 s**)
- T3 (machining step 3, **15 s**)
- T4 (exchange platform)

After these operations, the cube is considered finished and deposited in T4, which serves as an exchange zone between the right-hand and left-hand cell. The cube is then picked up *(and directly stored at T0')*.The CR cobot then retrieves the finished cubes and places them in a grid-organized palletizing area, thus completing a closed cycle combining initial quality control, multi-stage processing, inter-cell transfer, and final storage.

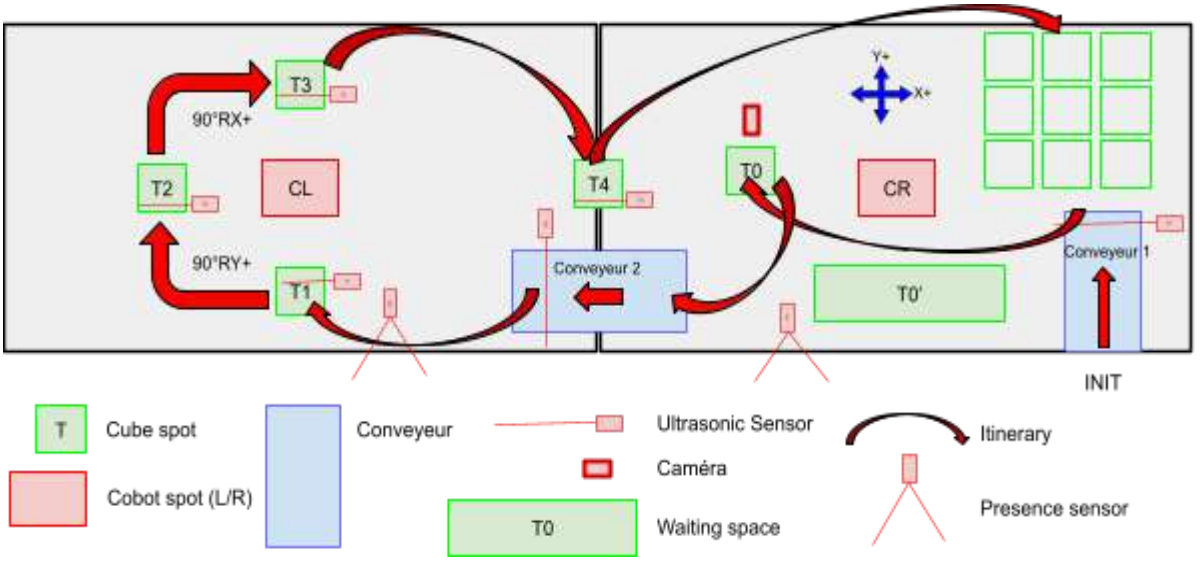
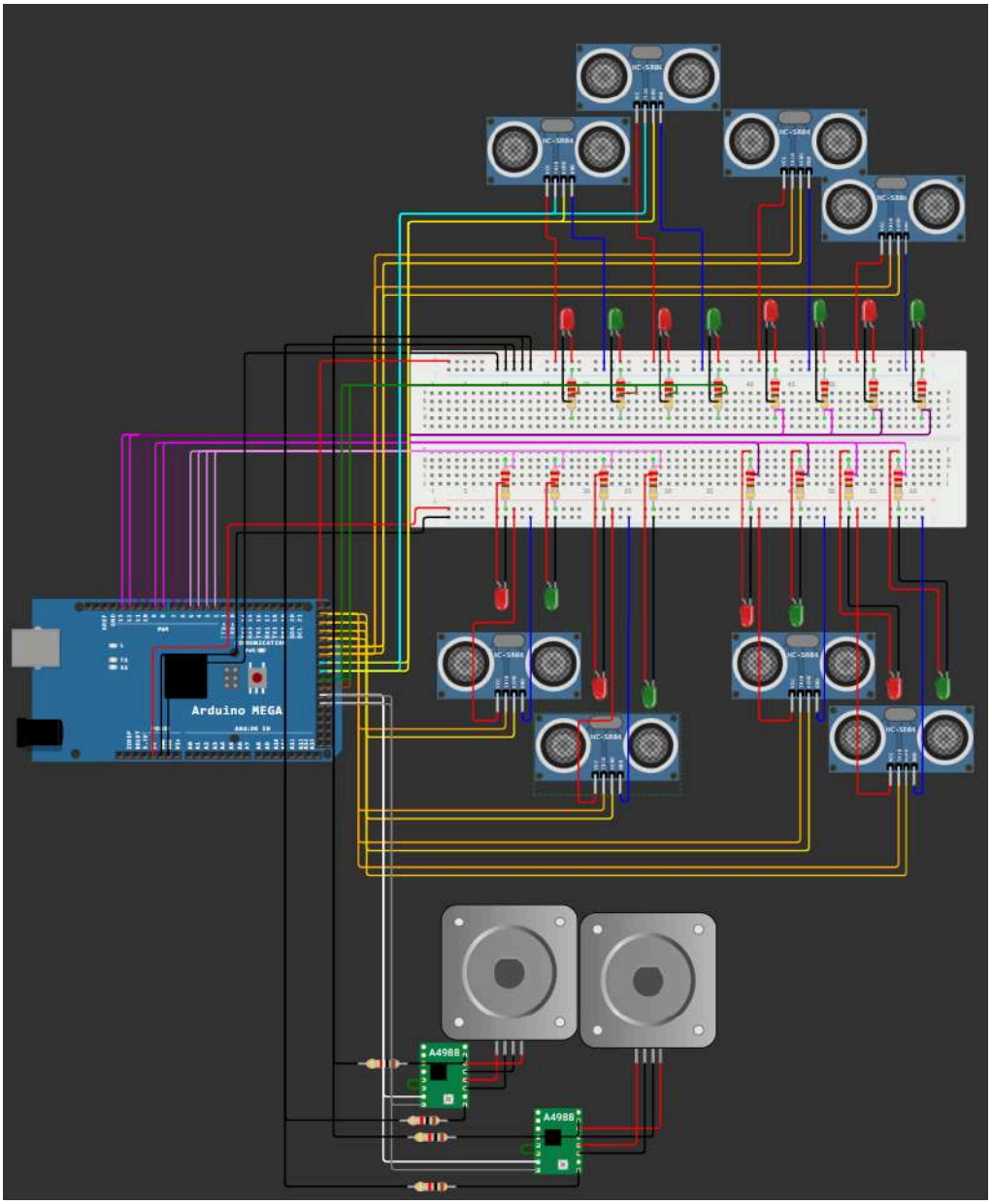
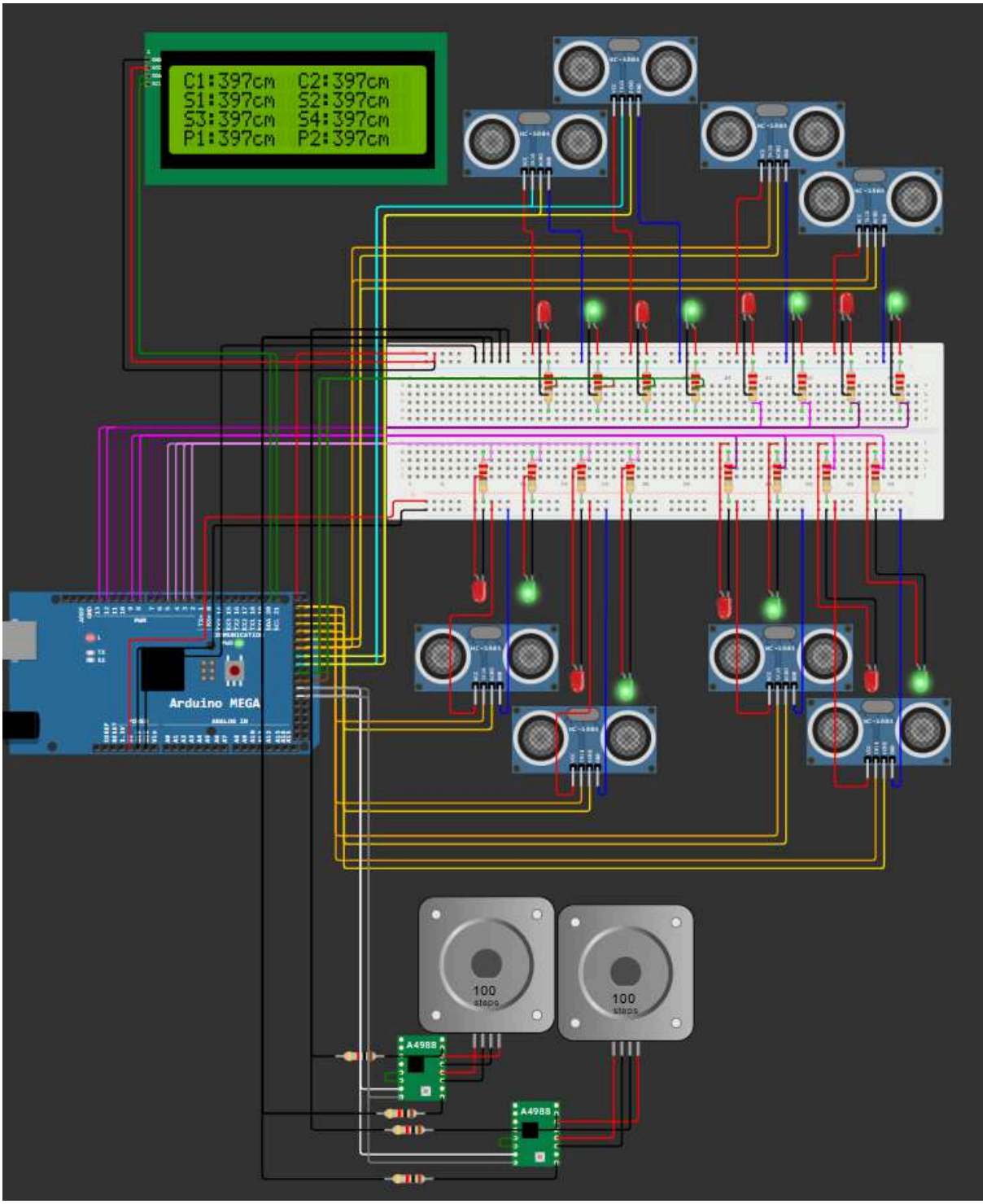
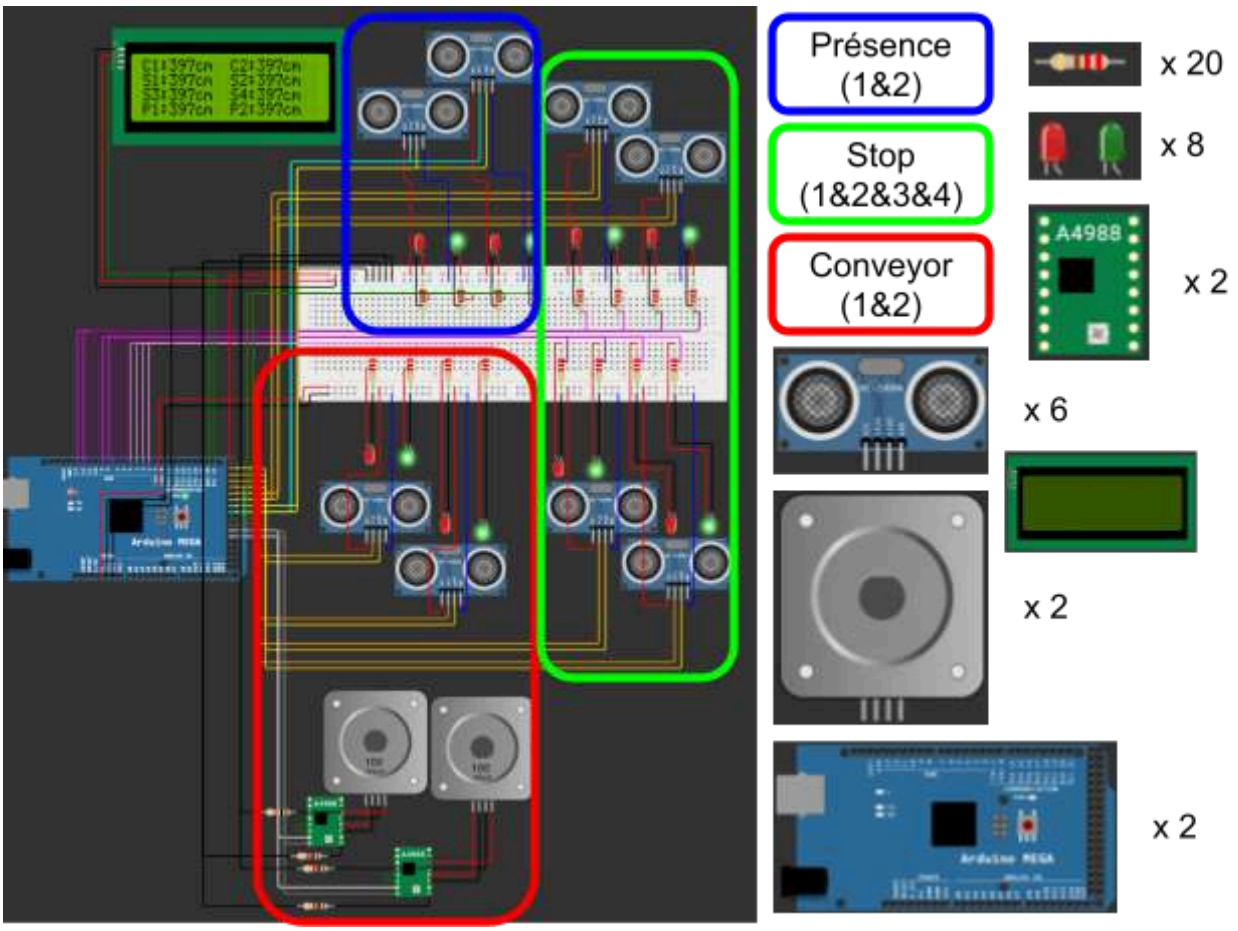


Schéma électrique :




C1:397cm C2:397cm
S1:397cm S2:397cm
S3:397cm S4:397cm
P1:397cm P2:397cm
Arduino MEGA
100
steps
100
steps
A4988
A4988


Présence
(1&2)
Stop
(1&2&3&4)
Conveyor
(1&2)
x 20
x 8
A4988
x 2
x 6
x 2
x 2

After these operations, the part is considered finished and transferred to T4, an exchange zone that interfaces with conveyor 2. This conveyor transports the finished cubes to the right cell. Upon arrival, the parts are first directed to T0, an identification and orientation station (including detection of a red face via sensors/camera) to prepare the correct positioning for the next step. If the conveyor is busy or the robot is unavailable, the parts are temporarily stored in a buffer zone T0', which plays a key role in regulating the flow. The CR cobot then retrieves the cubes from T0 or T0’ and places them in a grid-like palletizing area, according to a defined spatial coordinate system (X+ and Y+ axes). This setup illustrates a typical industrial system combining a sequence of processes, automated transfer, vision-based quality control, and buffer management, with the goal of maintaining a continuous flow while avoiding waiting times between the two robots.
Dans la zone gauche, un cobot (CL) manipule les cubes entre plusieurs positions discrètes (T1, T2, T3) suivant une trajectoire cyclique intégrant des changements d’orientation (90° selon les axes X et Y), ce qui correspond typiquement à des opérations de préparation, de tri ou de reconfiguration de pièces. Une fois prêts, les cubes sont acheminés vers une position de transfert (T4), qui joue un rôle d’interface avec le convoyeur 2. Ce convoyeur assure alors le transport automatique vers la cellule droite. Dans la zone droite, les pièces arrivent d’abord dans un espace tampon (T0’), essentiel pour découpler les dynamiques entre les deux cellules et éviter les ruptures de flux. Le cobot droit (CR) prélève ensuite les cubes depuis ce buffer (via une position T0), puis les distribue soit vers une zone de stockage organisée en grille, soit vers une boucle interne pour d’éventuelles étapes supplémentaires. L’ensemble du système est supervisé par des capteurs et une caméra qui garantissent la détection des pièces, le suivi du flux et la coordination des actions, illustrant ainsi une architecture classique de ligne automatisée avec synchronisation, bufferisation et gestion des flux tirés/poussés.

THEORIE